

Static Site Generators

IN THE AGE OF AI



**Building and Maintaining Content
with AI Agents**

by Polla Fattah
and Collection of LLMs



A COMPREHENSIVE PRACTICAL GUIDE

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Dr. Polla Abdulhamid Fattah

Department of Software and Informatics Engineering, Salahaddin University-Erbil (SUE)

Artificial Intelligence and Innovation Centre (AIIC), UKH

Draft 0.2 • September 2026 • Erbil, Kurdistan Region, Iraq

Table of Contents

Chapter 01	Your First Hugo Website	4
Chapter 02	Write and Publish Content Locally	17
Chapter 03	Organise a Useful Website	32
Chapter 04	Understand the HTML Behind Your Pages	47
Chapter 05	Practical CSS for Your Hugo Site	61
Chapter 06	Track and Recover Your Hugo Site with Git	74
Chapter 07	Publish Your Hugo Site with GitHub Pages	85
Chapter 08	Work with an AI Agent on Your Hugo Site	97
Chapter 09	Give Your Hugo Content a Consistent Structure	108
Chapter 10	Use Hugo Templates to Display Your Content	121
Chapter 11	Build Reusable Hugo Layouts	134
Chapter 12	Build a Resource Directory with JSON	146
Chapter 13	Create and Maintain Content with AI Agents	159
Chapter 14	Check Every Contribution with CI/CD	174
Chapter 15	Help Readers Find and Use Your Content	186
Chapter 16	Publish in Multiple Languages	199
Chapter 17	Add Interactive Features Responsibly	213
Chapter 18	Maintain, Migrate, and Recover	225
Chapter 19	Build Your Own Publishing Project	237

CHAPTER 01

Your First Hugo Website

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Draft 0.2, September 2026

Checked with Hugo 0.150.0 through 0.166.0. Requires Hugo 0.146.0 or later for the template layout convention.

You have something worth sharing: notes from your work, an explanation you have written, a useful collection of resources, or a project you want others to understand. A website gives those materials a place where people can find and read them.

In this chapter, you will make the first small part of that website. You will open a local preview, change an introduction, and watch the page update. You will also make a deliberate mistake and repair it.

The first result is modest: a readable home page headed **My Knowledge Notebook**, with an introduction and links to two sections on the page. Across the book, this notebook will grow into a site with articles, projects, and resources. Later, you will ask an AI agent to help maintain it, review the agent's changes, and publish approved updates.

For now, success means being able to say: **"I know which file contains these words, I can change them, and I can put them back."**

What you will be able to do

By the end of this chapter, you should be able to:

- Start and stop a local Hugo preview.
- Find the file containing the home-page introduction and edit it.
- Explain the roles of the content file, layout, and browser preview.
- Recover from a small editing mistake and preserve a working copy.

No previous HTML, CSS, Git, or programming experience is required. The starter includes a little code for you to copy. You are not expected to understand that code yet.

1.1 A quick picture of what we are building

Imagine a notebook whose pages can be displayed as a website. You write the words in ordinary text files. A layout supplies the page structure, and a stylesheet supplies its appearance. Hugo combines them into files a browser can display.

Hugo is a **static site generator**, often shortened to **SSG**. “Generate” means that it produces the website output from your source material. “Static” describes how that output can be served; it does not mean you can never update the site or add interaction.

For this first exercise, remember three roles:

Role	In our project
You write and edit the content.	A Markdown file contains the introduction.
Hugo builds the page.	It combines that content with the supplied layout.
Your browser displays the result.	You view the page through a local preview address.

Hugo’s official [quick-start guide](#) demonstrates the same basic relationship between a project, its content, and its local preview. Our exercise uses a smaller embedded starter so this chapter can stand alone.

1.2 Get the tools ready

You need a desktop or laptop computer, a browser, a text editor, and Hugo. You can read the chapter on a phone, but the practical steps assume a computer.

A **text editor** saves the actual characters in a file. Use Visual Studio Code, or another editor you already know that saves plain text. A word processor is not suitable for these project files. VS Code is available from its [official download page](#).

A **terminal** is a place where you enter commands. Each command below tells your computer to perform one action. Type the command and press Enter. Do not type the surrounding code fences or a prompt such as `PS>` or `$`.

For this chapter, you do not need a GitHub account, a domain name, or a paid AI subscription.

Install Hugo for your operating system

If Hugo is already installed, go directly to the verification command below.

Windows: Open PowerShell from the Start menu. If Windows Package Manager (`winget`) is available, the following installation command is listed in Hugo’s documentation:

```
winget install Hugo.Hugo.Extended
```

Follow the installer prompts. Then close and reopen PowerShell. If you were using VS Code’s terminal, restart VS Code as well so it can find the newly installed program. If `winget` is unavailable, use the prebuilt-binary instructions in the [official Windows installation guide](#).

The command installs the extended edition. This chapter uses only core features and also works with the standard edition; there is no need to replace a working standard installation solely for this exercise.

macOS: If Homebrew is already installed, open Terminal and run:

```
brew install hugo
```

If Homebrew is not installed, follow the prebuilt-binary route or another supported route in the [official macOS installation guide](#).

Linux: Follow the [official Linux installation guide](#) for your distribution. Check the installed version rather than assuming your distribution's package is recent enough for this exercise.

Check that Hugo can run

In your terminal, enter:

```
hugo version
```

You should see a line identifying Hugo and its version. Extra information about the edition, operating system, or build is normal. Record the version you installed in a separate learning note; it will help if you need to troubleshoot later.

This starter uses the template conventions introduced in **Hugo 0.146.0**. Its build and live-edit exercises were verified with **Hugo 0.150.0 through 0.166.0**. Use version 0.146.0 or later; older distribution packages (such as legacy packages from Debian/Ubuntu `apt`) do not support `all.html` and will fail to build. In Hugo 0.158.0 and later, site language configuration uses `locale` instead of `languageCode` to avoid deprecation warnings. See the [template-system overview](#) for the relevant changes.

If the command is not recognised: stop here and fix the installation. Reopen your terminal first. If that does not help, return to the installation guide for your operating system. Your website files are not the cause of an unrecognised `hugo` command.

1.3 Create your project folder

Using your normal file manager, create a folder named `my-knowledge-site` somewhere convenient, such as inside Documents.

Open VS Code and choose **File** → **Open Folder**, then select that folder. If your editor asks whether you trust the folder, remember that this is the new folder you just created for the exercise.

In the editor's file explorer, create these folders and files. Paths in this table are relative to `my-knowledge-site`.

File	Where it goes	Purpose
<code>hugo.toml</code>	Directly inside the project folder	The site's basic settings
<code>_index.md</code>	Inside a folder named <code>content</code>	The home-page text
<code>all.html</code>	Inside a folder named <code>layouts</code>	The supplied page structure
<code>site.css</code>	Inside <code>static</code> , then a folder named <code>css</code>	The supplied appearance

The resulting paths are `hugo.toml`, `content/_index.md`, `layouts/all.html`, and `static/css/site.css`.

Pay attention to the underscore in `_index.md`. In Hugo, a leading underscore designates a section or branch bundle (such as the home page), whereas `index.md` without an underscore designates an individual standalone leaf article. Also make sure the files are not accidentally named `hugo.toml.txt` or `all.html.txt`. Creating them inside the code editor helps avoid hidden filename-extension problems.

You can either create these four files by hand or obtain them ready-made from the companion starter repository (`ssg-playground`, branch `chapter-01`).

Copy the contents from **Starter files** near the end of this chapter into the four matching files. Copy only what is inside each code block; do not include the backticks. Save every file.

The HTML and CSS are supplied materials. Leave them unchanged for now. We will examine their structure in later chapters. This small starter performs the presentation role that a larger theme would normally provide.

Pause and check: the editor should show the four files at exactly the paths listed above. You do not need to run `hugo new site` or install a separate theme for this embedded starter.

1.4 Open your first preview

In VS Code, choose **Terminal** → **New Terminal**. The integrated terminal normally opens in the project folder. Confirm that you are in the folder containing `hugo.toml`:

- In PowerShell, type `Get-Location` to show the folder and `Get-ChildItem` to list its contents.
- In macOS or Linux terminals, type `pwd` to show the folder and `ls` to list its contents.

If you are in the wrong place, close that terminal, open the correct project folder in the editor, and create a new terminal there.

Now run:

```
hugo server
```

Leave the terminal running. Read its output and find the local address, normally:

```
http://localhost:1313/
```

Enter that address in your browser's address bar (or hold **Ctrl** and click the link directly in the VS Code terminal, or **Command** on macOS). If Hugo reports a different address or port, use the one it reports.

You should see:

- The site name, **My Knowledge Notebook**, near the top.
- A navigation row with **Home**, **My interests**, and **Next steps**.
- The heading **Welcome to my knowledge notebook**.
- A short introduction and two sections underneath it.

The navigation links for My interests and Next steps move within this same page. We will add separate pages later.

This is a local preview. `localhost` refers to the computer you are using. Opening that address on somebody else's computer will not open your website. We will make the site publicly reachable in the publishing chapter.

Hugo's development server watches project files and normally rebuilds and refreshes the preview after changes. The terminal stays occupied while that server runs; this is expected behaviour. [Hugo server documentation](#)

1.5 Make the page yours

Open `content/_index.md` in the editor. Find this sentence:

```
Hello! I am Dana. This is where I collect useful ideas, learning notes, and small projects.
```

Dana is a fictional example. Replace the name and introduction with your own wording. For example:

```
Hello! I am Sara. I use this notebook to explain what I am learning and share useful resources.
```

Save the file with **Ctrl+S** on Windows/Linux or **Command+S** on macOS. Look at your browser. The introduction should change, while the colours and page layout remain the same.

If it does not change, refresh the browser once. Then check that you saved the correct file and that the terminal has not reported an error.

You have now completed the central operation you will repeat throughout the book:

Edit the source, save it, and inspect the result.

Change the heading as well

At the top of the same file, find:

```
title: "Welcome to my knowledge notebook"
```

Change only the text inside the quotation marks:

```
title: "Welcome to Sara's learning space"
```

Save and inspect the page again.

The lines between the two `---` markers form the **front matter** in **YAML** format: a small block of structured metadata about the page. Here, its `title` supplies the large page heading. The writing below the second marker forms the page's main content.

You only need to recognise that distinction today. Chapter 2 develops Markdown, and Chapter 9 explains structured metadata and configuration in detail.

1.6 Understand what changed

You edited the writing without editing the layout. The starter keeps those responsibilities in separate files:

File	What you would change there
<code>content/_index.md</code>	The home-page heading and main text
<code>hugo.toml</code>	The site-wide name and basic settings
<code>layouts/all.html</code>	The structure used to display pages
<code>static/css/site.css</code>	Colours, spacing, typography, and other styling

The name at the top of the site comes from `hugo.toml`. The larger heading inside the article comes from `content/_index.md`. They can be different because they describe different things: the whole site and the current page.

Hugo reads the project and generates the browser-facing output. Continue editing the source files listed above. If you later encounter a generated folder such as `public`, changes made there may be overwritten by another build.

You may notice expressions such as `{{ .Title }}` in the layout. They mark places where Hugo inserts information. You do not need to write template expressions yet; first become comfortable changing content and recognising its effect.

1.7 Practise recovering from a mistake

Before experimenting, save your current work. Then copy the introduction paragraph into a separate temporary note outside the project. This gives you a small, clear recovery reference.

In `content/_index.md`, replace the introduction paragraph with:

```
This paragraph was changed by mistake.
```

Save the file and inspect the preview. The website still works, but its content is wrong.

Return to the editor. Use **Undo** until your previous introduction returns, then save again. If the editor's undo history is unavailable, restore the paragraph from your temporary note and save it.

Check the browser to confirm the original introduction is back.

This distinction will matter when we introduce AI agents: **a page can build successfully and still contain the wrong information**. Seeing a successful build is one check; reading the result is another.

For this exercise, undo and a temporary copy are sufficient. The Git chapter will give you a more dependable record of changes and a way to restore earlier versions.

1.8 Where AI agents will fit

An AI agent can be given access to project files and asked to propose or perform a change. In later chapters, you will set up an agent and control the scope of its work. Today, consider this illustrative task:

Read `content/_index.md`. Propose a clearer version of its introduction using only the information already present. Keep the meaning and first-person voice. Do not change the title, section headings, configuration, layout, or stylesheet. Show the proposed replacement before editing the file.

This instruction identifies the file, the intended improvement, and the limits. If an agent invented a qualification or project, you would reject or correct that addition even if its wording sounded polished.

No agent installation is required for this chapter. You have already practised the human actions that make later agent work inspectable: locating the source, reading changes, previewing the page, and restoring earlier text.

1.9 Try a small independent change

Without changing the layout or stylesheet:

1. Add one interest under **My interests**, following the existing list format.
2. Rewrite the sentence under **Next steps** to describe something you want to share.
3. Save the file and verify both changes in the browser.

Keep the section headings unchanged for this exercise because the starter's navigation links point to them. We will learn how headings and links relate in the next chapter.

Now explain aloud, or in a learning note:

- Which file did you change?
- Which visible parts of the page changed?
- Why did the appearance remain consistent?

If you can answer those questions, you have begun to understand the workflow rather than merely repeat commands.

1.10 Stop, restart, and preserve your work

Click inside the terminal running Hugo and press **Ctrl+C**. This stops the preview server; it does not delete your files. On macOS, the terminal interruption is also Ctrl+C, not Command+C.

The browser may continue to show the last page it loaded. That does not mean the preview server is still running. Refreshing or navigating after stopping the server may show a connection error.

To resume, open the project folder and run:

```
hugo server
```

Open the reported address again.

For a simple end-of-chapter checkpoint, stop the server, save all files, and use your file manager to copy the entire `my-knowledge-site` folder to a sibling folder named `my-knowledge-site-ch01-backup`. Do not put the backup inside the active project. Continue working in the original folder in Chapter 2.

Completion check

- ☐ I can run `hugo version` and identify my installed version.
- ☐ I can open the local website preview.
- ☐ I can change the introduction and page title in `content/_index.md`.

- ☐ I can distinguish content, configuration, layout, and styling.
- ☐ I have restored an intentionally incorrect paragraph.
- ☐ I can stop and restart the preview.
- ☐ I have preserved a working copy of the project.

In Chapter 2, we will add a separate article and learn the Markdown needed to structure it, link it, and illustrate it.

Troubleshooting when you need it

What you see	What to check or do
<code>hugo</code> is not recognised or not found.	Reopen the terminal or editor after installation. Verify the installation and executable path using the official operating-system guide.
Hugo cannot locate the configuration or project.	Check that the terminal is in the folder containing <code>hugo.toml</code> , rather than its parent or the <code>content</code> folder.
A configuration parsing error appears.	Compare <code>hugo.toml</code> with the supplied starter. Keep straight quotation marks, matching pairs, and the equals signs.
A front-matter parsing error appears.	Check the two <code>---</code> markers and the matching quotation marks around the title in <code>content/_index.md</code> .
The browser shows a connection error.	Check that <code>hugo server</code> is still running and that you entered its reported address, including <code>http://</code> and the port.
The terminal reports that port 1313 is already in use.	Stop an earlier preview with Ctrl+C . On Windows, closing a terminal tab without stopping Hugo may leave an orphaned <code>hugo.exe</code> running; terminate it in Task Manager or use <code>hugo server --port 1314</code> .
The home page is missing or Hugo warns about a missing layout.	Check the spelling and placement of <code>layouts/all.html</code> , confirm that its code was saved, and check the Hugo version.
The page appears without the supplied styling.	Check that <code>site.css</code> is inside <code>static/css</code> , and compare the stylesheet link in <code>layouts/all.html</code> with the starter.

What you see	What to check or do
Your change does not appear.	Save the file, check for terminal errors, refresh the preview, and confirm that the editor and server are using the same project folder.
You see <code>---</code> or template expressions as literal text.	Open the address from <code>hugo server</code> , rather than double-clicking a source file in the file manager. Check that code was copied into the right files.

When an error occurs, change one thing at a time and try again. Keep the error message: it provides useful evidence for troubleshooting and for later work with an agent.

Starter files

These four blocks contain the complete starter for this draft. Copy each into the file named above it. They are provided as project materials; studying HTML, CSS, and template syntax is deferred to later chapters.

File 1: `hugo.toml`

```
baseUrl = 'https://example.org/'
locale = 'en'
title = 'My Knowledge Notebook'
```

`https://example.org/` is a placeholder, not your published website. In Hugo v0.158.0+, `locale = 'en'` is the standard language configuration (older Hugo versions used `languageCode = 'en'`). Hugo's local server supplies the preview address. We will configure the real public address when publishing.

File 2: `content/_index.md`

```

---
title: "Welcome to my knowledge notebook"
---

Hello! I am Dana. This is where I collect useful ideas, learning notes, and small projects.

## My interests

- Learning new things
- Explaining useful ideas
- Sharing small projects

## Next steps

I will use this notebook to publish my first article and organise useful resources.

```

File 3: `layouts/all.html`

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{{ .Title }} | {{ .Site.Title }}</title>
  <link rel="stylesheet" href="{{ "css/site.css" | relURL }}">
</head>
<body>
  <a class="skip-link" href="#main">Skip to content</a>
  <header>
    <p class="site-name">{{ .Site.Title }}</p>
    <nav aria-label="Main navigation">
      <a href="{{ "" | relURL }}">Home</a>
      <a href="{{ "" | relURL }}#my-interests">My interests</a>
      <a href="{{ "" | relURL }}#next-steps">Next steps</a>
    </nav>
  </header>
  <main id="main" tabindex="-1">
    <article>
      <h1>{{ .Title }}</h1>
      {{ .Content }}
    </article>
  </main>
  <footer>A place to learn, explain, and share.</footer>
</body>
</html>

```

File 4: static/css/site.css

```
* { box-sizing: border-box; }
body {
  margin: 0;
  background: #f5f3ed;
  color: #263238;
  font-family: system-ui, sans-serif;
  line-height: 1.7;
}
header, main, footer {
  width: min(100% - 2rem, 48rem);
  margin-inline: auto;
}
header { padding-block: 2rem 1rem; }
.site-name { font-size: 1.25rem; font-weight: 700; }
nav { display: flex; flex-wrap: wrap; gap: 1rem; }
a { color: #005b66; text-decoration: underline; text-decoration-color: #005b66; text-decoration-offset: 0.2em; }
main {
  padding: clamp(1rem, 4vw, 2rem);
  background: #ffffff;
  border: 1px solid #d5d9d8;
  border-radius: 0.75rem;
  overflow-wrap: anywhere;
}
h1 { font-size: clamp(1.7rem, 5vw, 2.5rem); line-height: 1.2; }
h2 { margin-top: 2rem; line-height: 1.3; }
footer { padding-block: 1.5rem; }
a:focus-visible { outline: 3px solid #005b66; outline-offset: 4px; }
.skip-link { position: absolute; top: -10rem; left: 1rem; }
.skip-link:focus {
  top: 0.5rem;
  padding: 0.5rem 1rem;
  background: #ffffff;
}
```

The starter uses `all.html` as a general HTML layout, following Hugo’s newer template conventions. It intentionally keeps presentation local and small; no external theme download, Git submodule, JavaScript package, or custom font is required for this exercise. Later chapters will develop the layout into reusable parts.

Editorial note for the author (remove before publication)

This is a self-contained chapter draft for review. To honour the request for one Markdown file, the starter is embedded in the chapter instead of referring to an unavailable companion download. This is a provisional adaptation of the blueprint’s packaged starter-theme approach; it does not select a third-party theme for the whole book.

Before publication, choose whether to package this small starter or adapt the exercise to the final theme. A downloadable, versioned starter should make copying four files unnecessary in the main reading path; the embedded source can remain as a fallback. Supply matching screenshots from that final starter.

Documentation was consulted on 17 September 2026. The four starter blocks were extracted directly from this Markdown file and tested with Hugo 0.150.0, standard edition, on Linux amd64. The production build passed with warnings treated as failures. Checks confirmed the home-page content, navigation target IDs, and copied stylesheet. A running local server reflected a changed introduction, an intentional mistaken edit, and restoration of the corrected text. These checks verified server responses; they did not constitute a visual browser audit or a beginner usability trial.

Package-manager commands install the versions available from their respective repositories; they do not pin the final book's software baseline. Before publication, select a supported release baseline and rerun the exercises against it. A representative beginner trial, visual browser inspection, and Windows/macOS installation walkthroughs remain necessary before describing this chapter as publication-ready.

CHAPTER 02

Write and Publish Content Locally

*Static Site Generators in the Age of AI**Building and Maintaining Content with AI Agents**Draft 0.2, September 2026**Checked with Hugo 0.150.0 through 0.166.0 (standard and extended editions).*

In Chapter 1, you changed a home page. Now you will give your website something more substantial to read: its first article.

The article will describe a small thing you have learned. You will give it sections, add links and a screenshot, and make it reachable from the home page. Along the way, you will learn the Markdown needed to make a piece of writing understandable both as a text file and as a web page.

Here, **publish locally** means making the article appear in the site you preview on your computer. You will not upload it to a public host in this chapter.

What you will be able to do

By the end, you should be able to:

- Create an article in its own folder and control whether it is a draft.
- Structure its text with headings, paragraphs, lists, emphasis, and code formatting.
- Add working links and an image with meaningful alternative text.
- Check the rendered article, repair a broken image reference, and make the article available in a normal local preview.

2.1 Start from your working website

Open the original `my-knowledge-site` project from Chapter 1. Keep your personalised introduction. Its relevant files are:

Existing file	Role
<code>hugo.toml</code>	Site settings
<code>content/_index.md</code>	Home-page content
<code>layouts/all.html</code>	The supplied general layout

Existing file	Role
<code>static/css/site.css</code>	The supplied styling

If you need to rebuild that starting point, Chapter 1 includes all four starter files, or you can switch to the `chapter-01` branch in the companion `ssg-playground` repository. No new software, theme, or account is required here.

Open a terminal in the folder containing `hugo.toml`. If a preview server is still running, stop it with **Ctrl+C**. Then start it with one additional option:

```
hugo server -D
```

The `-D` option (short for `--buildDrafts`) tells Hugo to include pages marked as drafts. Leave this terminal running and open the address it reports, normally `http://localhost:1313/`.

You should see the home page you already know. We will keep using the same editor, project, and browser while adding one article.

2.2 Make an article before studying the syntax

Inside `content`, create a folder named `articles`. Inside it, create another folder named `first-learning-note`. In that folder, create a file named `index.md`.

Its complete path, relative to the project, is:

```
content/articles/first-learning-note/index.md
```

Copy this small article into that file and save it:

```
---  
title: "My first learning note"  
draft: true  
---
```

Today I learned how to change a page in my knowledge notebook.

What I tried

I edited my introduction, saved the file, and checked the local preview.

What I learned

The words on the page come from a content file that I can edit.

Open the article directly in your browser:

```
http://localhost:1313/articles/first-learning-note/
```

Use your server's actual port if it differs from 1313. Include the final slash.

You should see the article title and two sections, inside the same styled layout as your home page. The title is supplied by the front matter; the body comes from the writing below it.

Your home page does not yet link to this article. That is expected. The Chapter 1 starter displays page content but does not automatically create an article directory. We will add an explicit link shortly and develop wider navigation in Chapter 3.

First checkpoint: you have created a second page without copying the HTML layout or stylesheet. Hugo has used the existing layout to display the new content.

2.3 Understand the file you just created

The top of the file contains two pieces of metadata:

Field	Meaning in this exercise
<code>title</code>	The title displayed by our layout
<code>draft: true</code>	Exclude the article from a normal build; include it when draft rendering is explicitly enabled

Keep `true` and `false` unquoted. They represent yes/no values rather than ordinary text. The quotation marks around the title enclose a text value.

Hugo supports several front-matter formats. We are using YAML between `---` markers. For now, learn these two fields; you do not need a complete configuration-language lesson. [Hugo front-matter documentation](#)

Below the closing marker is the article body, written in Markdown. Markdown uses visible characters such as `##` and `-` to describe structure. Hugo turns that structure into HTML for the browser.

Why this file is called `index.md`

The article has a folder of its own so its text and image can stay together. Hugo calls a folder containing `index.md` and associated resources a **leaf page bundle**. The home page uses `_index.md`, with an underscore, because it is a different kind of page. Preserve both filenames as shown. [Hugo page bundles](#)

Source location	Address in this project's local preview
<code>content/_index.md</code>	<code>/</code>
<code>content/articles/first-learning-note/index.md</code>	<code>/articles/first-learning-note/</code>

The article's folder name supplies the last part of its address in this setup. Changing its displayed title does not change that folder name. Later configuration can alter URL behaviour; this is the default arrangement used by our starter.

2.4 Give the writing a clear structure

Continue editing the same article. Add one small feature at a time, save, and check the browser.

Paragraphs and headings

A blank line separates paragraphs:

```
I started with a short introduction.  
  
Then I changed the words and checked the preview.
```

Pressing Enter once inside a paragraph usually does not create a new paragraph in the rendered page. Use a blank line when you mean to start a new thought.

In our layout, the front-matter title already becomes the main page heading. Start body sections with `##`, and use `###` for a subsection:

```
## What I tried  
  
I changed the introduction on my home page.  
  
### How I checked it  
  
I saved the file and read the updated text in the browser.
```

Use heading levels to express the relationship between ideas. Do not choose `###` just because you prefer a smaller font. Styling comes later.

Emphasis and short code references

Use a pair of double asterisks for a short piece of important text, and a pair of single asterisks for emphasis:

```
My rule is **change one thing at a time** and check it *before continuing*.
```

Use backticks around a filename or short command:

```
I edited `content/_index.md` and used `hugo server -D` to preview drafts.
```

The backticks make technical text easier to distinguish from the surrounding sentence. Displaying a command in an article does not execute it.

Lists

Use a bulleted list for related items:

- A content file contains my writing.
- A layout supplies the page structure.
- A stylesheet controls the appearance.

Use a numbered list when the order matters:

1. Change a sentence.
2. Save the file.
3. Inspect the preview.

Keep blank lines before and after these blocks. That makes the source easier to read and helps avoid accidental formatting interactions.

Hugo's Markdown support includes features beyond basic Markdown. Different publishing tools can support different extensions, so check rendered output when moving content between systems. [Hugo content formats](#)

2.5 Add a useful link

A Markdown link has readable text in square brackets and a destination in parentheses:

```
I can consult the [Hugo documentation](https://gohugo.io/documentation/) when I need help.
```

Add this sentence to your article, save, and click the rendered link. The text tells the reader what the destination contains. "Click here" would provide less context.

For a link back to this site's home page, add:

```
[Return to my home page](../../..)
```

From `/articles/first-learning-note/`, the first `../` goes up to `/articles/`, and the second reaches the site's home location. These are **web-address relationships**, not instructions to browse folders on your computer.

This link is deliberately relative to the article's address. It also remains inside the site when the same folder structure is hosted under a prefix such as `/my-knowledge-site/`. Moving the article to a different depth would require reviewing the link. Later we will introduce Hugo's tools for managing links as sites grow.

Link to a section on the same page

Append this section to your article:

```
## My next step
```

```
I will write another short note about something I can explain clearly.
```

Near the article's opening paragraph, add:

```
[Jump to my next step](#my-next-step)
```

With this starter, Hugo automatically gives that heading the identifier `my-next-step` by converting the text to lowercase, replacing spaces with hyphens, and removing punctuation. Click the link and look at the browser address: it should end in `#my-next-step`. On a short page, the visual movement may be slight.

This also explains Chapter 1's navigation. Its My interests and Next steps links target specific home-page headings (`#my-interests` and `#next-steps`). Changing a heading can change its generated identifier, so check links that point to it.

2.6 Illustrate the article with your own screenshot

Open your site's home page. Use your operating system's screen-capture tool to capture just the relevant page area. Save the image as a **PNG file** named `notebook-preview.png`.

Web asset best practice: Never include spaces, uppercase letters, or special characters in web image filenames (use `notebook-preview.png`, avoid `My Screenshot.png`). Spaces in filenames become `%20` in URLs and lead to broken links.

If the capture tool puts the image on the clipboard, paste it into a basic image editor and save or export it as PNG. Renaming a JPEG extension to `.png` does not convert its format. Use the tool's actual Save or

Export option. (If using the companion `ssg-playground` repository on branch `chapter-02`, a sample image is already included beside the article).

Place the file next to the article:

```
content/articles/first-learning-note/notebook-preview.png
```

The screenshot is a reader-created asset, so this single chapter file needs no separate image download. Capture your own example site and avoid including unrelated tabs, notifications, or private material in the image.

Add the following to the article after the section explaining what you learned:

```
## My notebook in the browser
```

```
![The notebook home page showing its introduction, interests, and next steps](notebook-preview.png)
```

```
*My home page after editing the introduction. Screenshot by the author.*
```

Save and inspect the article. The exclamation mark makes this an image rather than a text link. The path names a file in the same page bundle. Do not write `content/articles/...` inside these parentheses: that is a source-file location, not the image's browser address.

The words inside the square brackets are **alternative text**. They describe the useful information in the image for someone who cannot see it. Adjust the wording if your screenshot shows something different. The italic sentence is a visible caption; it serves a different purpose from alternative text.

If you use another person's image in future, establish permission or an appropriate licence and retain the required attribution. Finding an image online does not by itself grant permission to republish it.

Keep the image within the page

Chapter 1's small stylesheet did not include image sizing or code block styling. Open `static/css/site.css` and add these rules at the end, outside its existing braces:

```
article img {  
  display: block;  
  max-width: 100%;  
  height: auto;  
  border-radius: 0.5rem;  
  margin-block: 1rem;  
}  
  
article pre {  
  max-width: 100%;  
  overflow-x: auto;  
  background: #eae8e1;  
  padding: 0.75rem 1rem;  
  border-radius: 0.5rem;  
}
```

Save and refresh the preview. The first rule lets large images shrink to the available width while preserving their proportions. The second lets long code samples scroll within their own area with distinct background styling. Resize the browser to check the result. These are supplied styling adjustments; Chapter 5 will explain CSS properly.

2.7 Display a command without running it

Sometimes a learning note needs to show several lines exactly. A **fenced code block** begins and ends with three backticks. The word after the opening backticks identifies the content type.

Copy the following three-line sample into the body of your article:

```
```text  
hugo server -D
```
```

Copy the inner three-backtick opening line, the command, and the closing line; the outer frame is only how this book displays the example. The article will show the command as a code sample. It will not start another server.

If all the writing after a code block appears as code, look for a missing closing fence. You can close a server terminal with Ctrl+C; closing a Markdown code block requires the matching backticks in the file. They solve different problems.

Optional: a small comparison table

If your note needs a simple comparison, try:

| Action | What I check |
|------------------|---|
| --- | --- |
| Edit a paragraph | The meaning is still correct. |
| Add a link | The intended page opens. |
| Add an image | It loads and has useful alternative text. |

The separator row is part of the syntax. Your current CSS may display the table with minimal styling. That is acceptable for this exercise: the goal is to recognise a structured comparison. Tables are optional here and are not required for the completed article.

2.8 Test and repair a broken image reference

Before changing anything, confirm that your image currently loads. Then temporarily change only the image filename in the Markdown:

```
![The notebook home page showing its introduction, interests, and next steps](notebook-preview-missing.png)
```

Save and refresh the article. The image should fail to load. Depending on your browser, you may see its alternative text or a broken-image indicator. Hugo may still report a successful build: the ordinary Markdown image reference in this starter does not guarantee that the destination exists.

Repair the reference:

1. Look inside the article folder and read the actual filename.
2. Restore `notebook-preview.png` in the Markdown, matching spelling and letter case.
3. Save and refresh the article.
4. Confirm that the image appears again.

You have practised checking the source against the output. Keep that habit when an AI agent proposes filenames or links. A plausible-looking reference is not evidence that its destination exists.

2.9 Make the article ready and link it from the home page

Read the article as a visitor would. Check its meaning, heading order, links, image, and alternative text. Replace the sample sentences with accurate statements about your own work where appropriate.

When it is ready, change this front-matter line:

```
draft: false
```

Save the file. In the terminal, stop the current server with **Ctrl+C**, then restart without the draft option:

```
hugo server
```

Open the article's address again. It should still appear. You have confirmed that the article no longer depends on a draft-inclusive preview.

If it disappears, check whether `draft` is still `true` in the correct file. Do not immediately add `-D` back and assume the issue is solved; that would hide the distinction you are trying to verify.

The draft field is a publishing control, not an access-control mechanism. The source remains readable to anyone with access to the project, and a draft-inclusive deployment would include the draft. Never use this flag to protect confidential writing.

Link the article from the home page

Open `content/_index.md`. Leave your introduction, interests, and Next steps section intact. Add this new section at the end of the body:

```
## Latest writing

- [My first learning note](articles/first-learning-note/)
```

Save, visit the home page, and click the new link. From the home page, `articles/first-learning-note/` points to the article's published path. There is no `content/` prefix and no `index.md` in the destination.

Now click the article's **Return to my home page** link. Also try the shared navigation at the top: Home opens the home page, while My interests and Next steps return to the corresponding home-page sections. They do not point to sections inside the article.

This modest manual link is enough for two pages. Do not worry if visiting `/articles/` itself shows a heading without an article list; our starter has no automatic listing loop yet. We will improve organisation next and build reusable listings in the template chapters.

Second checkpoint: a visitor can now open the home page, follow a link to a reviewed article, see its image, and return home.

2.10 Try an independent variation and save your checkpoint

Make one more improvement to this article:

1. Add a subsection describing one difficulty you encountered and how you resolved it.
2. Include either a numbered procedure or a short code sample where it helps the explanation.
3. Check the article in the normal preview without `-D`.

4. Follow the home-page link and both article links again.

The new writing should report what actually happened. Do not turn an illustrative example into an invented personal experience.

If you already use a browser chatbot, an optional writing-only exercise is to ask it to suggest clearer headings for a paragraph you supply. Compare its suggestions with the original meaning before changing your file. An account or agent setup is not required for this chapter; direct project access comes later.

Explain the result to yourself

- Why does this article use `index.md` while the home page uses `_index.md`?
- What changes when you remove `-D` from the server command?
- Why does the image reference contain only its filename?
- Which link would need review if you moved the article deeper into the URL structure?

You should be able to answer using this project. Memorising every Markdown feature is unnecessary.

Save your Chapter 2 checkpoint

Stop the preview with Ctrl+C and save all edited files. Copy the project folder to a sibling folder named `my-knowledge-site-ch02-backup`, outside the active project. Continue working in the original folder in Chapter 3.

Your changes should be limited to:

| File | Change |
|--|--|
| <code>content/articles/first-learning-note/index.md</code> | New article, ending with <code>draft: false</code> |
| <code>content/articles/first-learning-note/notebook-preview.png</code> | Your screenshot |
| <code>content/_index.md</code> | Added Latest writing section |
| <code>static/css/site.css</code> | Added image and code-block sizing rules |

The configuration and layout from Chapter 1 remain sufficient for this exercise. Generated files may also change when Hugo runs; they are output, not additional authoring tasks.

Completion check

- ☐ My article opens in the ordinary `hugo server` preview.
- ☐ Its front-matter title supplies the main heading, and body sections use appropriate heading levels.
- ☐ Its links open the intended destinations.
- ☐ The screenshot is beside `index.md` and displays correctly.
- ☐ Its alternative text describes what the screenshot actually shows.
- ☐ I repaired the intentionally broken image reference.
- ☐ The home page links to the article without replacing my earlier content.
- ☐ I have saved a working Chapter 2 checkpoint.

Chapter 3 will develop this two-page beginning into a clearer website structure, with pages, sections, and navigation chosen around the visitor's needs.

Troubleshooting when you need it

| Problem | Likely cause and next check |
|--|--|
| The new article returns a 404. | Check the exact path, that the file is named <code>index.md</code> , that it is saved, and whether a draft-inclusive preview is needed. |
| The home page works but <code>/articles/</code> shows no list. | The minimal layout does not generate an automatic list. Open the article's full address or use the manual home-page link. |
| The article disappears after restarting the preview. | You may have removed <code>-D</code> while <code>draft</code> remains <code>true</code> . Review the article and change the flag when ready. |
| The page title appears twice. | Remove an extra <code>#</code> title from the body; this layout already renders the front-matter title. |
| The image does not load. | Check the actual filename, extension, letter case, and placement beside <code>index.md</code> . Ensure the saved file really is a PNG. |
| The image extends beyond the article. | Confirm the added CSS rules are outside existing braces, save the stylesheet, and refresh. |
| A link opens a path containing <code>content/</code> . | You used a source path in place of a website address. Compare with the chapter's working link examples. |

| Problem | Likely cause and next check |
|---|--|
| A section link does not jump to the intended heading. | Check whether the heading changed and whether the fragment still matches its generated identifier. |
| A heading displays its hash characters. | Use a space after <code>##</code> and make sure the line is not inside a code block or accidentally indented. |
| Much of the article appears as code. | Look for an unclosed three-backtick fence earlier in the file. |
| A pasted table shows pipe characters. | Check its separator row and blank lines, and verify the result in Hugo rather than assuming every Markdown viewer behaves identically. |

Completed article for comparison

The following is a complete reference version of `content/articles/first-learning-note/index.md`. It assumes your screenshot is saved beside it. Preserve your own accurate wording if you have personalised the exercise. Copy the content inside the outer four-backtick display block; retain the inner three-backtick code block as part of the article.

```
---
title: "My first learning note"
draft: false
---
```

Today I learned how to change a page in my knowledge notebook.

[\[Jump to my next step\]\(#my-next-step\)](#)

What I tried

I edited my introduction, saved the file, and checked the local preview.

How I checked it

1. Change a sentence.
2. Save the file.
3. Inspect the preview.

What I learned

The words on the page come from a content file that I can edit.

My rule is ****change one thing at a time**** and check it ***before continuing***.

- A content file contains my writing.
- A layout supplies the page structure.
- A stylesheet controls the appearance.

I edited ``content/_index.md``. To include drafts in the local preview, I used:

```
```text
hugo server -D
```
```

I can consult the [\[Hugo documentation\]\(https://gohugo.io/documentation/\)](https://gohugo.io/documentation/) when I need help.

My notebook in the browser

![The notebook home page showing its introduction, interests, and next steps](notebook-preview.png)

My home page after editing the introduction. Screenshot by the author.

My next step

I will write another short note about something I can explain clearly.

[\[Return to my home page\]\(../../..\)](#)

Editorial note for the author (remove before publication)

This draft continues the exact four-file starter embedded in Chapter 1. It does not assume a third-party theme, an automated article listing, a content-generation archetype, a Git repository, or an installed AI agent. The shared layout already renders individual pages. The added CSS supports screenshots and code blocks without requiring a separate styling lesson.

The screenshot is deliberately created by the reader, allowing this deliverable to remain one Markdown file. A companion package should eventually provide a reusable example image and matching screenshots of the finished page. The complete reference article provides a recovery point; a packaged Chapter 1 checkpoint is still desirable for readers who need to restart the project.

The chapter teaches links relative to the rendered URL under the starter's existing configuration. Template helpers and content-aware link resolution should be introduced later without retroactively claiming that ordinary relative links automatically survive arbitrary moves. Repeat the subdirectory checks when the actual publishing workflow is introduced.

Validation on 17 September 2026 reconstructed the starter from the current saved Chapter 1 and extracted the article examples and CSS directly from this chapter. Hugo 0.150.0, standard edition, on Linux amd64 built the completed project with warnings treated as failures. Both domain-root and `/my-knowledge-site/` base URLs passed checks for local links, heading targets, stylesheet paths, and the image resource. Checks also confirmed one main heading, rendered emphasis, and the image's alternative text.

Separate builds confirmed draft exclusion without `-D` and inclusion with it. A running local server served the draft article and image, returned 404 for the intentionally broken image path, and served the repaired image. Restarting without `-D` after setting `draft: false` retained the article. A small generated PNG was used only as a path-resolution test fixture; it was not a screenshot and is not part of the deliverable. Actual screenshot capture, visual browser review, cross-platform walkthroughs, and a representative beginner trial remain unverified and are required before publication.

CHAPTER 03

Organise a Useful Website

*Static Site Generators in the Age of AI**Building and Maintaining Content with AI Agents*

Your website now has a home page and an article. You know how to edit both, but a new visitor does not know where your files are. They need clear routes through the information you have published.

In this chapter, you will add an About page, give Articles a useful landing page, introduce a Projects section, and collect a few resources. Then you will replace the small home-page navigation with links that work across the website.

The aim is a site whose organisation makes sense to someone who has never seen your project folder. Every destination we add will contain something useful, even if it is only a short explanation and one carefully chosen link.

What you will be able to do

By the end, you should be able to:

- Choose pages and section names around a visitor's needs.
- Distinguish an individual page from a section landing page and use the appropriate index filename.
- Connect the home page, sections, and individual items with working links and shared navigation.
- Check a visitor's route through the site and recover from a change that breaks an address.

3.1 Start with the website you already have

Continue in the original `my-knowledge-site` folder. Your Chapter 2 article should be marked `draft: false`, have its screenshot beside it, and be linked from the home page.

The files we depend on are:

| File | Existing purpose |
|--|---|
| <code>content/_index.md</code> | Personal introduction and Latest writing link |
| <code>content/articles/first-learning-note/index.md</code> | First article |

| File | Existing purpose |
|--|---|
| <code>content/articles/first-learning-note/notebook-preview.png</code> | Article screenshot |
| <code>layouts/all.html</code> | Shared page structure and navigation |
| <code>static/css/site.css</code> | Styling, including Chapter 2's image sizing |
| <code>hugo.toml</code> | Basic site settings |

If you need to rebuild that starting point, switch to the `chapter-02` branch in the companion `ssg-playground` repository, or review the Chapter 2 completed files.

Open a terminal in the folder containing `hugo.toml`:

```
hugo server
```

Open the address reported in the terminal, normally `http://localhost:1313/`. Follow the Latest writing link and confirm the article still works. Return to the home page.

We will use normal, non-draft pages for the short examples in this chapter. They are local practice content and must be reviewed before any later public deployment. No new installation is needed.

3.2 Add an About page first

A new visitor may want to know what your notebook is for. Give them a short answer.

Inside `content`, create an `about` folder and a file named `index.md` inside it. Its path is `content/about/index.md`.

Copy this complete file, then replace the introduction with accurate wording about yourself or your work:

```
---
title: "About this notebook"
draft: false
---
```

This notebook collects things I am learning and projects I am developing.

What you can find here

I write short explanations, describe small projects, and collect resources that help me learn.

Where to begin

Read [\[my first learning note\]\(../articles/first-learning-note/\)](#) for an example of how I record what I have tried.

Save and open:

```
http://localhost:1313/about/
```

Use your actual server port if it is different. Check the title, read the text, and follow the link to the article.

You have added a new destination using skills from Chapter 2. It does not need a new stylesheet or a new HTML layout.

First checkpoint: a visitor who opens the About page can understand the site's purpose and reach an example of its writing.

3.3 Decide what belongs where

Before adding more folders, write a short answer to these questions in a learning note outside the project:

1. Who is this website primarily for?
2. What should that person be able to find or do?
3. Which existing content answers those needs?

For our example, the primary visitor is a fellow learner who wants to understand an idea, inspect a small project, or find a useful reference. That gives us a manageable structure:

| Visitor's question | Destination | What belongs there |
|---|-------------|--|
| What is this website, and where should I begin? | Home | A brief introduction and selected starting points |
| Who is writing, and what is the purpose? | About | Relevant background and the notebook's scope |
| What can I learn here? | Articles | Explanations and learning notes |
| What has the author been working on? | Projects | Descriptions of work, its status, and related evidence |
| Where can I read more? | Resources | A small collection of links with explanations |

These names are a starting point, not a universal formula. A course website might use Lessons and Exercises; a research group might use Research and Publications. Choose words your intended visitors recognise.

Avoid adding a menu item simply because you might write something for it later. A smaller site with useful destinations is easier to understand than a large collection of empty pages.

The organisation of information and the routes connecting it are often called **information architecture**. At this scale, the table above is enough planning. You do not need a complex diagram to begin.

3.4 Make Articles a useful landing page

Your article already exists under `content/articles/`. In Chapter 2, visiting `/articles/` did not show an article list because our minimal layout has no listing loop.

We can make that address useful with ordinary Markdown. Create `content/articles/_index.md`. Notice the underscore: this file describes the section that contains articles, rather than a single article at the end of the structure.

Use this complete content:

```

---
title: "Articles"
draft: false
---

Short notes about ideas I have tried and things I am learning.

## Start reading

- [My first learning note](first-learning-note/): Editing a page, checking the result, and recording what changed.

```

Save and open:

```
http://localhost:1313/articles/
```

The page now explains what the section contains and provides a link to the article. Click it.

Hugo recognises top-level content directories as sections. Adding `_index.md` gives this section its own title and introductory content. Hugo also supports nested sections, but we will keep our structure shallow. [Hugo sections](#)

The list you just wrote is **manual**. Adding another article file will not automatically add a bullet here. For now, maintain the list yourself. The template chapters will show how to generate such lists from content.

Keep the two index filenames distinct

| File | Meaning in this project |
|--|---|
| <code>content/_index.md</code> | Home-page content |
| <code>content/articles/_index.md</code> | Articles section introduction and manual list |
| <code>content/articles/first-learning-note/index.md</code> | One article and its associated image bundle |
| <code>content/about/index.md</code> | One standalone About page |

Do not rename the section's `_index.md` to `index.md`. A leaf bundle built around `index.md` has different rules for the files beneath it; the two names are not interchangeable. [Hugo page-bundle documentation](#)

3.5 Add Projects and a Resources page

The website itself is a small project you are developing. You can describe its purpose and current state without claiming that it is finished.

Create a `projects` folder inside `content`. Then create the two files below.

Section page: `content/projects/_index.md`

```
---
title: "Projects"
draft: false
---
```

Small projects I am developing, with notes about their purpose and progress.

Current work

- [\[My knowledge notebook\]\(Learning-notebook/\)](#): A website for organising explanations, project notes, and useful resources.

Project page: `content/projects/learning-notebook/index.md`

```
---
title: "My knowledge notebook"
draft: false
---
```

Purpose

I am building a small website where readers can find my learning notes and follow my projects.

Current status

This is a work in progress. I have a home page and an article, and I am improving the site's organisation.

What I have learned

I can edit Markdown, preview a page, and check links and images.

Read [\[my first learning note\]\(../articles/first-learning-note/\)](#) for an example.

[\[Back to Projects\]\(../\)](#)

Create the `learning-notebook` folder before saving its `index.md`. Save both files, open `/projects/`, follow its project link, and use Back to Projects to return.

The project description is intentionally short. Change any sentence that does not match what you have actually done. A statement such as "work in progress" helps readers interpret the page accurately.

Avoid making a copy of the first article inside Projects. The article explains an experience; the project page describes the wider work and links to that experience. Keeping one article avoids maintaining two copies of the same text.

A small Resources page

For the moment, Resources can be one page containing a few references. It does not need a section full of individual resource pages.

Create `content/resources/index.md`:

```
---
title: "Resources"
draft: false
---
```

References that support the work recorded in this notebook.

Website publishing

- [Hugo documentation](https://gohugo.io/documentation/): The official reference for Hugo configuration, content, and templates.
- [Hugo page bundles](https://gohugo.io/content-management/page-bundles/): An explanation of grouping a page with related resources.

Examples from this notebook

- [My first learning note](../articles/first-learning-note/): A practical record of editing and checking content.
- [My knowledge notebook project](../projects/learning-notebook/): The purpose and current state of this website.

Save and open `/resources/`. Follow each link. The external links leave your site; the internal examples stay within it.

The explanation after each link tells the visitor why it is included. That is more useful than collecting many unexplained URLs. When maintaining a resource list, keep links you have checked and can describe accurately.

3.6 Add navigation and clear starting points

The navigation supplied in Chapter 1 links to Home and two headings on the home page. Our site now needs routes to its main destinations.

Open `layouts/all.html`. Find the existing block beginning with `<nav aria-label="Main navigation">` and ending with `</nav>`.

Replace **that block only** with:

```
<nav aria-label="Main navigation">
  <a href="{{ "" | relURL }}">Home</a>
  <a href="{{ "about/" | relURL }}">About</a>
  <a href="{{ "articles/" | relURL }}">Articles</a>
  <a href="{{ "projects/" | relURL }}">Projects</a>
  <a href="{{ "resources/" | relURL }}">Resources</a>
</nav>
```

Save the file and refresh the preview. The new navigation should appear on the home page, article, and new pages because they all use the shared layout.

Do not replace the whole layout with this fragment. Keep the surrounding document, the stylesheet link, the Skip to content link, the main content area, and the footer.

You need only recognise two things today: the words between the link tags are the visible labels, and the quoted paths inside the template expressions identify the destinations. `relURL` makes those links relative to the configured site base, including a hosting subdirectory. The paths supplied to it here deliberately have no leading slash. [Hugo relative-URL function](#)

We will explain the HTML in Chapter 4 and the template expressions in the later templating chapters. This supplied replacement is a small, controlled change rather than a requirement to learn the full template language now.

Three distinct things that work together

Thing	What it does	What it does not do by itself
Content files and folders	Supply pages and their organisation	Decide which destinations appear in the navigation
A navigation link	Offers a route to a destination	Create the destination page
A shared layout	Renders repeated page structure	Automatically infer the best navigation for visitors

Hugo has a menu system that can separate navigation data from its rendering. We will use more structured approaches when we have enough template knowledge. For this exercise, the five explicit links make the relationship visible and easy to inspect. [Hugo menus](#)

The My interests and Next steps sections can stay on your home page. We are removing their top-navigation entries, not deleting the content or changing its existing heading identifiers.

Give the home page clear starting points

A visitor may scan the body of the home page before noticing its navigation. Help them choose a useful next step.

In `content/_index.md`, preserve your existing introduction, My interests, Next steps, and Latest writing. Add this section at the end:

```
## Explore the notebook

- [About](about/): What this notebook is for.
- [Articles](articles/): Explanations and learning notes.
- [Projects](projects/): Work in progress and what I have learned from it.
- [Resources](resources/): References and useful examples.
```

Save and inspect the home page. These links repeat some navigation destinations, but add context. They do not copy the destination pages' full content.

A home page should help people begin. As the site grows, you can shorten the introduction or select a smaller set of featured items if the page becomes crowded. You do not need to show every future article on the home page.

Second checkpoint: the site now has consistent navigation and useful landing pages, and its original article remains reachable at its existing address.

3.7 Choose stable names, addresses, and organising tools

Use short, descriptive folder names. The examples use lowercase letters and hyphens: `first-learning-note` and `learning-notebook`. This is a practical convention for our English-language project, not a rule that all websites must use English URLs.

Avoid names such as `page2`, `new-final`, or `test-copy` for destinations you intend to keep. They say little to a visitor and become confusing as the site grows.

Here is the completed map. Preview paths are relative to the local site address.

Content source	Preview path	Purpose
<code>content/_index.md</code>	<code>/</code>	Home
<code>content/about/index.md</code>	<code>/about/</code>	About
<code>content/articles/_index.md</code>	<code>/articles/</code>	Article landing page
<code>content/articles/first-learning-note/index.md</code>	<code>/articles/first-learning-note/</code>	First article
<code>content/projects/_index.md</code>	<code>/projects/</code>	Project landing page

Content source	Preview path	Purpose
<code>content/projects/learning-notebook/index.md</code>	<code>/projects/learning-notebook/</code>	Project description
<code>content/resources/index.md</code>	<code>/resources/</code>	Resource collection

Our Markdown links are relative to the page where they appear. For example, `learning-notebook/` works from the Projects landing page, while `../projects/learning-notebook/` works from Resources. Copying a relative link to a page at another location can change what it points to.

A displayed title and a URL can change independently

In `content/about/index.md`, change the title from “About this notebook” to “Why I keep this notebook”. Save and reopen `/about/`.

The page heading changes, but the address remains `/about/` in our current configuration. The navigation still says About because its label was written separately in the layout. Restore the original title or keep the new one if it suits your writing.

Renaming a content folder is different: in this project it changes the corresponding URL. Once a site has public readers, existing bookmarks and inbound links matter. Later chapters will cover redirects and planned migrations.

Sections, tags, and categories: different organising tools

Sections answer “where does this item belong in the site?” Tags and categories can describe subjects shared by content in different locations.

For our growing notebook:

Organising choice	Example	Use
Section	Articles	Groups a kind of content and gives it a landing page
Category	Web publishing	A broad subject grouping, if we choose to use one
Tag	Markdown	A narrower topic that could connect an article and a project

Hugo calls such subject groupings **taxonomies**. Categories and tags are conventional names; Hugo does not force categories to be broad or tags to be narrow. That distinction is an editorial convention we would adopt and document. [Hugo taxonomies](#)

We will configure and display these labels in later content-modelling and template exercises. Simply adding a `tags` field does not make our current minimal layout display tag badges or useful linked lists. For this chapter, sections and explicit links are sufficient.

Resist creating a label for every word in an article. Before introducing a grouping, decide how it will help a visitor find related material and who will keep its names consistent.

3.8 Find and repair a broken route

This exercise is for your local practice copy. Do not perform unplanned moves on a public site.

First open `/projects/` and confirm that My knowledge notebook opens the correct project page. Then:

1. Stop the preview server with `Ctrl+C`.
2. In the editor, rename the folder `content/projects/learning-notebook` to `content/projects/learning-notebook-test`. Keep its `index.md` inside it.
3. Leave the links in the Projects and Resources pages unchanged.
4. Restart `hugo server`, open `/projects/`, and click its project link.

The link still points to the old address, so a freshly generated preview should no longer find the project there. Open `/projects/learning-notebook-test/` directly to confirm that the page exists at its new address.

The content has not been deleted. Its destination has moved while the links have stayed the same. Hugo can build successfully even when a manually written link no longer reaches a page.

If the old page remains visible, your browser may be serving a cached copy of the earlier page. Perform a hard refresh (**Ctrl+Shift+R** on Windows/Linux or **Cmd+Shift+R** on macOS), or restart Hugo with `--renderToMemory` to guarantee that no stale disk files are served:

```
hugo server --renderToMemory
```

For this exercise, recover by restoring the original destination:

1. Stop the server.
2. Rename the folder back to `learning-notebook`.
3. Restart the normal `hugo server` preview.
4. Test the project links from both Projects and Resources.

We deliberately keep the original address. If a real move were needed, we would identify all affected links and consider redirects as part of the change.

3.9 Test the site as a visitor

Open a new browser tab at the home page. Follow each route below using links rather than typing every destination:

Visitor task	Route to try	What success looks like
Understand the purpose	Home → About	A clear explanation and a useful example link
Read something	Home → Articles → first learning note	The right article, including its screenshot
Inspect a project	Home → Projects → My knowledge notebook	An accurate purpose and status, plus related reading
Find a reference	Home → Resources → Hugo documentation	The intended external reference opens
Recover orientation	Open the article directly, then use navigation	Main site destinations are available without starting at Home

Use the keyboard for one route as well. Press Tab to move through links, observe the visible focus indicator supplied by the stylesheet, and press Enter on the link you want. The first link should offer Skip to content. It remains useful even though we changed the main navigation.

Resize the browser to a narrow window. The existing navigation styling should let its links wrap rather than forcing a single long row. Check this in your actual browser; an automated build cannot establish that the site is comfortable to use.

If a route is confusing, change its label or explanation before adding more content. An accessible destination is not necessarily an understandable one.

3.10 Try an independent improvement and preserve the result

Choose one of these small tasks:

- Add a second resource you have read, explain its relevance, and test its link.
- Improve the About page so it names a specific audience and explains what they can expect.
- Add one accurate sentence to the project page describing its next planned improvement, clearly distinguishing planned work from completed work.

Do not create new sections for this exercise. Practise making the existing structure more useful.

An optional AI-assisted planning task is to supply the visitor questions and current navigation labels to a chatbot and ask it to identify one confusing label. Ask for reasons, not a redesigned website. Decide whether the suggestion matches your intended audience before editing anything. Agent installation and direct file access still come later.

Explain in a learning note:

1. Why does Articles have `_index.md` while About has `index.md` ?
2. Why did one navigation edit affect several pages?
3. Why does creating a new page not automatically add a link to our manual lists?
4. Why is changing a title usually a smaller routing change than renaming a folder in this setup?

Preserve the completed structure

Save your files and stop the preview. Copy the active project to a sibling folder named `my-knowledge-site-ch03-backup`. Keep it outside the project and continue using the original folder for Chapter 4.

The required changes are:

File	Change
<code>content/about/index.md</code>	New About page
<code>content/articles/_index.md</code>	New section introduction and manual article link
<code>content/projects/_index.md</code>	New section introduction and manual project link
<code>content/projects/learning-notebook/index.md</code>	New project description
<code>content/resources/index.md</code>	New resource collection
<code>content/_index.md</code>	Added Explore the notebook section
<code>layouts/all.html</code>	Replaced only the main navigation block

Your existing article, screenshot, configuration, and stylesheet remain in place. If you chose an independent improvement, its content may differ from the examples; its links should still work.

Completion check

- ☐ Every main navigation item opens a useful page in the normal preview.
- ☐ Articles and Projects have working landing pages.
- ☐ The original article still uses its Chapter 2 address and displays its image.
- ☐ My manual lists link to the correct items.

- ☐ I have distinguished site structure, navigation, and subject labels.
- ☐ I restored the original project folder name after the recovery exercise.
- ☐ I followed the visitor routes, including one with the keyboard.
- ☐ I saved a working Chapter 3 checkpoint.

Chapter 4 will inspect the HTML generated by these pages. We will connect the content you write, the navigation fragment you replaced, and the structure the browser actually receives.

Troubleshooting when you need it

Symptom	Check
Articles has a heading but no links.	Confirm you saved the supplied body in <code>content/articles/_index.md</code> . This starter does not generate a list automatically.
An article becomes unavailable after creating the section page.	Confirm the section file is <code>_index.md</code> , not <code>index.md</code> , and that the article is still in its original folder.
An item exists but is not shown in the navigation.	Navigation entries are explicit in <code>layouts/all.html</code> ; adding a content file does not edit that block.
An item appears in navigation but returns a 404.	Check its destination file, spelling, draft status, and the path inside the navigation expression.
Only the navigation appears and the rest of the page is gone.	You may have replaced the whole layout with the fragment. Restore <code>layouts/all.html</code> from the Chapter 2 backup, then replace only its <code><nav>...</nav></code> block.
A changed navigation label still opens the old destination.	Labels and destinations are separate. Inspect the link's path as well as its visible wording.
A link contains the right words but opens the wrong relative path.	Check the page from which it is followed. <code>../</code> moves up from that page's URL, not from the project root.
The old project page appears during the rename exercise.	Check the URL, make a fresh request, and use the in-memory preview described in the exercise to rule out stale generated output.

Symptom	Check
A new section appears twice on the home page.	Add the Explore section once. Remove only an accidental duplicate, preserving your earlier personalised content.
The article screenshot no longer matches the site's navigation.	It records the Chapter 2 state. That is acceptable as a historical screenshot; capture a new one only if you intend the article to describe the current appearance.

CHAPTER 04

Understand the HTML Behind Your Pages

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

You have written articles, created sections, and changed the navigation of your website. Now we will look at what the browser receives when it displays those pages.

The title on the screen is not simply large text. The navigation is not merely a row of coloured words. They have a structure that the browser can interpret. Understanding that structure will help you make deliberate changes, diagnose mistakes, and assess code suggested by an AI agent.

You will begin by inspecting your existing article. Then you will make a temporary browser-only change, update the site's footer in its source file, and investigate how a link can look correct while pointing to the wrong place.

What you will be able to do

By the end, you should be able to:

- Inspect a rendered page and recognise its main HTML elements and attributes.
- Distinguish Markdown, Hugo templates, generated HTML, and the browser's live document.
- Make and verify a small persistent change to the shared layout.
- Check headings, links, images, and the skip-link target for basic structural correctness.

You do not need to memorise an HTML reference or learn JavaScript in this chapter. We will work mostly with structures already present in your site.

4.1 Inspect the article and change it in the browser only

Start from the Chapter 3 project, with its seven authored pages and five navigation links (or checkout companion branch `chapter-03` in `ssg-playground`). Keep your Chapter 3 backup outside the active project.

In the project terminal, run:

```
hugo server
```

Open the address reported by Hugo. Use Articles to reach your first learning note, normally at:

```
http://localhost:1313/articles/first-learning-note/
```

In a desktop browser, right-click the article's main heading and choose **Inspect** or **Inspect Element**, or press **F12** (or **Ctrl + Shift + I** on Windows/Linux, **Cmd + Option + I** on macOS). If you are using a trackpad, use its context-menu gesture. Browser developer tools should open beside or below the page.

Look for the **Elements** panel in Chromium-based browsers such as Chrome or Edge, or the **Inspector** in Firefox. The exact interface varies. You can also open developer tools through the browser's menu and use its element-picker tool to select the heading. [MDN: browser developer tools](#)

You should find something like:

```
<h1>My first learning note</h1>
```

If you personalised the title, your words will differ. The important part is the surrounding `h1` element.

Now inspect the first paragraph. You should find a `p` element containing its text. Inspect a section heading such as What I tried and look for `h2`.

You are already reading HTML by matching visible page content to the structure that displays it.

Make a change that does not touch your files

In the Elements or Inspector panel, find the text inside the article's `h1`. Double-click the text, if your browser supports that, and change it to:

```
This is a temporary browser edit
```

Press Enter. If direct text editing is unavailable, right-click the element in the panel, choose **Edit as HTML**, and change only its text. Preserve the opening `<h1>` and closing `</h1>` tags.

The visible heading should change. Now reload the page without editing or saving any project file.

The original heading should return. In an ordinary developer-tools session, this edit changes the browser's current document only. It does not update your Markdown file or Hugo template. This exercise assumes you have not configured developer-tool workspaces or persistent local overrides.

First checkpoint: you can inspect the heading, change it temporarily, and explain why a reload restores the source-based result.

This is a useful way to try an idea, but a persistent correction belongs in the appropriate project file. Keep this distinction in mind when someone demonstrates a change using only browser tools.

4.2 Read the pieces of an HTML element

HTML stands for **HyperText Markup Language**. Its elements describe the structure and meaning of a document.

Consider an ordinary link:

```
<a href="https://gohugo.io/documentation/">Hugo documentation</a>
```

Piece	What it means
<code><a ...></code>	Opening tag for a link element
<code>href="..."</code>	An attribute specifying its destination
Hugo documentation	The visible link text
<code></code>	Closing tag

Elements can be nested. Close inner elements before their enclosing elements:

```
<p>Please <strong>check the link</strong> before publishing.</p>
```

Use straight quotation marks around attribute values. Keep spaces between attributes. Indentation makes relationships easier to read but does not create those relationships; the markup does.

Some elements, such as `img`, `meta`, and `link`, are **void elements** and have no closing tag in HTML. Do not add `</img>` after an image. [MDN: basic HTML syntax](#)

A link's text should help readers predict its destination. A working address paired with a misleading label is still a poor link. [MDN: anchor element](#)

4.3 Make one persistent HTML change

We will improve the footer so it includes a route to the About page. Because the footer belongs to the shared layout, the change should appear across the site.

Open `layouts/all.html` in your editor. Near the end, find:

```
<footer>A place to learn, explain, and share.</footer>
```

Replace that entire footer element, and only that element, with:

```
<footer>
  <p>Learn, review &amp; share.</p>
  <p class="footer-note">
    Read <a href="{{ "about/" | relURL }}">about this notebook</a>.
  </p>
</footer>
```

Save, then check the home page and the first article. Scroll to the bottom. Both should display the new footer. Its About link should open `/about/` in this local setup. Reload the page: the change should remain because you saved it in the source layout.

The paragraphs sit inside `footer`; the link sits inside the second paragraph. That is nesting in a real project change.

The character reference `&` produces a visible ampersand, `&`. For literal angle brackets in ordinary HTML text, `<` and `>` represent `<` and `>`. When writing code examples in Markdown, continue using code fences rather than manually converting all their characters.

The `footer-note` class does not introduce a new visual style by itself. We will use it as a convenient styling target in Chapter 5.

Why there are braces inside the link

The source contains:

```
<a href="{{ "about/" | relURL }}">about this notebook</a>
```

The browser should receive a completed link, normally:

```
<a href="/about/">about this notebook</a>
```

The expression inside `{{ ... }}` is processed by Hugo before the browser receives the page. It adjusts the URL for the site's configured base. It is not browser-side HTML syntax. Keep this supplied expression intact; its full explanation belongs in the templating chapters. [Hugo relative-URL function](#)

Second checkpoint: you made a small source change, predicted which pages it would affect, and verified that the change survives a reload.

4.4 Follow the journey from content to browser

The same article appears in several forms. Each has a different role:

Form	Where you see it	Example
Content source	<code>content/articles/first-learning-note/index.md</code>	A front-matter title and Markdown body
Layout source	<code>layouts/all.html</code>	<code><h1>{{ .Title }}</h1></code> and <code>{{ .Content }}</code>
Generated HTML	The response served by Hugo or files created by a build	<code><h1>My first learning note</h1></code> followed by paragraphs and sections
Live browser document	Elements or Inspector	The browser's current tree of elements, including temporary changes

The browser's live document is called the **Document Object Model**, or **DOM**. The browser constructs it by parsing HTML. Scripts or developer tools can subsequently change it.

If your browser offers **View Page Source**, open it for the local article and search for its title. This view exposes the HTML response rather than your Hugo template. The Elements panel shows the browser's parsed, live document; it can differ after edits or browser corrections. You may also see development-only code added for Hugo's live reload.

For everyday authoring, make persistent changes in your content or layout source. Editing a generated `public/.../index.html` file is unreliable because another build can overwrite it.

If the screen literally displays `{{ .Title }}`, check that you opened the Hugo preview address rather than double-clicking `layouts/all.html` in the file manager. A browser cannot execute Hugo template expressions.

Recognise the structure already in your layout

Read `layouts/all.html` from top to bottom. You do not need to change it again yet.

Part	Job in our starter
<code><!doctype html></code>	Selects modern HTML parsing behaviour
<code><html lang="en"></code>	Contains the document and declares English as its main language
<code><head></code>	Holds information about the document and linked resources
<code><meta charset="utf-8"></code>	Declares the character encoding

Part	Job in our starter
Viewport <code>meta</code> element	Helps the layout use the device's viewport appropriately
<code><title></code>	Supplies the document title normally shown in the browser tab
Stylesheet <code><link></code>	Tells the browser which CSS resource to load
<code><body></code>	Contains the page's displayed structure and content

The `head` element is different from `header`. The former holds document information; the latter can provide introductory content within the body. Likewise, a document's `title` and its visible `h1` serve different roles even when their wording overlaps.

Within this site's body, you will find:

Element	Meaning here
<code>header</code>	Site name and introductory navigation
<code>nav</code>	Major navigation links
<code>main</code>	This page's primary content
<code>article</code>	The page's self-contained piece of writing
<code>footer</code>	Closing site information and the new About link

These are **semantic elements**: their names communicate purpose. `section` can group a thematic part of a document, typically with a heading; `aside` can hold related supplementary content. Neither is needed just to make a coloured box. Choose markup for its role, then use CSS for appearance. [MDN: structuring documents](#)

The English language declaration fits our current example. It is not a rule for all pages in the future bilingual site. We will revisit language and direction together in the multilingual chapter.

4.5 Connect Markdown to the HTML it creates

Return to your article's Markdown file and compare it with the browser inspector:

Markdown or metadata	Typical HTML in our starter
Front-matter <code>title</code>	An <code>h1</code> inserted by the layout
<code>## What I tried</code>	An <code>h2</code> section heading
A paragraph separated by blank lines	A <code>p</code> element
<code>**important words**</code>	A <code>strong</code> element
<code>*emphasised words*</code>	An <code>em</code> element
A bulleted list	<code>ul</code> containing <code>li</code> elements
A numbered list	<code>ol</code> containing <code>li</code> elements
Text inside single backticks	A <code>code</code> element
A fenced code block	A block that normally contains <code>pre</code> and <code>code</code>
A Markdown link	An <code>a</code> element with <code>href</code>
A Markdown image	An <code>img</code> element with <code>src</code> and <code>alt</code>

Inspect one example from each of three rows. You are not expected to rewrite the article in HTML. The value of this comparison is knowing what your authoring choices produce.

Heading level is a structural choice

Our layout supplies one main `h1` per page. Body sections use `h2`; their subsections use `h3`. This is our straightforward page structure, not a claim that font size determines importance.

Replacing an `h2` with a bold paragraph may preserve a similar appearance but removes a section heading from the document structure. Screen-reader users can use heading navigation, so that difference matters. Conversely, making text an `h2` only to enlarge it gives the document a misleading structure. [MDN: heading elements](#)

If you find two large article titles, inspect them and trace each back to its source. A common cause in our project is adding `# My first learning note` to the Markdown body when the layout already renders the front-matter title. Remove the unnecessary body title rather than hiding it with CSS.

4.6 Read attributes without guessing

Attributes add information to elements. Three groups matter immediately in this project.

Destinations and image descriptions

Inspect the screenshot in your first article. Its HTML should contain an image source and alternative text, similar to:

```

```

`src` identifies the resource. `alt` supplies a text alternative appropriate to the image's purpose. The visible italic caption you added in Chapter 2 is separate from `alt`.

An informative image needs useful alternative text; a purely decorative image may appropriately have `alt=""`. Omitting the attribute is different from deliberately giving an empty alternative. Check the meaning in context instead of automatically describing every image the same way. [MDN: image element](#)

Identifiers and classes

An `id` identifies one element within a page. Its value should be unique in that document. The same value can appear on different pages: each page has its own document. A fragment such as `#main` can point to an element with `id="main"`. [MDN: id attribute](#)

A `class` assigns a reusable label. Multiple elements may share the same class, and an element can have more than one class. CSS and scripts can use these labels to select elements. A class name does not itself create styling. [MDN: class attribute](#)

In our site, `class="site-name"` already has a stylesheet rule, while the new `class="footer-note"` has no specific rule yet. Neither needs to be globally unique.

Existing accessibility-related attributes

The navigation's `aria-label="Main navigation"` supplies an accessible name for that navigation region. Our `main` element has `tabindex="-1"`, which makes it focusable without adding it as an ordinary stop in sequential Tab navigation. This supports the skip-link arrangement in the starter.

Keep these attributes while making unrelated edits. Use native HTML elements appropriately and understand the purpose of any accessibility attribute you add. Adding attributes at random is not an accessibility review.

4.7 Trace and repair the skip link

The first link in the layout is:

```
<a class="skip-link" href="#main">Skip to content</a>
```

Its destination is the existing element:

```
<main id="main" tabindex="-1">
```

Together, they offer a route past repeated navigation to the page's content. The stylesheet keeps the link out of view until it receives focus.

Reload the home page and place keyboard focus in the page. Press Tab from the beginning of the page's focus order until Skip to content appears. Browser chrome can receive focus first; keep track of where focus is rather than assuming the first keypress always reaches the page. Activate the link with Enter. The address should acquire `#main`, and the browser should move to the main-content target. You may need a longer page to notice much scrolling.

Now make a deliberate source mistake. In `layouts/all.html`, change only the skip link's destination from `#main` to `#missing-main`. Do not change the `main` element's identifier. Save and reload.

The link still looks like a link. Its activation may add `#missing-main` to the address, but no element with that identifier exists. Hugo may build successfully because a successful template render does not prove that every fragment has a matching target.

Use the inspector's search facility to look for `id="missing-main"`. Compare the result with `id="main"`. Repair the source link by restoring `href="#main"`, save, and repeat the keyboard check.

The lesson is specific: matching words in the address bar are not enough. Verify that the target exists and that the route is useful.

4.8 Know which source file to edit

When something looks wrong, first decide where the information came from:

Observation	Source to inspect first
The article's title is wrong.	Its front-matter <code>title</code>
A body heading or sentence is wrong.	The page's Markdown body
A main navigation label is wrong on every page.	The navigation block in <code>layouts/all.html</code>
The About link in the new footer is wrong everywhere.	The footer block in <code>layouts/all.html</code>
The screenshot's alternative text is wrong.	The image syntax in the article's Markdown
The site name is wrong everywhere.	<code>title</code> in <code>hugo.toml</code>

Observation	Source to inspect first
The font, colour, or spacing is wrong.	<code>static/css/site.css</code> , after checking the underlying HTML
An inspector-only edit disappears.	Make the intended change in its persistent source file.

Do not put the same paragraph into both the Markdown body and the shared layout. The layout would then repeat it across unrelated pages. Do not move page-specific writing into the layout just because the browser inspector made it look like one continuous HTML file.

Raw HTML embedded in Markdown also depends on the generator's rendering configuration. For this chapter, put the footer HTML in the supplied layout and keep ordinary page writing in Markdown. There is no need to change raw-HTML rendering settings.

4.9 Optional recognition: tables and form controls

You will encounter more HTML in later chapters and in agent-generated proposals. Recognise the purpose of these examples; do not add them to the project for this exercise.

A small data table

```
<table>
  <caption>Notebook content</caption>
  <thead>
    <tr><th scope="col">Section</th><th scope="col">Purpose</th></tr>
  </thead>
  <tbody>
    <tr><td>Articles</td><td>Learning notes</td></tr>
    <tr><td>Projects</td><td>Descriptions of ongoing work</td></tr>
  </tbody>
</table>
```

`tr` identifies a row; `th` a header cell; `td` a data cell. The caption identifies the table, and `scope="col"` associates each header with its column. Tables are useful for tabular information, not for positioning the page's navigation and content. [MDN: table element](#)

A labelled input

```
<label for="contact-email">Email address</label>
<input id="contact-email" name="email" type="email">
```

The label's `for` value matches the input's `id`. The `name` can identify the value when form data is submitted; it serves a different purpose from `id`. A placeholder is not a substitute for a persistent label. [MDN: label element](#)

This fragment is not a working contact system. An actual form needs a deliberate submission route and processing arrangement. Its HTML alone does not deliver messages, store records, or establish that submitted information is safe. We will build a complete modest integration in the interaction chapter. [MDN: form element](#)

4.10 Judge an agent’s proposed change, then make your own

Consider these two illustrative proposals. No agent account is needed.

Proposal	What you should examine
Replace the article's <code>h1</code> with a paragraph to reduce its size.	The goal concerns appearance, but the change removes the main heading. Keep the heading and address size through CSS later.
Shorten every link label to "Read more".	Check whether visitors can still distinguish the destinations, especially when encountering links outside their surrounding paragraphs.

Ask an agent to explain which files and elements it intends to change and why. Compare that explanation with the rendered result. A confident description does not replace inspecting the markup or checking keyboard behaviour.

For an optional writing exercise, describe how you would request the footer change we made: identify the file, the existing block, the required wording, the About destination, and what must remain intact. We will turn instructions like this into practical agent tasks later.

Make one independent change and preserve it

Change only the first paragraph of your new footer to a short sentence that fits your notebook. Preserve the About link, the `footer-note` class, and the rest of the layout. If you use an ampersand in HTML text, practise writing it as `&`.

Then:

1. Check the footer on Home, About, and your first article.
2. Activate its About link from the article.
3. Reload and confirm the source change persists.
4. Confirm the repaired skip link still targets `main`.

Save all files, stop the preview, and copy the project to a sibling folder named `my-knowledge-site-ch04-backup`. Continue using the original project for Chapter 5.

Only `layouts/all.html` needs a permanent change in this chapter. The temporary inspector edit does not belong in any file. The intentionally broken skip-link destination must be repaired before creating the checkpoint.

Completion check

- ☐ I can locate the article's `h1`, a body heading, a link, and its image in the inspector.
- ☐ I can explain why a temporary browser edit disappears on reload.
- ☐ I can recognise elements, attributes, nesting, and void elements.
- ☐ I can distinguish the page's Markdown from its shared layout and generated HTML.
- ☐ My updated footer appears across the site and its link works.
- ☐ The skip link and main-content identifier match again.
- ☐ I know which source file to inspect for common content and layout problems.
- ☐ I saved a working Chapter 4 checkpoint.

Chapter 5 will use CSS to change appearance deliberately. The `footer-note` class, browser inspector, and structural distinctions you learned here will give those styling changes a clear foundation.

Troubleshooting when you need it

Symptom	Check
Developer tools select the wrong element.	Use the element picker again or expand the nearby nodes in Elements/Inspector until you find the relevant text.
The inspector's heading differs from the source file.	Reload to discard temporary DOM edits; confirm you are previewing the same project and page.
The footer changes on every page.	That is expected: all seven authored pages currently use the shared layout.
The footer change disappears after a reload.	You may have edited only the browser DOM. Save the change in <code>layouts/all.html</code> .
Hugo reports a template error near the new link.	Restore the exact <code>{{ "about/" relURL }}</code> expression, keeping both brace pairs and straight quotes.

Symptom	Check
The layout displays template braces literally.	Open the Hugo server address, not the layout file directly.
The new class produces no visible change.	No class-specific CSS rule has been added yet. A class is a label, not a style definition.
<code>&amp;</code> appears literally on the page.	Check whether you double-escaped it as <code>&amp; amp;</code> or placed the example inside a code block.
The address ends in <code>#missing-main</code> but nothing useful happens.	Restore the skip link to <code>#main</code> ; a fragment needs a corresponding element identifier.
The page still looks acceptable after a markup mistake.	Browsers can recover from malformed HTML. Inspect the resulting structure rather than treating appearance as proof of correctness.

Completed layout for comparison

Use this as a recovery reference for `layouts/all.html` if an edit went wrong. It preserves Chapter 3's navigation and includes the new footer. Retain your own accurate footer wording if you completed the independent exercise.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{{ .Title }} | {{ .Site.Title }}</title>
  <link rel="stylesheet" href="{{ "css/site.css" | relURL }}">
</head>
<body>
  <a class="skip-link" href="#main">Skip to content</a>
  <header>
    <p class="site-name">{{ .Site.Title }}</p>
    <nav aria-label="Main navigation">
      <a href="{{ "" | relURL }}">Home</a>
      <a href="{{ "about/" | relURL }}">About</a>
      <a href="{{ "articles/" | relURL }}">Articles</a>
      <a href="{{ "projects/" | relURL }}">Projects</a>
      <a href="{{ "resources/" | relURL }}">Resources</a>
    </nav>
  </header>
  <main id="main" tabindex="-1">
    <article>
      <h1>{{ .Title }}</h1>
      {{ .Content }}
    </article>
  </main>
  <footer>
    <p>Learn, review & share.</p>
    <p class="footer-note">
      Read <a href="{{ "about/" | relURL }}">about this notebook</a>.
    </p>
  </footer>
</body>
</html>
```

CHAPTER 05

Practical CSS for Your Hugo Site

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Your notebook already has a readable layout. In Chapter 4, you inspected its HTML and added a footer note. Now you will give that note a style and make two small adjustments to the reading experience.

The visible result is small but deliberate: the footer note sits below a thin rule in quieter text, body text reads a little larger, and your article's section headings have clearer space above them. Everything else should look as it did at the end of Chapter 4.

Our aim is modest: understand enough CSS to customise this Hugo website and check an agent's proposed changes. You do not need to learn every property, build a design system, or become a professional web designer. Work through the three edits, learn how to inspect their effects, and keep the reference material for when you need it.

What you will be able to do

By the end, you should be able to:

- Find the stylesheet used by this Hugo project and explain how pages load it.
- Read a simple CSS rule and change text size, spacing, or a border deliberately.
- Recognise the existing rules that help the site fit smaller screens.
- Diagnose a selector mistake and check the result across pages and screen widths.

Start with the working Chapter 4 project (or checkout companion branch `chapter-04` in `ssg-playground`). Its footer should contain the paragraph with `class="footer-note"`, and its skip link should be repaired. Keep your Chapter 4 backup outside the active project.

The only source file we will change is `static/css/site.css`. No new software or account is required.

5.1 Give the footer note a style

Run the preview from your project terminal:

```
hugo server
```

Open the address Hugo reports and scroll to the footer. In your editor, open `static/css/site.css`. Add this rule at the end of the file, after the final closing brace:

```
.footer-note {  
  margin-top: 0.75rem;  
  padding-top: 0.75rem;  
  border-top: 1px solid #c5ccce;  
  color: #46545b;  
}
```

Save and return to the browser. Reload if necessary. You should see a fine line above the About note, space around that line, and a slightly softer colour for its ordinary text. The link keeps its existing link colour and underline.

Check the footer on Home and your first article. The same change should appear on both, because both pages use the same stylesheet and footer markup.

You have made your first targeted CSS change. Before adding anything else, understand how it found the right paragraph.

Read the rule you just added

CSS stands for **Cascading Style Sheets**. It describes the presentation of HTML elements.

In this rule, `.footer-note` is the **selector**: it selects elements whose class list includes `footer-note`. The dot belongs to the CSS selector; it is not part of the HTML class name.

The declarations inside the braces follow the pattern `property: value;`. For example, `color: #46545b;` sets a text colour. Use a colon between the property and value, and a semicolon to end each declaration. [MDN: getting started with CSS](#)

Compare the two files without changing the HTML:

```
<p class="footer-note">
```

```
.footer-note {  
  color: #46545b;  
}
```

The second snippet illustrates the connection; do not add a second rule just to copy it.

First checkpoint: the footer changed on more than one page, and you can explain how its HTML class connects to its CSS rule.

5.2 Know where CSS belongs in this Hugo project

Our starter uses a deliberately simple arrangement:

Place	Role
<code>content/</code>	The Markdown writing and page metadata
<code>layouts/all.html</code>	The shared HTML structure and stylesheet link
<code>static/css/site.css</code>	The stylesheet you edit
<code>public/css/site.css</code>	The published copy after an ordinary build

Hugo copies files from `static/` into the published site. In our default configuration, `static/css/site.css` becomes `css/site.css` within the output. The word `static` does not appear in its public URL. Hugo also supports an `assets/` directory for resources used through its asset-processing features; we do not need that workflow here. [Hugo: directory structure](#)

Our layout already includes:

```
<link rel="stylesheet" href="{ "css/site.css" | relURL }}">
```

Keep that line intact. On the local preview at the site root, it normally produces a link to `/css/site.css`. All seven authored pages load the same CSS file. Hugo supplies the files and resolved address; the browser applies the CSS.

To see what the browser receives, open `/css/site.css` on the same preview address, for example `http://localhost:1313/css/site.css` when Hugo is using that address. You should see the stylesheet text, including your new rule. Return to the page afterwards.

Edit the source under `static/`, not the generated copy under `public/`. A build can replace the latter. Putting CSS into the article's Markdown body will not update the shared stylesheet.

Other Hugo themes may provide a custom-CSS setting or use an asset pipeline. Follow that theme's documented extension mechanism if you later adopt one. The filename and directory arrangement in this chapter belong to our starter; they are not a universal theme customisation recipe.

5.3 Learn only the selectors you meet here

You can understand most of our stylesheet with these examples:

Selector	What it selects in this site
<code>body</code>	The document body
<code>h2</code>	All second-level headings

Selector	What it selects in this site
<code>.footer-note</code>	Elements carrying the <code>footer-note</code> class
<code>article img</code>	Images inside an article, including nested descendants
<code>header, main, footer</code>	Each of these three element types
<code>a:focus-visible</code>	Links when the browser determines focus should be visibly indicated

The space in `article img` describes a relationship. The commas in `header, main, footer` group separate selectors. Keep that distinction when reading an agent's suggestion.

Why the link did not take the footer's colour

Many text properties, including colour and font family, are inherited from a parent when the element has no applicable value of its own. The footer paragraph gets our new colour. Its link already has a colour from the starter's `a` rule, so it does not simply inherit the paragraph's colour.

When declarations compete on the same element and property, CSS uses the **cascade**. In our plain stylesheet, a class selector normally outweighs an element selector for otherwise comparable normal declarations. Between equally specific, otherwise comparable rules, the later declaration wins. "The last rule always wins" is therefore an unreliable shortcut. [MDN: handling CSS conflicts](#)

For now, inspect the relevant element and look for the property in the Styles or Rules panel. Overridden declarations are commonly crossed out. The Computed panel shows the resolved value. You do not need to calculate every possible specificity case.

Keep edits in the existing rule where practical. Adding repeated overrides or `!important` without understanding the competing rule makes a small stylesheet harder to maintain.

5.4 Make article text comfortable to read

Find the existing `body` rule. Add one declaration immediately after `font-family`:

```
font-size: 1.125rem;
```

The complete rule should now be:

```
body {  
  margin: 0;  
  background: #f5f3ed;  
  color: #263238;  
  font-family: system-ui, sans-serif;  
  font-size: 1.125rem;  
  line-height: 1.7;  
}
```

Replace or edit the original rule; do not append this entire block as a second `body` rule.

Save and read a paragraph in your first article. With a root font size of 16 CSS pixels, `1.125rem` is 18 CSS pixels. Browser settings may give a different result: `rem` refers to the root element's font size, not to a guaranteed physical size. Text that has its own font-size rule, such as our main heading, will retain that separate sizing. [MDN: font-size](#)

Our existing `system-ui, sans-serif` font stack asks for a system interface font and provides a generic fallback. It avoids needing a font download for this exercise. The existing unitless `line-height: 1.7` makes line spacing respond to the text size.

Try reading several sentences, not just the title. Is the text comfortable? Are navigation labels still usable? This value is a starting choice for our notebook, not a rule that all websites must use.

Three units worth recognising

Unit	Useful interpretation here
<code>rem</code>	Relative to the root font size; used for text and spacing
<code>px</code>	CSS pixels; used here for a thin border and focus outline
<code>%</code>	Relative to a reference size determined by the property; our image rule uses it to fit the containing area

Keep the existing `clamp(...)` expressions for now. They already limit our heading size and content padding while allowing them to vary with the viewport. You need to recognise their purpose before you need to write them from memory.

5.5 Adjust spacing without changing the content

Find the existing `h2` rule and change `margin-top` from `2rem` to `2.5rem`:

```
h2 {  
  margin-top: 2.5rem;  
  line-height: 1.3;  
}
```

Save and compare two sections of the first article. The extra separation should help reveal where a new section begins. You did not add blank paragraphs or change the heading level.

Inspect a heading and look for the browser's box-model display. Its labels help distinguish three ideas:

Property group	Where the space or line belongs
Margin	Outside the element's border
Border	Around the element's padding and content
Padding	Between the content and its border

The footer exercise uses all three: its margin separates it from nearby content, its border provides a dividing line, and its padding separates that line from its text. Vertical margins between ordinary blocks can collapse rather than simply add together, so use the inspector when measured gaps surprise you.

Our starter begins with `* { box-sizing: border-box; }`. This makes specified box widths include padding and borders, which helps avoid accidentally making a box wider than intended. You can retain it without studying alternative box models now. [MDN: the box model](#)

Second checkpoint: you have adjusted text and spacing through their existing rules, while retaining the article's headings and Markdown structure.

5.6 Understand the responsive behaviour already supplied

A content website needs to remain readable when the available width changes. Our starter already includes several useful rules. Leave them in place and observe what they do.

A reading area with a maximum width

```
header, main, footer {  
  width: min(100% - 2rem, 48rem);  
  margin-inline: auto;  
}
```

In this page, the width is the smaller of the available width minus `2rem` and `48rem`. This provides side space on narrow screens and prevents the reading area expanding indefinitely on wide ones. `margin-`

`inline: auto` centres these blocks in our current layout.

Avoid replacing this with a large fixed width simply to match one screenshot. Resize the preview and watch the content width change.

Navigation that can wrap

```
nav {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 1rem;  
}
```

The starter uses Flexbox to arrange the links. `flex-wrap: wrap` allows them to continue onto another line when there is not enough room. `gap` separates the items and wrapped lines. This small use of Flexbox is sufficient for our navigation; a full Flexbox course is unnecessary here. [MDN: flex-wrap](#)

Images and code that fit the article

Chapter 2 supplied:

```
article img {  
  display: block;  
  max-width: 100%;  
  height: auto;  
}  
  
article pre {  
  max-width: 100%;  
  overflow-x: auto;  
}
```

Keep these rules. The image can shrink to fit the article while retaining its proportions. Long preformatted code can scroll within its own area instead of forcing the whole page wider. These rules do not reduce the image file's download size.

A **media query** applies rules when a condition, such as viewport width, is met. You may encounter one in a theme, but our current changes do not need a new breakpoint. Flexible sizing and wrapping already address this chapter's layout needs. [MDN: responsive design](#)

Check the result at more than one size

Check Home, your first article, and Resources. Narrow the browser window or use its responsive device preview (toggle with **Ctrl + Shift + M** on Windows/Linux or **Cmd + Shift + M** on macOS) at about 360 CSS pixels wide, then try a wider view. Also increase browser zoom to 200% and check that you can still reach and read the content; return to your usual zoom afterwards.

Look for clipped words, links that overlap, distorted images, and page-wide sideways scrolling. Navigation wrapping is expected. A long code block scrolling within its own box is different from the entire page requiring horizontal scrolling.

A responsive preview is a useful check, but it does not reproduce every physical device. When available, inspect the site on an actual phone as well.

5.7 Diagnose one small mistake

Temporarily change the selector of your new rule from `.footer-note` to `.footer_note`, using an underscore. Leave its declarations and the HTML unchanged. Save and reload.

The line and new paragraph colour should disappear. The rule is valid CSS, but it no longer selects the paragraph. Hugo may report no error: it copies this stylesheet without checking whether its selectors match your intended HTML.

Inspect the paragraph and compare its class with the stylesheet selector character by character. The expected rule will not appear among its matching rules. Restore `.footer-note`, save, and confirm that the styling returns.

Use this short sequence when a change seems to do nothing:

1. **Check the file.** Did you save `static/css/site.css` in the project being previewed? Does the served CSS contain the edit?
2. **Check the match.** Does the selector describe the intended element? Is the class spelling exact?
3. **Check the declaration.** Are the property, colon, value, semicolon, and braces correct? Is another rule overriding it?

Temporary developer-tools edits are useful experiments, but normally disappear on reload. Save the intended correction in your source stylesheet.

Symptom	Useful next check
The whole site suddenly looks unstyled.	Inspect the stylesheet link and whether its URL loads successfully.
Only one new declaration has no effect.	Check spelling and the property's accepted values; browsers commonly ignore invalid declarations.
The footer text changes but its link does not.	Inspect the existing <code>a</code> colour rule; this is expected in our example.
Styling changes on every page.	They share the stylesheet. Check the selector's scope before making it broader.

Symptom	Useful next check
A saved change is absent from the served CSS.	Verify the active project and file, then try a reload that bypasses the browser cache.

5.8 Finish with a small independent decision

Choose a heading gap that suits your article. Change only `h2`'s `margin-top` to `2rem`, `2.25rem`, or `2.5rem`. Keep the value you find easiest to read, and explain why in one sentence.

Check the footer, headings, and navigation on the three pages used above. Then use Tab to move through links. Keep the existing visible focus outline and the working Skip to content link. The `a:focus-visible` rule helps keyboard users locate their current link; removing it for a cleaner screenshot would make the site harder to use. [MDN: :focus-visible](#)

For this exercise, retain our text and background colours, and retain link underlines. Future palette changes should include a contrast check; visual preference alone does not establish readability. Standard accessibility guidelines (WCAG AA) require a contrast ratio of at least 4.5:1 for normal body text and 3:1 for large text. Modern browser developer tools display this ratio automatically in their color picker.

When reviewing an agent's styling proposal, ask three questions: which selector changes, which pages it affects, and how the result behaves at a narrow width. For example, "Add space above article section headings by editing the existing `h2` rule in `static/css/site.css`; preserve heading levels and check the article on a narrow screen" is a bounded task you can assess.

Save your Chapter 5 checkpoint

Save your files, stop the preview, and copy the project to a sibling folder named `my-knowledge-site-ch05-backup`. Continue working in the original project.

Completion check

- ☐ I can find the editable stylesheet and its served URL.
- ☐ I can read a selector and a `property: value;` declaration.
- ☐ My footer note has its intended border, spacing, and text colour.
- ☐ I adjusted body text and heading spacing in the existing rules.
- ☐ I repaired the deliberate selector mistake.
- ☐ I checked several pages at narrow and wide widths and with increased zoom.
- ☐ Links remain recognisable and keyboard focus remains visible.
- ☐ I saved a working Chapter 5 checkpoint.

This is enough CSS for our next steps. Grid layouts, animation, complex selectors, preprocessors, utility frameworks, and comprehensive theme design are outside this chapter's scope. Look up additional features when a concrete site requirement calls for them.

Chapter 6 introduces Git so that you can record changes, compare versions, and recover work more reliably than by copying whole project folders.

Troubleshooting when you need it

Symptom	Useful next step
Hugo cannot find <code>static/css/site.css</code> .	Check the path from the project folder: <code>static</code> , then <code>css</code> , then the file. Files under <code>static</code> are served from the site root.
An edit keeps disappearing.	Check whether you opened a copy under <code>public/</code> . That folder holds generated output; edit the source stylesheet under <code>static/</code> .
All styling vanishes after a rename.	The filename must match the stylesheet link in <code>layouts/all.html</code> . Changing one requires changing the other.
A declaration is ignored and nothing reports an error.	Hugo copies this file without checking it. Check the property name and its unit; browsers silently discard declarations they cannot parse.
One mistake affects rules below it.	A missing semicolon or brace lets the parser run on to the next one. Restore it and reload before making further edits.
Text size changes everywhere, not only in the article.	Check whether you edited the <code>body</code> rule. A narrower selector limits how far the change reaches.
Increased zoom breaks the layout.	Compare it with a narrow window; both should reflow. A fixed pixel width is the usual cause.
The keyboard focus outline has gone.	Restore the <code>a:focus-visible</code> rule. Keyboard users need it even when it is not part of your visual preference.
A comment appears on the page.	CSS comments use <code>/* ... */</code> . Markdown and HTML comment syntax do not apply in this file.

Symptom	Useful next step
The colour differs from the value you typed.	Check that the value is a valid colour, then check whether a later rule of equal or greater specificity overrides it.

Completed stylesheet for comparison

This is a recovery reference for `static/css/site.css`, including the earlier starter rules. It reflects the guided values; retain your chosen heading gap if you completed the independent exercise. You do not need to memorise or retype this entire file.

```
* { box-sizing: border-box; }

body {
  margin: 0;
  background: #f5f3ed;
  color: #263238;
  font-family: system-ui, sans-serif;
  font-size: 1.125rem;
  line-height: 1.7;
}

header, main, footer {
  width: min(100% - 2rem, 48rem);
  margin-inline: auto;
}

header { padding-block: 2rem 1rem; }
.site-name { font-size: 1.25rem; font-weight: 700; }
nav { display: flex; flex-wrap: wrap; gap: 1rem; }
a { color: #005b66; text-decoration-offset: 0.2em; }

main {
  padding: clamp(1rem, 4vw, 2rem);
  background: #ffffff;
  border: 1px solid #d5d9d8;
  border-radius: 0.75rem;
  overflow-wrap: anywhere;
}

h1 { font-size: clamp(1.7rem, 5vw, 2.5rem); line-height: 1.2; }
h2 { margin-top: 2.5rem; line-height: 1.3; }
footer { padding-block: 1.5rem; }

a:focus-visible { outline: 3px solid #005b66; outline-offset: 4px; }
.skip-link { position: absolute; top: -10rem; left: 1rem; }
.skip-link:focus {
  top: 0.5rem;
  padding: 0.5rem 1rem;
  background: #ffffff;
}

article img {
  display: block;
  max-width: 100%;
  height: auto;
}

article pre {
  max-width: 100%;
  overflow-x: auto;
}

.footer-note {
  margin-top: 0.75rem;
  padding-top: 0.75rem;
}
```

```
border-top: 1px solid #c5ccce;  
color: #46545b;  
}
```

CHAPTER 06

Track and Recover Your Hugo Site with Git

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

In the previous chapters, you kept copies of the project before continuing. Those copies helped you recover, but comparing them becomes awkward. Which one contains the footer improvement? What changed in the stylesheet? Can you recover a file without replacing the whole site?

Git gives your project a history of recorded checkpoints. In this chapter, you will record your working Hugo site, save one useful content change, and recover from a deliberately poor styling change.

We will learn a small local workflow. You do not need a GitHub account yet. Branching strategies, merge conflicts, and advanced history editing can wait until there is a practical reason to use them.

What you will be able to do

By the end, you should be able to:

- Record the site's source files in a local Git repository while excluding generated output.
- Inspect changes, select them for a commit, and record a meaningful checkpoint.
- Distinguish removing a change from the next commit from discarding a file edit.
- Find recent checkpoints and restore a deliberately changed file.

Start with the working Chapter 5 project and keep its existing backup (or switch to companion branch `chapter-05` in `ssg-playground`). Save all open files. Stop the Hugo preview for the initial setup so that the terminal is available for the commands below.

6.1 Prepare Git in the right folder

Open a terminal in the project folder that contains `hugo.toml`, `content`, `layouts`, and `static`. In an editor with an integrated terminal, opening the project folder first usually makes this straightforward. Check the terminal's location; opening a file in the editor does not necessarily change it. (If you are following along in the `ssg-playground` companion repository, it is already a Git repository—you can simply switch between chapter branches without re-running `git init`).

Run:

```
git --version
```

If Git is available, you will see a version number. If the command is not recognised, use the [official Git installation page](#) and follow the instructions for your operating system. Reopen the terminal afterwards and repeat the check. Installing Git and creating a GitHub account are separate tasks.

The examples in this draft were tested with Git 2.51.1. Use a recent supported installation rather than trying to match that exact version. Commands below are entered one line at a time and work in a normal PowerShell, macOS, or Linux terminal with Git installed.

Before creating a repository, run:

```
git rev-parse --show-toplevel
```

For the plain project created in this book, an error containing `not a git repository` is expected at this point. It means no enclosing repository was found.

If a path appears instead, Git already manages this folder or a parent folder. Do not create a nested repository blindly. If the path is this project, use its existing history and skip `git init`. If it is an unrelated parent, put the standalone Hugo project outside that repository before following this beginner setup.

Create the local repository

For our project without an existing repository, run:

```
git init -b main
```

This creates the repository's metadata and names the initial branch `main`. A branch names a line of development; we will use just this one for now. Git stores its local history in the project's `.git` directory. Leave that directory under Git's management. [Git: init](#)

No checkpoint has been recorded yet, and nothing has been uploaded.

Choose the author identity for your commits

Replace both placeholders before running:

```
git config user.name "Your Chosen Author Name"
git config user.email "you@example.com"
```

These settings apply to this repository because we have not used `--global`. They identify the author of future commits; they do not sign you into a service. Check them with:

```
git config user.name
git config user.email
```

The chosen name and email become part of commit history, which may later be shared. Use an identity you intend to share. If you already use GitHub and prefer its private commit address, copy the exact `noreply` address from your account settings. Do not invent one. Changing your setting later does not rewrite old commits. [Git: first-time setup](#), [GitHub: commit email addresses](#)

6.2 Save the working site as your first checkpoint

Tell Git which generated files to leave out

Create a plain-text file named `.gitignore` beside `hugo.toml`. Include the initial dot and ensure your editor does not add `.txt` to its name. If the file already exists, retain its contents and add any missing rules below.

```
# Hugo output and generated files
/public/
/resources/
/.hugo_build.lock
/hugo_stats.json

# Local operating-system metadata
.DS_Store
Thumbs.db
```

For our starter, these Hugo outputs can be recreated. We want the source content, layouts, CSS, configuration, and article image in history. The `/resources/` entry refers to the generated directory at the repository root; it does not exclude our `content/resources/` page.

Ignore rules normally keep matching untracked files out of staging. They do not erase files or stop tracking something already committed. This list is tailored to the book's project, not every possible Hugo build arrangement. [Git: gitignore](#)

Select and inspect the initial files

Run:

```
git status
```

Before the first commit, the source files appear as **untracked**: present on disk but not yet recorded by Git. Generated directories covered by `.gitignore` should not appear in the ordinary list.

Select the book's source files:

```
git add .gitignore hugo.toml content layouts static
```

This prepares their current contents in Git's **staging area**, also called the index. Staging chooses what the next commit will contain. It is separate from saving a file in your editor. [Git: add](#)

Review that selection:

```
git status
git diff --cached --stat
git diff --cached
```

The status should now describe changes to be committed. The short summary lists files and change counts; the full diff shows text being added. An image normally appears as a binary change rather than readable lines. If Git opens a scrolling viewer, use Space to move forward and **q** to return to the prompt.

Check that the expected source files are present, including `content/resources/index.md`, and that generated `public/` files and backup folders are absent. Review the content too: an API key or private note should not enter history just because it sits inside an otherwise useful directory.

If the selection contains something unexpected, correct it before committing. The remainder of this chapter assumes the ordinary book project shown above.

Record the checkpoint

Run:

```
git commit -m "Save Hugo site after Chapter 5"
git status
```

A **commit** records the staged project state with an identifier, author information, and a message. The `-m` option supplies that message directly. The command should report a new commit, and status should report a clean working tree when no other changes remain. [Git: commit](#)

A clean working tree describes Git's tracked state; it does not certify that the website is correct. Ignored files can still exist, and the browser checks from Chapter 5 still matter.

First checkpoint: your working Hugo source now has a recorded local version. You have not published the site.

6.3 Understand the three places an edit can be

Before the next exercise, distinguish these:

Place	Meaning	Useful question
Working tree	The project files you edit on disk	What have I changed?
Staging area	The prepared contents for the next commit	What am I about to record?
Commit history	Previously recorded project states	What did I save earlier?

Saving in the editor updates the working file. `git add` stages its current contents. `git commit` records the staged state.

If you stage a file and then edit it again, the additional edit is not automatically included. Stage it again if it belongs in the same commit, then review the staged diff. This is why a file can appear in both the staged and unstaged sections of `git status`.

For this chapter, make one small change at a time. That makes the distinction easier to see and the resulting history easier to read.

6.4 Record a useful content change

Open `content/articles/first-learning-note/index.md`. Add this section at the end, with a blank line before the heading:

```
## My publishing checklist

- Read the page in the local preview.
- Check its links and image description.
- Review the changed files before recording a checkpoint.
```

Save. Start `hugo server` in another terminal if convenient, or run it in the same terminal and stop it after inspecting the article. Check that the section appears correctly. A Git diff will help review the writing, but the preview reveals how it renders.

Back at an available terminal in the project root, run:

```
git status
git diff -- content/articles/first-learning-note/index.md
```

The file should be modified but not staged. The diff should show the new section with `+` prefixes. Removed text uses `-`; unchanged context helps locate the edit. The prefixes are part of the comparison display, not extra Markdown to copy.

Without `--cached`, this diff compares the working file with its staged version. With `--cached`, it compares the staged version with the last commit. The `--` separates options from the file path. [Git: diff](#)

Stage and review:

```
git add content/articles/first-learning-note/index.md
git diff --cached -- content/articles/first-learning-note/index.md
```

Now run the unstaged comparison again:

```
git diff -- content/articles/first-learning-note/index.md
```

It should print nothing if you made no further edits after staging. Your change has not vanished: it is in the staged comparison.

Practise taking a file out of the next commit

Before committing, run:

```
git restore --staged -- content/articles/first-learning-note/index.md
git status
```

Open the Markdown file. Your publishing checklist should still be there. The command removed the change from the staging area while preserving the working file. Status should show it as an unstaged modification.

Stage it again and record it:

```
git add content/articles/first-learning-note/index.md
git diff --cached -- content/articles/first-learning-note/index.md
git commit -m "Add a publishing checklist to the first learning note"
git status
```

Use messages that explain the purpose of a change. "Add a publishing checklist" is more useful when browsing history than "update" or "stuff". A single commit should represent a change you can describe coherently.

Second checkpoint: you reviewed a content change, staged it, unstaged it without losing it, and then committed it deliberately.

6.5 Recover from a deliberately poor CSS edit

First run `git status`. Begin this exercise only when your working tree is clean. This keeps unrelated work out of the recovery example.

Open `static/css/site.css`. In the `body` rule, temporarily replace:

```
font-size: 1.125rem;
```

with:

```
font-size: 6rem;
```

Save and inspect the article in Hugo's preview. Much of the inherited text should become impractically large. The page may still build successfully; the problem is the styling decision.

Do not stage or commit this deliberate mistake. Inspect it:

```
git diff -- static/css/site.css
```

Confirm that the only edit to this file is the font-size experiment. Then discard that working-file edit:

```
git restore -- static/css/site.css
```

This command overwrites that file's unstaged changes with its staged version. Because we began clean and never staged the experiment, that version is also the last committed one. It discards *all* unstaged changes in the named file, not just the font-size line. [Git: restore](#)

Check the file, reload the preview if needed, and run:

```
git status
```

The text size should be restored and the working tree clean. The committed publishing checklist remains in the article.

Two similar commands with different effects

Command used in this chapter	What happens
<code>git restore --staged -- path/to/file</code>	Removes staged changes relative to the last commit; keeps the working file
<code>git restore -- path/to/file</code>	Replaces the working file with its staged version; discards its unstaged edits

These examples concern a file already in the first commit. If you accidentally staged the CSS mistake, plain `git restore` will not undo that staged version. In this exercise, unstage the stylesheet first using the first form, review its remaining diff, and then use the second form to discard the experiment.

For a file containing both useful work and a mistake, correct the unwanted line in your editor instead of restoring the whole file. The path-specific recovery above is appropriate because we deliberately isolated one unwanted edit.

6.6 Read the history you have created

Run:

```
git log --oneline -5
```

For a new repository following this chapter, you should see two commits, newest first: the publishing checklist and the Chapter 5 baseline. Each begins with an abbreviated identifier. Your identifiers will differ from anyone else's. [Git: log](#)

You can inspect the latest committed change to the article with:

```
git diff HEAD~1 HEAD -- content/articles/first-learning-note/index.md
```

Here, `HEAD` identifies the current commit and `HEAD~1` its first parent, the preceding checkpoint in our simple history. This comparison should show the checklist addition. It requires the two commits we have just made.

Your record starts when you begin committing. Git has not reconstructed the individual steps from Chapters 1–5; they are represented together by the baseline commit. It also cannot reliably recover arbitrary edits that were never recorded.

Keep ordinary device backups. Local Git history lives on the same computer as the files and does not protect against losing that computer. If you copy the repository as a backup, include its `.git` directory so the history travels with it. Chapter 7 will add a remote copy on GitHub and the first publishing workflow.

6.7 Make your own small commit

Improve one sentence in `content/about/index.md`. Preserve its front matter and links. Then complete this cycle without copying a whole project folder:

1. Save and inspect the About page in the local preview.
2. Use `git status` and a file-specific `git diff` to review your change.
3. Stage the About file and inspect its staged diff.
4. Commit with a message explaining the improvement.
5. Check that status is clean and the new commit appears in the recent log.

If you forget the syntax, use the command card below. Understanding which state you are inspecting matters more than memorising options.

This is also the foundation for later agent work. Start from a known checkpoint, give the agent a bounded task, inspect every changed file, and decide what to record. A commit documents a change; it does not establish that AI-generated text is accurate or that the website behaves correctly.

Completion check

- ☐ Git manages the Hugo project folder I intended.
- ☐ The initial commit contains the site's source and required image, with generated output excluded.
- ☐ I can distinguish saving, staging, and committing.
- ☐ I reviewed and committed the article's publishing checklist.
- ☐ I unstaged a change without deleting the working edit.
- ☐ I discarded the isolated CSS experiment and checked the restored result.
- ☐ I made an independent About-page commit and can find it in the log.
- ☐ My working tree is clean, and I understand that the site is still local.

Your latest commit is the Chapter 6 checkpoint. Keep the earlier backup, but you no longer need a new sibling folder for every small revision.

This is enough Git for our next steps. Branching, merge conflicts, remote collaboration, and history editing are outside this chapter's scope. Learn each one when a concrete need appears rather than in advance.

Chapter 7 gives the project a public address. You will send this history to GitHub and let a supplied workflow build and publish the site from it.

Troubleshooting when you need it

Symptom	What to check
<code>git</code> is not recognised.	Install Git for your operating system and reopen the terminal.
<code>not a git repository</code> appears after setup.	Check the terminal's folder. Run commands in the project where you initialised Git.
Git asks who you are when committing.	Set the repository's author name and email, then retry the commit.

Symptom	What to check
A path does not match any files.	Check the project root and exact spelling. Use quotes around a path containing spaces.
<code>git diff</code> is empty, but you know you changed something.	Check status and <code>git diff --cached</code> ; the edit may be staged. An untracked file also does not appear in an ordinary unstaged diff.
A file has both staged and unstaged changes.	You edited it again after staging. Review both comparisons before deciding what to include.
A generated file is still tracked despite <code>.gitignore</code> .	Ignore rules do not remove existing tracked files. Diagnose how it entered the repository before committing more output.
Git warns about LF and CRLF line endings.	These are different newline conventions. Check whether the command completed and inspect the diff; do not rewrite every file merely to silence a warning.
<code>nothing to commit</code> appears.	Check status: the edit may already be committed, unsaved in the editor, ignored, or not staged.
A restore command did not undo the mistake.	Check whether the mistake is staged. Apply the two-state explanation in Section 6.5.

Branching, collaboration, and undoing changes already shared with others require additional decisions. We will introduce them where needed. For now, the useful habit is to preview, review, stage, and commit a small understandable change.

Command card: the local workflow

Run commands from the repository root. Replace example paths where necessary.

Purpose	Command
See what is staged, unstaged, or untracked	<code>git status</code>
Review an unstaged file edit	<code>git diff -- path/to/file</code>
Stage the current contents of a file	<code>git add path/to/file</code>
Review everything selected for the next commit	<code>git diff --cached</code>

Purpose	Command
Record the staged changes	<code>git commit -m "Describe the change"</code>
Review recent checkpoints	<code>git log --oneline -5</code>
Unstage a tracked file while keeping its edit	<code>git restore --staged -- path/to/file</code>
Discard a tracked file's unstaged edits	<code>git restore -- path/to/file</code>

CHAPTER 07

Publish Your Hugo Site with GitHub Pages

*Static Site Generators in the Age of AI**Building and Maintaining Content with AI Agents*

Your website now has content, a readable layout, and a local Git history. This chapter gives it an address other people can visit.

You will put the existing repository on GitHub, let GitHub Actions build it with Hugo, and publish the result through GitHub Pages. Then you will make one small update and follow it from your editor to the live page.

This is our first publishing workflow. We will use a supplied configuration and explain its important parts. You do not need to learn all of GitHub Actions, YAML, or website hosting before your site can go online.

What you will be able to do

By the end, you should be able to:

- Connect your local Git repository to an empty GitHub repository and push your commits.
- Configure the site's public address and use a supplied Hugo publishing workflow.
- Find a deployment's status and verify the published pages, links, and image.
- Publish a reviewed content update and recognise where to investigate a failure.

Start with the completed Chapter 6 project (or switch to companion branch `chapter-06` in `ssg-playground`). Its current branch should be `main`, its changes should be committed, and its local preview should work. The deliberate CSS mistake must already be repaired.

You need internet access, a GitHub account, and permission to create a repository in that account. We will also install GitHub CLI for browser-based sign-in from the terminal. The route below uses a **public repository**, GitHub-hosted standard runners, and the supplied `github.io` address. GitHub Pages is available for public repositories on GitHub Free; service limits still apply. No paid domain is needed for this exercise. [GitHub: about Pages](#)

A public repository exposes its committed source and history, including commit author details. Review what you recorded in Chapter 6 before uploading it. A Hugo page marked `draft: true` can be absent from the rendered site while its Markdown remains visible in a public repository.

7.1 Give the project a home on GitHub

Sign in to GitHub in your browser, or create an account and complete the required account setup. Create a **new repository** with these choices:

Setting	Choice for this exercise
Owner	Your personal account
Repository name	<code>my-knowledge-site</code>
Visibility	Public
Initialise with a README	Leave off
Add a <code>.gitignore</code> template	Leave off; we already have one
Choose a licence during creation	Leave unset for this exercise

We need an empty remote repository because your local project already has its own commits. A licence is a separate publishing decision you can make deliberately later; a public repository does not by itself grant a general licence to reuse all its contents.

After creation, GitHub should show setup instructions for the empty repository. Keep that page open. Copy its **HTTPS repository URL**, which should have this shape:

```
https://github.com/YOUR-USERNAME/my-knowledge-site.git
```

`YOUR-USERNAME` is a placeholder. Use the account name from the repository URL, not your profile's display name. If that repository name is already in use, choose an unused name and substitute it consistently throughout the chapter.

Sign in from the terminal

Install GitHub CLI using the instructions for your operating system on the [official GitHub CLI site](#). Reopen your terminal if necessary, then check:

```
gh --version
```

`gh` is GitHub CLI. It supplements the `git` commands you learned earlier.

Run:

```
gh auth login --hostname github.com --git-protocol https --web --scopes workflow
```

Follow the displayed instructions to open the browser, enter the device code if requested, and authorise GitHub CLI for your account. The additional `workflow` scope allows uploading the Actions workflow file we will add below. Complete any account verification there. Then run:

```
gh auth setup-git
gh auth status
```

These commands connect Git's authentication to your GitHub CLI sign-in and let you check which account is active. Your Chapter 6 commit email is author metadata; it is not a login credential. This route does not require putting a password or token into your project files. [GitHub CLI: login](#), [GitHub CLI: configure Git authentication](#)

Connect and push the existing repository

In the terminal at your Hugo project root, run:

```
git status
git branch --show-current
git remote -v
```

The book project should be clean, on `main`, with no remote listed yet. If `origin` already exists, inspect its address before proceeding; do not replace an existing connection just to match the example. The workflow below assumes the `main` branch created in Chapter 6.

Replace the placeholder URL with the one copied from your empty repository:

```
git remote add origin https://github.com/YOUR-USERNAME/my-knowledge-site.git
git push -u origin main
```

A **remote** is a named connection to another repository. `origin` is our name for this one. The push transfers your committed history; `-u` sets the upstream relationship so later `git push` commands know where to send this branch. Unsaved or uncommitted edits are not included. [Git: push](#)

Refresh the repository page. You should see `hugo.toml`, the source folders, and your earlier commits. You should not need to upload `public/` or drag individual files into GitHub's browser editor.

First checkpoint: your source and Git history are now on GitHub. The website itself has not yet been deployed.

7.2 Distinguish the source address from the website address

For the project repository we just created, the addresses normally have these forms:

Address	Purpose
<code>https://github.com/YOUR-USERNAME/my-knowledge-site</code>	Browse source files and project history
<code>https://YOUR-USERNAME.github.io/my-knowledge-site/</code>	Visit the published website
The address printed by <code>hugo server</code>	Preview on your computer

This chapter uses a **project site**, so the repository name appears in the website's path. A repository named exactly `YOUR-USERNAME.github.io` follows a different convention; do not rename our project to that special name for this exercise.

Git stores versions. GitHub hosts the shared repository. GitHub Actions runs the build instructions. GitHub Pages serves the generated website. These services work together, but pushing source files is not itself proof of a successful deployment.

In your repository's browser interface, open **Settings** → **Pages**. Under **Build and deployment**, set **Source** to **GitHub Actions**. Interface labels may move over time; the important choice is a custom Actions workflow rather than publishing directly from a branch folder. [GitHub: custom Pages workflows](#)

7.3 Set the public address in Hugo

Open `hugo.toml`. Replace only its `baseUrl` value, using your actual username and repository name:

```
baseUrl = 'https://YOUR-USERNAME.github.io/my-knowledge-site/'
```

Keep the final slash. Preserve the existing site title and language settings. (Note: in modern Hugo v0.158.0+, `languageCode` is deprecated in favor of `locale = 'en'`. If your build warns about `languageCode`, remove it and use `locale = 'en'` so that `--panicOnWarning` does not abort on the deprecation warning).

The repository path matters. A site published below `/my-knowledge-site/` needs navigation and stylesheet addresses that include that prefix where appropriate. Our shared layout already uses Hugo's `relURL` function for those links. The article's relative image and content links were also designed for the existing folder structure.

Preview with:

```
hugo server
```

Open the **full address reported by Hugo**. With the configured project path, the preview may now be under `/my-knowledge-site/` rather than at the local server's root. Check Home, Articles, the first article,

and its image. Stop the preview with Ctrl+C when finished.

Then run an ordinary build:

```
hugo --minify --panicOnWarning
```

`--minify` reduces generated output where supported. `--panicOnWarning` makes warnings fail this build, so they receive attention before publication. Hugo writes the result to `public/`, which stays excluded from Git by the earlier ignore rule.

The workflow will obtain the actual Pages base address from GitHub and pass it to Hugo at build time. Keeping `hugo.toml` accurate also makes local builds useful. If you later rename the repository or add a domain, revisit the configuration and links together.

7.4 Add the supplied publishing workflow

Using your editor, create a folder named `.github` at the project root, a `workflows` folder inside it, and a file named `hugo.yaml` inside that:

```
.github/workflows/hugo.yaml
```

Paste the following complete file. Use spaces for indentation, and do not include the Markdown fence markers. You do not need to replace anything inside this workflow for the personal project repository described here.

```

name: Publish Hugo site

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-24.04
    env:
      HUGO_VERSION: "0.150.0"
    steps:
      - name: Check out the source
        uses: actions/checkout@v7

      - name: Read the Pages configuration
        id: pages
        uses: actions/configure-pages@v6

      - name: Install Hugo
        shell: bash
        run: |
          curl --fail --location --retry 3 \
            --output "$RUNNER_TEMP/hugo.tar.gz" \
            "https://github.com/gohugoio/hugo/releases/download/v${HUGO_VERSION}/hugo_${HUGO_VERSION}_linux-amd64.tar.gz"
          mkdir -p "$RUNNER_TEMP/hugo-bin"
          tar -xzf "$RUNNER_TEMP/hugo.tar.gz" -C "$RUNNER_TEMP/hugo-bin" hugo
          echo "$RUNNER_TEMP/hugo-bin" >> "$GITHUB_PATH"

      - name: Build the website
        env:
          PAGES_BASE_URL: ${ steps.pages.outputs.base_url }
        run: hugo --minify --panicOnError --baseURL "${PAGES_BASE_URL}/"

      - name: Upload the generated website
        uses: actions/upload-pages-artifact@v5
        with:
          path: public

  deploy:
    needs: build
    runs-on: ubuntu-24.04
    environment:
      name: github-pages

```

```

url: ${ steps.deployment.outputs.page_url }}
steps:
  - name: Publish to GitHub Pages
    id: deployment
    uses: actions/deploy-pages@v5

```

The workflow uses GitHub’s checkout and Pages actions, following the build-and-deploy arrangement documented by Hugo and GitHub. It retains the Hugo version used for the earlier chapter examples. Action versions are explicit too; they should be reviewed when maintaining the book. [Hugo: hosting on GitHub Pages](#), [Checkout action](#), [Configure Pages action](#), [Upload Pages artifact action](#), [Deploy Pages action](#)

Understand the parts you are relying on

Part	Its job
<code>on: push</code> for <code>main</code>	Starts the workflow when commits are pushed to this branch
<code>workflow_dispatch</code>	Allows a manual run from GitHub’s Actions interface
<code>permissions</code>	Allows source reading and the authenticated Pages deployment
<code>concurrency</code>	Groups Pages runs so deployments do not run simultaneously
<code>build</code>	Gets the source, installs Hugo, and produces the site
<code>PAGES_BASE_URL</code>	Passes GitHub’s Pages address to the build
Upload step	Packages <code>public/</code> as a deployment artifact
<code>deploy</code> with <code>needs: build</code>	Publishes after the build job succeeds

An **artifact** here is a packaged build result passed between jobs. It is not another source commit. The `github-pages` **environment** records the deployment target and its URL. GitHub supplies the workflow’s authentication; do not paste your own account credentials into this YAML file.

The `${{ ... }}` expressions belong to GitHub Actions. They are not Hugo template expressions, even though both use braces. The shell script in the install step runs on GitHub’s Linux runner, including when your own computer runs Windows.

Our starter uses plain CSS and no external theme, so this workflow does not install Node.js, Go modules, or a Sass toolchain. A future theme with extra build requirements would need corresponding changes. Copying a workflow does not automatically satisfy every theme’s dependencies.

This is a small automated build-and-deploy pipeline. Chapter 14 will add the fuller CI/CD workflow: checks on proposed changes, clearer quality gates, and controlled deployment. For now, every push to `main` is a publishing action once this workflow is installed.

7.5 Publish and inspect the first deployment

Review the configuration and new workflow before recording them:

```
git status
git add hugo.toml .github/workflows/hugo.yaml
git diff --cached
git commit -m "Configure GitHub Pages publishing"
git push
```

Review the entire staged diff. In particular, confirm the real `baseURL`, the exact workflow file path, and the `main` trigger. There should be no unrelated edits in this commit.

Open the repository's **Actions** tab. Select **Publish Hugo site** and open the run associated with the commit you just pushed. The run should show a build job followed by a deployment job.

Wait for both jobs to finish successfully. If a job fails, open its first failed step and read the message before changing files. A successful push means the source reached GitHub; it does not mean Hugo built or Pages deployed successfully.

Open the website URL shown by the deployment or by **Settings** → **Pages**. Use that reported URL rather than guessing. The first deployment may take a little time to become available. [GitHub: configuring a publishing source](#)

Verify the actual website

Visit these on the published site:

- Home and every main navigation destination.
- The first article, including its screenshot and publishing checklist.
- The project detail page and its link back to the article.
- The footer's About link from a nested page.

Check that the stylesheet loaded and that the content is readable at a narrow width. Open a nested page directly in a new tab and reload it. Its address should remain on the published site's domain and project path, with no `localhost` address involved.

Finally, try the public URL in a private browser window while signed out of GitHub. For this public project site, visitors should not need your account or the Hugo server running on your computer.

Second checkpoint: both workflow jobs succeeded, and you verified the rendered site at its public address.

7.6 Publish one small improvement

Open the first article's Markdown file. Add this item to the publishing checklist created in Chapter 6:

- Check the published page after deployment.

Save, preview locally, and check the diff. Then use the familiar cycle with one new final step:

```
git diff -- content/articles/first-learning-note/index.md
git add content/articles/first-learning-note/index.md
git diff --cached
git commit -m "Add a live-site check to the publishing checklist"
git push
```

Open the new Actions run, confirm that it corresponds to this commit, and wait for deployment to finish. Reload the published article and find the new checklist item. If it is not visible yet, check the deployment status before blaming the browser cache.

You have now completed the whole loop: edit, preview, review, commit, push, watch the deployment, and verify the live page. Repeat this loop for future small updates.

For this exercise, make edits in your local project. Editing independently in GitHub's browser interface would create remote commits that your local branch must first incorporate. We will introduce that collaboration workflow deliberately later.

7.7 Recognise a failed build without publishing a mistake

Start from a clean working tree after the successful update. We will create a temporary local configuration error.

In `hugo.toml`, remove the final closing quote from the `baseURL` line. Save and run:

```
hugo --minify --panicOnWarning
```

Hugo should fail while reading the TOML configuration. Read the error and locate the damaged line. Do not stage or push this experiment.

Review the isolated change and restore it:

```
git diff -- hugo.toml
git restore -- hugo.toml
hugo --minify --panicOnError
git status
```

The second build should succeed and status should be clean. As in Chapter 6, restoring the file is appropriate here because the only unstaged change in it was our deliberate mistake.

If a similar error reached GitHub, the failed build would prevent the dependent deploy job from publishing that run. Fix the source locally, check the build, commit the correction, and push again. A previously successful Pages deployment normally remains available when a later build fails.

A technically successful deployment can still contain inaccurate writing or a broken link. The workflow we have supplied does not verify facts, every destination, or accessibility. The live checks remain part of your publishing responsibility.

7.8 Your independent publishing task

Improve one sentence on the About page. Use the same cycle to preview, commit, push, and verify it at the public address. Choose a change you can recognise clearly on the live page.

Record the public URL somewhere useful, such as your project notes. Your latest committed and pushed version is the Chapter 7 checkpoint.

Completion check

- ☐ My local branch is connected to the intended GitHub repository.
- ☐ The repository contains source and history, while generated `public/` output remains ignored.
- ☐ I can distinguish the repository URL, local preview, and public website URL.
- ☐ Pages uses GitHub Actions as its publishing source.
- ☐ Both build and deployment jobs completed successfully.
- ☐ I checked the live pages, nested links, stylesheet, and article image.
- ☐ My checklist update and independent About-page edit reached the live site.
- ☐ I repaired the temporary local configuration error before committing anything else.
- ☐ I understand that future pushes to `main` trigger publication.

Chapter 8 introduces the primary AI agent environment. With local preview, Git history, and publishing in place, you will be able to give an agent a small task and review its changes before they reach your website.

Troubleshooting when you need it

Symptom	First useful check
GitHub CLI is not found.	Complete its installation and reopen the terminal.
Authentication fails.	Use <code>gh auth status</code> to check the active account and complete the browser login. A Git author email is not authentication.
Pushing the YAML file is rejected for a missing <code>workflow</code> scope.	For the GitHub CLI browser-login route used here, run <code>gh auth refresh --hostname github.com --scopes workflow</code> , complete its authorisation, then retry the push. See GitHub CLI: refresh authentication .
Git says <code>origin</code> already exists.	Inspect <code>git remote -v</code> ; determine whether it points to the intended repository before changing it.
A push is rejected because the remote has different commits.	Check whether you initialised the GitHub repository with a README or edited it online. Do not force-push over unexplained work; reconcile the histories before continuing.
No workflow appears.	Check that <code>.github/workflows/hugo.yaml</code> was committed and pushed to <code>main</code> , and that Actions is enabled for the repository.
GitHub reports invalid workflow syntax.	Check the file extension, spaces, indentation, and copied <code>\${{ ... }}</code> expressions. Do not include Markdown backticks.
Configure Pages or deployment fails.	Confirm Settings → Pages uses GitHub Actions, then read the failed step. Account, repository, or environment restrictions may need attention.
The workflow waits for approval.	Check whether the <code>github-pages</code> environment has deployment protection rules and who may approve the run.
Installing Hugo fails.	Inspect the download step and version string. A network/download failure is different from a content or template error.
Hugo builds locally but fails remotely.	Compare Hugo versions and confirm that all required source files are committed. Also check exact filename case: the runner uses Linux.

Symptom	First useful check
The first public URL returns 404.	Check that deployment finished, use its reported URL, and confirm the project-name path. Allow initial publication time before retrying.
The page appears without styling or navigation goes to the wrong place.	Check the project path and generated stylesheet/link addresses. Preserve the layout's <code>re1URL</code> expressions.
A new local edit is absent online.	Confirm it was saved, committed, pushed, and successfully deployed. Identify the commit attached to the run.
A draft is visible as source on GitHub.	Hugo's draft setting controls rendering, not access to files in a public repository.

Custom domains, DNS, alternative hosts, detailed workflow security, and collaboration policies are beyond this first publishing exercise. Keep the working setup small and introduce those topics when the project needs them.

CHAPTER 08

Work with an AI Agent on Your Hugo Site

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

You can now write a page, recognise its HTML and CSS, save a Git checkpoint, and publish an update. Those skills make an AI agent more useful: you can give it a specific job and judge what it actually changes.

Our first job is small. The Resources page already contains useful links. You will ask an agent to add a short section explaining how a reader might use them. The result should be easy to inspect: one new heading and three bullets in one Markdown file.

We will use **Codex CLI** as the primary agent environment. The aim is to learn a repeatable working method, not to install every available AI tool. Alternative editor setups can use the same task and review criteria.

What you will be able to do

By the end, you should be able to:

- Start the agent in the intended Hugo project and inspect its working permissions.
- Provide a clear task, relevant files, boundaries, and observable success criteria.
- Review the changed files and rendered page independently of the agent's summary.
- Keep an accepted change in Git and recover from an isolated mistake.

Start from the working Chapter 7 project with a clean Git working tree (or switch to companion branch `chapter-07` in `ssg-playground`). The chapter's local exercises also work if you have completed its source configuration but have not yet published. You need internet access and an account with access to Codex.

Check your account's current access and usage limits before starting. Signing in with ChatGPT uses the access available through that account or workspace; API-key sign-in uses separately billed API access. Installing a client does not provide unlimited model usage. We will use **Sign in with ChatGPT** and will not configure an API key in this exercise. [OpenAI: authentication](#)

8.1 Know what you are opening

An agent combines a model with tools that can inspect files, make edits, and run commands. Its usefulness depends on the tools, context, and permissions available in that session.

Term	Meaning in this chapter
Model	The system interpreting the request and generating responses or actions
Agent client	The application connecting the model to project tools
Codex CLI	The terminal client used for our practical exercises
Editor	The application where you inspect and edit files, such as VS Code
Workspace	The project location the agent is working with

An ordinary chat without access to your files can suggest a change, but cannot establish that it changed your local project. An agent with file access can act on the project, so its output needs review as well as reading.

VS Code is an editor, not itself a particular model. An agent extension adds the relevant capabilities. Codex has an official IDE integration, including a VS Code extension; follow its own setup instructions if you later choose that interface. You do not need both interfaces for this chapter. [OpenAI: Codex IDE extension](#)

The CLI runs on your computer, but that does not mean its model runs offline. Relevant prompts and project context can be sent to the model service. Use the book's practice project and the account arrangements appropriate to your material.

8.2 Install, sign in, and check the project

Use the [official Codex CLI installation page](#). Choose one installation route. The standalone installers below avoid introducing a separate Node.js setup solely for this chapter.

On Windows, in PowerShell:

```
powershell -ExecutionPolicy Bypass -c "irm https://chatgpt.com/codex/install.ps1 | iex"
```

On macOS or Linux, in a terminal:

```
curl -fsSL https://chatgpt.com/codex/install.sh | sh
```

These commands download and execute OpenAI's installer. Use the official page as the source for updated instructions. On a managed computer, follow your organisation's software-installation policy rather than changing system restrictions to force an installation.

Reopen the terminal if the installer requests it. Check:

```
codex --version
```

Keep the reported version in your notes; interface details can change. If Codex was already installed, check its version and continue without installing a second copy.

Sign in

Run:

```
codex login
```

Complete the browser sign-in using your intended ChatGPT account. GitHub authentication from Chapter 7 does not sign you into Codex. Do not paste passwords or account tokens into the Hugo project.

On Windows, complete the client's native sandbox setup when prompted. Some setup steps may require administrator approval. Use the current [Windows sandbox instructions](#) if your device cannot complete them. The native Windows route is sufficient for this plain Hugo project; WSL is a separate environment and is not an additional requirement for the book.

Confirm your starting point

In the terminal opened at the Hugo project root, run:

```
git rev-parse --show-toplevel
git status
hugo version
```

Confirm that Git reports your intended project, the working tree is clean, and Hugo is available in this terminal. Save or resolve your own unfinished edits before involving the agent. A clean starting point makes its changes distinguishable from yours.

If a usage limit or account restriction prevents agent access, you can still study the task and apply the comparison example manually. That practises the Hugo change, but it does not complete the hands-on agent exercise.

8.3 Give the project a short instruction file

Create `AGENTS.md` beside `hugo.toml`. If one already exists, read it and retain relevant guidance rather than replacing it blindly. For our starter, use:

Project guidance

This is a small Hugo knowledge website using Markdown and plain CSS.

Files

- Content lives in `content/`.
- The shared layout is `layouts/all.html`.
- The stylesheet is `static/css/site.css`.
- Site configuration is `hugo.toml`.
- The publishing workflow is `.github/workflows/hugo.yaml`.

Working agreements

- Read the relevant source before proposing or making a change.
- Change only the source files requested for the current task.
- Preserve existing front matter, URLs, and authored facts unless the task asks otherwise.
- Do not invent experiences, qualifications, sources, or claims about the author.
- Use the installed Hugo; do not add dependencies or change the publishing workflow unless requested.
- Do not edit generated `public/` or `resources/` files by hand.
- When asked to check a change, run `hugo --minify --panicOnWarning` and report the result accurately.
- If a check cannot run, explain what prevented it and what remains unchecked.
- Leave staging, committing, pushing, and deployment to the reader unless explicitly delegated.

This file gives Codex reusable project guidance. It does not create operating-system permissions or guarantee compliance. Codex may also load applicable instructions from other locations; task-specific instructions still need to be explicit. [OpenAI: AGENTS.md](#)

Read the file yourself, then save a checkpoint:

```
git add AGENTS.md
git diff --cached -- AGENTS.md
git commit -m "Document the Hugo project's agent working agreements"
git status
```

The instruction file is ordinary project source and can be tracked. It is outside `content/`, so this starter will not turn it into a website page. Remember that pushing it to the public repository will make its text visible as source.

8.4 Ask the agent to inspect before it edits

Start a session from the project root:

```
codex --sandbox read-only --ask-for-approval on-request
```

Inside Codex, enter:

```
/status
```

Check the displayed project location and active settings. Slash commands go into the Codex interface; commands such as `git status` go into your ordinary terminal.

The read-only mode is useful for this first inspection. Requests to go beyond the configured boundary may trigger approval; do not approve file changes for this inspection task. The later workspace-write mode permits ordinary project edits and commands without asking about every one. Permissions can vary with installation and managed settings, so inspect the active session. [OpenAI: approvals and sandboxing](#)

Paste this request into the agent:

```
Read AGENTS.md, content/resources/index.md, layouts/all.html, and hugo.toml.  
Do not edit files or run commands that modify the project.
```

```
Explain briefly:
```

1. Which file contains the Resources page's writing?
2. Which shared layout renders it?
3. What project-path prefix does the configured public URL use?
4. Which existing internal links could help a reader get started?

```
Point to the relevant files. If something is missing, say so instead of guessing.
```

Compare the response with the files. It should identify the Resources Markdown file, the shared `all.html` layout, and your configured project path. It should recognise the existing links to the first learning note and notebook project.

If the agent describes a different generator or invents files, correct the working directory or ask it to reread the named sources. Do not build the next task on a mistaken description.

Use `/exit` to return to your terminal. Run `git status` again; inspection should have left no source changes. [OpenAI: CLI commands](#)

First checkpoint: the agent inspected the correct project, and you verified its account of the relevant files.

8.5 Give it one bounded editing task

Restart from the same project root with permission to make local edits:

```
codex --sandbox workspace-write --ask-for-approval on-request
```

A new session should not be assumed to remember the previous conversation. We will provide the full task again. Check `/status`, then paste:

```
Read AGENTS.md and content/resources/index.md before editing.

Task: Help a first-time visitor use the Resources page.
Edit only content/resources/index.md.

Immediately after the introductory paragraph, add a section headed:
## How to use these resources

Add exactly three short bullets, using no more than 70 words in the new section.
- One bullet should explain when to consult the existing Hugo documentation link.
- One should link to the existing first learning note as a practical example.
- One should link to the existing notebook project page for the site's purpose.

Reuse the destinations already present in this file. Do not add external sources.
Preserve the front matter and all existing text and links below the new section.
Do not change layouts, CSS, configuration, dependencies, or workflow files.
Do not stage, commit, push, or deploy.

Run hugo --minify --panicOnError if the installed tools and permissions allow it.
If the check is blocked, report that rather than changing the environment.
Finish by listing changed source files, the check actually run and its result,
and anything I still need to inspect in the browser.
```

Watch the activity. The task authorises an edit to one source file and a local build. Generated output from that build is expected; hand-editing generated files is not. A request to install a framework, change your deployment workflow, or publish the result would not be necessary for this task.

A prompt sets the intended scope. The workspace sandbox is broader than this one-file instruction; it is not a per-file guarantee. Read any approval request in relation to the actual task rather than approving it automatically.

What makes this request useful?

Part	What it contributes
Goal	Help a visitor use existing resources
Named source	Gives the agent a concrete place to inspect and edit
Placement and size	Defines a small, visible result
Existing destinations	Avoids inventing links and extra research

Part	What it contributes
Preservation requirements	Keeps earlier work intact
Verification and stopping point	Separates editing and checking from publication

You could make this edit yourself. That is a strength of the first exercise: the task is small enough for you to understand its entire result.

8.6 Review the files, not just the summary

When the agent finishes, read its report, then use `/exit`. In your normal terminal, run:

```
git status
git diff --name-only
git diff -- content/resources/index.md
git diff --cached
```

You should see one modified source file: `content/resources/index.md`. The staged diff should be empty because the agent was asked not to stage anything. Use status to notice unexpected new files too; an ordinary diff does not show untracked files.

Read every changed line. Check that:

- The new heading and three bullets appear in the requested place.
- The existing front matter, sections, and links remain intact.
- The new links reuse the correct destinations, including their relative-path form.
- The text describes the material accurately and does not invent claims about you.

Then run the build yourself:

```
hugo --minify --panicOnWarning
```

Check its exit result, not merely the presence of output in `public/`; old output can remain after a failed build. Next run `hugo server`, open its reported address, and visit Resources. Read the section and activate its links. Check the first learning note and project page, then return to Resources.

The agent's statement that a build passed is evidence about that command if it actually ran. It does not establish that the prose is accurate, every link works, or the page is easy to use. Ask for the command output when the report is unclear. An agent's second review can help, but it is not an independent guarantee.

One acceptable result for comparison

Your agent may choose different wording. This example shows the intended scale and destinations; it is not a transcript from a recorded agent run:

```
## How to use these resources
```

- Consult the [Hugo documentation](https://gohugo.io/documentation/) when you need details about configuration, content, or templates.
- Read [my first learning note](../articles/first-learning-note/) for a practical editing and checking example.
- Visit [my knowledge notebook project](../projects/Learning-notebook/) to understand this website's purpose.

Preserve the existing introduction above this section and the two original sections below it. There is no need to replace the whole page with the example.

If the result needs correction

Give a precise follow-up such as:

```
Keep the new section, but restore the original text under Website publishing.
That existing text was outside the requested edit. Change only
content/resources/index.md, then show the corrected diff and rerun the build.
Do not stage, commit, push, or deploy.
```

If the session has been closed, start it again in the project and name the file and current issue explicitly. Do not assume an earlier conversation is automatically present.

If you want to discard the entire attempt, inspect status first. For this one previously tracked file, with no other valuable edits and no staged change, use:

```
git restore -- content/resources/index.md
```

If it was staged, use Chapter 6's unstage step first. Inspect unexpected changes separately; restoring this one path does not undo other files or remove newly created files. Avoid a broad cleanup command merely to make status look clean.

8.7 Keep the accepted result, then test your review habit

When the Resources edit meets the requirements, stop the preview and record it yourself:

```
git add content/resources/index.md
git diff --cached
git commit -m "Add guidance for using the notebook resources"
git status
```

If the staged diff includes any unrelated changes, resolve them before committing. The expected checkpoint contains only the accepted Resources edit; `AGENTS.md` was already committed separately.

A link can be wrong even when the build passes

Begin this short experiment from that clean checkpoint. In the **new section only**, change the first-learning-note destination from:

```
../articles/first-learning-note/
```

to:

```
/articles/first-learning-note/
```

Save and build again. Hugo can still build successfully. However, on the project site below `/my-knowledge-site/`, a destination starting with `/articles/` goes to the domain root and omits the project prefix.

Inspect the link's resolved address in your local preview using the configured project path. Compare it with the working article address. Do not push this deliberate mistake.

Restore the isolated experiment:

```
git diff -- content/resources/index.md
git restore -- content/resources/index.md
hugo --minify --panicOnWarning
git status
```

This returns to the accepted section you just committed. Check its link again. You have demonstrated why the review must test a relevant outcome rather than simply accept “build passed”.

Second checkpoint: you accepted an agent-assisted change after checking it yourself, and caught a link problem that building alone did not detect.

8.8 Make one request of your own

Ask the agent to shorten one bullet in the new section while preserving its meaning and destination. Name the file, identify the bullet, and say what counts as an improvement. Reuse the same limits on other files and Git actions.

Review the diff and preview. If the change is useful, commit it yourself. If it is already concise enough or the rewrite loses meaning, keep the earlier version. Successful use of an agent includes deciding not to accept a suggestion.

You can keep the Chapter 8 checkpoint local. When you decide to publish the accepted work, use Chapter 7's `git push`, deployment-status check, and live-page verification yourself. Pushing `main` activates the publishing workflow, so it is a separate decision from accepting a local edit.

Completion check

- ☐ I installed or opened the primary agent client and confirmed my account access.
- ☐ I checked the project location and the session's working permissions.
- ☐ I created and committed concise project guidance in `AGENTS.md`.
- ☐ I verified the agent's initial description of the relevant Hugo files.
- ☐ I gave it a bounded task with clear success criteria.
- ☐ I inspected all changed and newly created files, not just its summary.
- ☐ I built and previewed the Resources change and tested its links.
- ☐ I committed the accepted result myself and repaired the deliberate link mistake.
- ☐ I can make a small follow-up request and decide whether to accept the result.

Keep the method small: supply relevant context, specify a useful outcome, review the changed files, and verify what matters on the page. Skills, plugins, multiple agents, unattended automation, and comparing many models can wait.

Chapter 9 returns to Hugo's content model so that later agent tasks have a clearer structure to work with.

Troubleshooting when you need it

Symptom	Useful next step
<code>codex</code> is not recognised.	Follow the official installation instructions, reopen the terminal, and check <code>codex --version</code> .
Sign-in or account access fails.	Check the intended account and its current access. GitHub sign-in and Codex sign-in are separate.
The agent describes files from another project.	Exit, check the terminal location, and restart in the Hugo repository.
The agent cannot edit during the inspection session.	That session is deliberately read-only. Use the workspace-write session for the editing exercise.

Symptom	Useful next step
A policy prevents using the supplied flags.	Inspect the managed requirements and current official setup guidance. Do not bypass organisational controls.
The agent cannot find Hugo.	Check <code>hugo version</code> in the same environment. A Windows installation and a WSL installation do not automatically share every tool.
The build cannot write its output.	Read the permission error and active writable roots. You can run the build in your normal project terminal and report the result.
The agent changes more than the named source file.	Stop and inspect all changes before staging. Ask for a scoped correction or restore only edits you have identified as unwanted.
The result claims research or checks that are not shown.	Ask what source or command supports the claim and perform the relevant verification yourself.
New files are missing from the diff.	Read <code>git status</code> ; untracked files require separate inspection.
An interrupted session leaves the project uncertain.	Check status and diffs before restarting. Re-state the task, current state, and remaining work.

Repository text, quoted prompts, and retrieved pages can contain instructions unrelated to your task. Treat such material as content to assess, not as authority to change credentials, publish, or expand the job. The reusable project guidance and the current task should remain explicit.

CHAPTER 09

Give Your Hugo Content a Consistent Structure

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Your website has one project page. Adding a second is easy: create another Markdown file and begin writing. After several projects, however, you might notice that one page explains its purpose, another lists tools, and a third never says whether the work has started.

A small shared structure makes those pages easier to write, compare, and maintain. It also gives an AI agent concrete requirements to follow. The structure should answer useful questions without making every page look like a long form.

In this chapter, you will improve the existing notebook project, create a reusable starter, and add a short page describing a planned reading list. Both project pages will carry consistent metadata and useful headings. They will continue to use the layout you already have.

What you will be able to do

By the end, you should be able to:

- Choose a few useful fields and headings for a repeatable kind of content.
- Read and edit the basic YAML values used in Hugo front matter.
- Create a project page from an archetype and distinguish its source path from its title.
- Preview a draft, review its information, and make it reachable from the Projects page.

Start from the completed Chapter 8 project with a clean Git working tree. (If you are following along in the companion repository `ssg-playground`, make sure you are on branch `chapter-08` or check out the completed chapter on branch `chapter-09`.) You need the same Hugo installation and editor; no new service or account is required. The agent review near the end is optional.

9.1 Improve the project you already have

Open `content/projects/learning-notebook/index.md`. Its front matter currently provides a title and a draft setting. Replace just that opening block with:

```

---
title: "My knowledge notebook"
description: "A personal website for learning notes, small projects, and useful references."
draft: false
params:
  status: "in-progress"
  tools:
    - "Hugo"
    - "Markdown"
---

```

Keep your existing body text and links. If you have personalised the title, retain your own title. The example describes the book project; adjust any statement that does not describe your actual work.

Next, update the paragraph under `## Current status` to reflect your progress. For example:

```

This is a work in progress. The site has organised content, a shared layout, and a Git history.
I am making its project descriptions more consistent.

```

Before the final Back to Projects link, add:

```

## Next step

Use the same small set of fields and headings when describing another project.

```

Save, run `hugo server`, and open the notebook project through Projects. You should see the revised paragraph and new heading. Its title, navigation, and existing links should still work.

You will **not** see an automatic status label or tools list. The shared layout currently displays the page title and Markdown body. Storing another value in front matter does not automatically teach that layout to display it. The description is also not automatically added to the HTML head by our minimal starter.

That distinction matters: the source now contains information that later templates can use, while the visible improvement comes from the body text you just edited.

First checkpoint: the existing project has useful metadata and an updated description, and you can explain which changes are visible on the page.

9.2 Decide what every project should explain

A **content model** is an agreement about the information a kind of content should contain. For this site, a project needs a name, a short description, a state of progress, and enough writing to explain the work.

We will use this small agreement:

Location	Field or heading	Our rule
Front matter	<code>title</code>	A readable project name
Front matter	<code>description</code>	One sentence explaining the project's purpose
Front matter	<code>draft</code>	Whether this page is still being prepared for publication
Under <code>params</code>	<code>status</code>	One of <code>planned</code> , <code>in-progress</code> , or <code>complete</code>
Under <code>params</code>	<code>tools</code>	A list of relevant tools; an empty list is acceptable
Body	Purpose	What the project is intended to achieve
Body	Current status	What has actually happened so far
Body	What I have learned	Specific learning, or an honest statement that work has not begun
Body	Next step	One concrete action, or a clear completion statement

The three status values are **our editorial choices**, not a built-in Hugo workflow. Hugo will not reject a fourth value merely because this table excludes it. We must review the values ourselves until we deliberately add validation later.

The table is also a scope limit. We do not need an owner field when every project belongs to the same author, a budget for a reading list, or a percentage complete that nobody can measure reliably.

Keep publication state separate from project progress

`draft` describes the **page**. `status` describes the **project**.

A page can be ready to publish even when its project is only planned. Conversely, a completed project can have an unfinished page that still needs editing. For example:

```
draft: false
params:
  status: "planned"
```

This combination is reasonable: readers can see an honest description of a plan. It does not claim that the planned work is finished.

Separate quick facts from explanation

The status value gives a short answer that a future layout can display or use for grouping. The Current status paragraph explains the evidence behind it. They serve different purposes, but must agree.

Avoid putting several sentences into `status`, or hiding the project's entire story in metadata. Keep the explanation in Markdown, where headings, links, and paragraphs are convenient to read and edit.

Likewise, an article and a project do not need identical models. A learning note might need a question and an explanation rather than a project status. We will improve one content family at a time.

9.3 Read only the YAML you need

Our pages already use YAML front matter between two `---` lines. This section explains the few shapes introduced by the project model. We will keep using YAML for these pages and TOML for `hugo.toml`; the formats do not need to match. Hugo supports several front matter formats. [Hugo: front matter](#)

A named value

```
title: "My knowledge notebook"
```

`title` is the key; the quoted text is its value. Keep the colon and the space after it. The quotation marks are straight quotes, not typographic opening and closing quotes copied from a word processor.

Use quotes for the text fields in this chapter. They make the boundaries clear, especially when a title contains a colon:

```
title: "Reading list: website publishing"
```

If text contains a double quotation mark, you can put the whole value in single quotes instead, provided the text does not itself require handling an apostrophe:

```
title: 'Notes on "static" websites'
```

You do not need every YAML quoting rule to complete this project. Follow the simple examples and use your editor's error messages when a value becomes more complicated.

A true-or-false value

```
draft: false
```

`false` is a Boolean value. Keep `true` and `false` lowercase and unquoted. Writing `"false"` gives the YAML reader text instead of a Boolean; relying on later conversion makes the source less clear.

The distinction is useful beyond this one field. A name is text, a yes-or-no setting is a Boolean, and several tools form a list.

Related values under one key

```
params:
  status: "in-progress"
  tools:
    - "Hugo"
    - "Markdown"
```

Indentation describes the relationships. `status` and `tools` belong under `params`; the two list entries belong under `tools`. Use spaces, with two spaces for each level shown here. Do not use tabs for indentation.

`params` is a mapping: it groups named values. The dash-prefixed entries form a list. To add another relevant tool later, add another entry at the same indentation. To record that no tools have been chosen, use:

```
params:
  status: "planned"
  tools: []
```

An empty list is clearer than inserting a tool merely to fill a space. It also preserves the expected value shape for later template code.

Hugo provides recognised fields such as `title`, `description`, and `draft`. Put our custom project information under `params`, following Hugo's documented convention. Do not add a second `params` block if one already exists; add the new values within the existing block. [Hugo: custom page parameters](#)

A short source-reading check

Before continuing, point to the value that answers each question in your notebook project:

1. What should the page be called?
2. Is the page ready for an ordinary build?
3. Is the project finished?
4. Which tools does the page record?

If you can answer by reading the front matter, you know enough YAML for the exercise. More formats and larger data collections belong in Chapter 12.

9.4 Keep the page structure and address predictable

Chapter 3 introduced two filenames that look almost the same. They still have different jobs:

Source	Job in our site
<code>content/projects/_index.md</code>	Introduces the Projects section and contains its manual links
<code>content/projects/learning-notebook/index.md</code>	Describes one project
<code>content/projects/reading-list/index.md</code>	Will describe our second project

The folder containing an `index.md` is a **leaf bundle**. It groups one page with any resources that belong to it. A bundle can start with only its Markdown file; an image is not required. The section uses `_index.md` because it can contain project pages beneath it. [Hugo: page bundles](#)

Do not rename `content/projects/_index.md` to `index.md` to make the names look consistent. Their different names express a useful structural difference.

Under our current configuration, the new folder will give the second project an address ending in:

```
/projects/reading-list/
```

On the GitHub project site, that follows `/my-knowledge-site/`. The page title can be “My website reading list” while its folder remains `reading-list`. Changing the title alone will not change this folder-derived address in our starter.

Keep folder names short, lowercase, and separated with hyphens. Avoid changing an established folder name just because you improve a title; existing links may rely on its address. Hugo supports explicit URL overrides, but we do not need them for this exercise.

If a project later needs a screenshot, place it beside that project’s `index.md` and use the relative image-link technique from Chapter 2. Keep site-wide assets such as the shared stylesheet in their existing location.

9.5 Make a reusable starter with an archetype

Copying an old project page can also copy its facts, links, and completion claims. A starter can provide the useful structure without carrying over another project’s story.

Hugo calls a new-content starter an **archetype**. Create a folder named `archetypes` beside `content`, then create `archetypes/projects.md`. If that file already exists in your project, inspect it and adapt it instead of replacing unrelated work.

Use this complete starter:

```

---
title: "Replace with a project name"
description: "Replace with one sentence about the project's purpose."
draft: true
params:
  status: "planned"
  tools: []
---

## Purpose

Explain what this project is intended to achieve.

## Current status

Describe what has actually happened so far.

## What I have learned

Record a specific lesson, or explain that work has not begun.

## Next step

Name one concrete action.

[Back to Projects](../)

```

This starter is deliberately plain. It supplies text and headings without generating dates, guessing titles, or introducing template expressions. You will replace its prompts before publishing.

Archetypes can supply both front matter and body content. They are used when new content is created; editing an archetype does not rewrite pages you already created from it. That is why we updated the notebook project separately. [Hugo: archetypes](#)

Create the second project

Stop the preview with **Ctrl+C**. From the project root, run:

```
hugo new content --kind projects projects/reading-list/index.md
```

The command asks Hugo to use the `projects` archetype and create a file beneath `content/`. Do not add a second `content/` prefix to the path shown. The `--kind` option selects the archetype for this creation step; it is not our project's progress status. [Hugo: new content command](#)

Open `content/projects/reading-list/index.md`. Confirm that it contains the starter's fields and headings. You should see `draft: true`, an empty tools list, and prompts to replace.

If the destination already exists, inspect the existing file. Do not use an overwrite option to force the command through. This exercise assumes that the second project has not yet been created.

Turn the prompts into useful content

Replace the generated file with this complete example, or equivalent truthful wording about your own planned reading list:

```
---
title: "My website reading List"
description: "A planned collection of resources for Learning website publishing."
draft: true
params:
  status: "planned"
  tools:
    - "Markdown"
---
```

Purpose

I plan to collect a small set of resources that help me learn website publishing.

Current status

This is a proposal. I have not yet selected or reviewed the resources.

What I have learned

The reading work has not begun. I will record useful findings as I review each resource.

Next step

Choose three resources and write one sentence explaining why each belongs in the list.

I will begin with the references on the [\[Resources page\]\(../../resources/\)](#).

[\[Back to Projects\]\(../\)](#)

Here, `Markdown` is a planned tool. For our model, the tools list records intended tools while a project is planned, and actual tools once work begins. Update it as the project changes.

Notice that the example does not claim to have read anything. Creating a page about a plan is not evidence that the plan has been carried out.

9.6 Preview the draft, then connect it to Projects

Start a preview that includes drafts:

```
hugo server -D
```

Use the address Hugo reports. Open the new project's route by adding `projects/reading-list/` to that base address. With the earlier project-site configuration and the usual port, this will normally be:

```
http://localhost:1313/my-knowledge-site/projects/reading-list/
```

Use your actual base path and port. The page is not yet linked from Projects, so opening the route directly is expected.

Read the whole page. Follow Resources and Back to Projects. Confirm that both stay within the configured site path. The relative destinations are written for a page two levels below the site's root, just like the existing notebook project.

The `-D` option includes draft pages in this preview. It does not edit the front matter or mark the page ready. Ordinary builds exclude drafts unless configured otherwise. [Hugo: draft field](#)

Make a deliberate publication decision

When the writing is ready, change only:

```
draft: false
```

Leave `status: "planned"` as it is. The page is ready; the reading project is still a plan.

Stop the server, then restart with the normal command:

```
hugo server
```

Reopen the page and check that it is available without draft inclusion. Restarting makes it clear which preview settings you are using.

Now open `content/projects/_index.md`. Preserve its introduction and existing project link. Under `## Current work`, add:

```
- [My website reading list](reading-list/): A planned collection of resources for learning webs  
  ite publishing.
```

Visit Projects through the navigation. Both project links should work, and each project should link back to Projects. The list is still manual: creating a page or adding metadata does not insert a link into it. Chapter 10 will introduce the template tools needed to automate a list.

If you prefer to keep your page as a draft, keep `draft: true` and postpone adding this public-facing link. A manually written link can point to a page that an ordinary build excludes.

A build can leave output from an earlier run in place. Changing a previously generated page back to a draft is therefore not a reliable way to withdraw an already published copy; removal needs a separate output and deployment check.

A draft setting also does not make source private. If you push a draft Markdown file to a public GitHub repository, its text remains readable in that repository even when the website omits the page.

Second checkpoint: a second project uses the agreed structure, appears in the normal preview, and is reachable through Projects without claiming that its planned work is complete.

9.7 Find and repair a front matter mistake

Stop the server. In the new reading-list page, temporarily remove the closing quote from the title line:

```
title: "My website reading list
```

Leave the rest of the file untouched. Run:

```
hugo --minify --panicOnWarning
```

The build should fail because the quoted value is unfinished. Read the error and identify the named source file. Its reported line may be where parsing became impossible rather than the exact place where you removed the quote.

Restore the correct line:

```
title: "My website reading list"
```

Build again. The successful build tells you that Hugo can process the repaired source. It does not establish that the project description is accurate or that its status follows our editorial agreement.

For example, `status: "planned"` is valid quoted text but misspells our chosen value. Likewise, a tools list can contain an invented claim without causing a build error. There are two different checks: whether the file can be processed, and whether its contents meet the model and describe reality.

Before saving the checkpoint, compare both project pages with the table in Section 9.2. Check that the status values use the agreed spelling and that the paragraphs support them. Read the new page for leftover starter prompts.

Do not judge a failed build by looking for an HTML file left in `public/`. Output from an earlier successful build may remain. Read the current command result and repair the source.

9.8 Review, make one choice, and save

For a small independent variation, choose one real next action for your reading-list project and rewrite its Next step paragraph in your own words. Make the action specific enough that you can later tell

whether you did it.

Then examine the tools list. Keep Markdown if it is relevant, choose a different actual plan, or use an empty list if undecided. Do not add fields simply to make the page look more sophisticated.

This is a modest exercise in modelling: decide what belongs in an existing field, retain its expected shape, and keep the explanatory writing consistent with it.

Optional: ask the agent to check the agreement

Using Chapter 8's read-only inspection approach, give the agent this request:

Read AGENTS.md and these files:

- content/projects/learning-notebook/index.md
- content/projects/reading-list/index.md
- archetypes/projects.md

Do not edit files or run commands that modify the project.

Our project model requires title, description, draft, params.status, and params.tools. Status must be planned, in-progress, or complete. Tools must be a list, which can be empty. The body should contain Purpose, Current status, What I have learned, and Next step.

Check both project pages against that agreement. Identify missing fields, wrong value shapes, leftover starter prompts, and contradictions between status and body text. Treat the archetype's prompts as intentional. Report file-specific findings. Do not invent facts or claim to verify real-world progress from these files alone.

Read the cited lines yourself. If the agent suggests a correction, decide whether it is supported before editing. It can compare the files with an explicit agreement; it cannot establish that you performed work outside those files.

Save the completed chapter

Run the ordinary build and inspect status:

```
hugo --minify --panicOnError  
git status  
git diff
```

The intended changes are the existing notebook project, the Projects section, the new archetype, and the new reading-list page. Remember that `git diff` alone does not display untracked files. Open both new files before staging them.

Stage only the chapter's four paths:

```
git add content/projects/learning-notebook/index.md content/projects/_index.md archetypes/projects.md content/projects/reading-list/index.md
git diff --cached
```

Check the complete staged result, including the new files. When it matches what you reviewed, commit:

```
git commit -m "Define a consistent project model and add a reading-list project"
git status
```

The working tree should be clean. Publishing remains the separate push-and-check process from Chapter 7; the local checkpoint is sufficient for continuing the book.

Completion check

- ☐ Both projects use the agreed fields and body headings.
- ☐ I can distinguish page readiness from project progress.
- ☐ I can read a text value, a Boolean, a mapping, and a list in the front matter.
- ☐ I created the second project from the projects archetype and replaced its prompts.
- ☐ I understand why changing an archetype does not update existing pages.
- ☐ I checked the second page in the normal preview and tested its links.
- ☐ I repaired the deliberate YAML mistake and reviewed the actual information.
- ☐ I inspected and committed the four intended source files.

Chapter 10 will use these consistent fields in Hugo templates. We will begin with small expressions and conditions, then use a loop to turn a collection of pages into a useful list.

Troubleshooting when you need it

Symptom	Useful next step
Status, tools, or description do not appear on the page.	Our current layout does not display them. Check the source; template use comes next.
Hugo reports a YAML parsing error.	Check quotes, colons, indentation, and the two front matter delimiters in the named file.
The new command produces a different starter.	Check <code>archetypes/projects.md</code> , the project root, and the <code>--kind projects</code> option.
The command says the destination exists.	Inspect that file rather than overwriting it. You may already have completed the creation step.

Symptom	Useful next step
The new page is absent from a normal preview.	Check <code>draft</code> , the filename, and the address. Use <code>-D</code> only when intentionally previewing drafts.
The page opens directly but is missing from Projects.	The section list is manual. Add its link after the page is ready.
A link leaves the GitHub project path.	Compare its relative destination with the examples and inspect the resolved address.
Editing the archetype does not change an old page.	That is expected; update existing content separately.
A build passes despite an incorrect project status.	Review the values against our model. No custom status validator has been installed.

CHAPTER 10

Use Hugo Templates to Display Your Content

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Your two project pages now contain useful information in their front matter. The description, status, and tools are organised, but visitors cannot see them yet. You also maintain a separate list of project links by hand.

A template can use those existing values to produce the page. Change a project's description once, and Hugo can use it both on the project page and in the Projects list.

We will make that happen in small steps. First, display a description. Next, show status and tools when they are available. Finally, let Hugo generate the Projects list from the pages in that section.

What you will be able to do

By the end, you should be able to:

- Read simple template expressions and connect their values to content files.
- Use `if` and `with` to display information conditionally.
- Use `range` to display a list, while tracking what the dot represents.
- Build an automatic Projects list and diagnose a common template error.

Start from the completed Chapter 9 checkpoint. (If you are following along in the companion repository `ssg-playground`, make sure you are on branch `chapter-09` or check out the completed chapter on branch `chapter-10`.) Both project pages should be ready for normal builds, with `draft: false`, and the Git working tree should be clean. If you deliberately retained the reading-list page as a draft, finish reviewing it before following the two-project examples below.

Use the same Hugo installation. These exercises continue the `layouts/all.html` convention used since Chapter 1 and tested with Hugo 0.150.0. We will edit that existing layout and the Projects section introduction. No new package, theme, or programming-language installation is required.

10.1 Display one value before learning more syntax

Open `layouts/all.html`. Inside its `article` element, find:

```
<h1>{{ .Title }}</h1>
{{ .Content }}
```

Insert the following block between those two lines:

```
{{ if .Description }}
  <p>{{ .Description }}</p>
{{ end }}
```

The article should now begin:

```
<article>
  <h1>{{ .Title }}</h1>
  {{ if .Description }}
    <p>{{ .Description }}</p>
  {{ end }}
  {{ .Content }}
</article>
```

Save and run `hugo server`. Open the notebook project through Projects. Its one-sentence description should appear between the title and Purpose heading. Open the reading-list project and check its description too.

Then visit About. If you have followed the supplied examples, that page has no description field, so the new paragraph should be absent. The title and ordinary Markdown content should still appear.

The three added lines mean: if this page has a non-empty description, put that description inside a paragraph. `end` closes the conditional block. The HTML opening and closing paragraph tags belong inside it, so a missing description does not leave an empty paragraph behind.

We have made the first useful template change. The same instruction serves both project pages because Hugo evaluates it separately for each page.

First checkpoint: descriptions appear where they are provided, and pages without descriptions still render normally.

10.2 Understand the HTML and template instructions together

Hugo templates use Go's template language. You do not need to learn the Go programming language or install its compiler for these exercises. Hugo already contains the template-processing machinery. [Hugo: introduction to templating](#).

Consider the line you have used since Chapter 1:

```
<h1>{{ .Title }}</h1>
```

The `h1` tags describe an HTML heading. The action between `{{` and `}}` asks Hugo for a value. When rendering the reading-list page, Hugo produces a heading such as:

```
<h1>My website reading list</h1>
```

The browser receives the completed HTML. It does not execute `.Title`, and this feature does not require JavaScript. Editing a template changes what Hugo generates during the next build.

Read the values already in our layout

Expression	Meaning at the outer level of this page layout
<code>.Title</code>	The current page's title
<code>.Description</code>	Its description from front matter
<code>.Content</code>	Its body rendered from Markdown into HTML
<code>.Params.status</code>	Its custom <code>status</code> parameter
<code>.Site.Title</code>	The site's title from configuration

The opening dot means **the current context**: the value or object the template is currently working with. At the outer level of this layout, that is the page Hugo is rendering.

Keep the spelling and capitalisation shown in the examples. Hugo's page methods use names such as `Title` and `Description`; our custom parameter key remains lowercase `status`.

You do not have to memorise every available method. For each new expression, ask which source value it should read and which visible result it should produce.

Conditions do not need a value to be printed

In the description block, `if .Description` checks whether the value is non-empty. The separate `{{ .Description }}` prints it. `if` itself does not change the current context.

Empty text, an empty list, and `false` all count as false in these conditions. A non-empty description counts as true. This is sufficient for the fields we are using. [Hugo: if](#)

Recognise a function and a pipe

The footer and navigation already contain this pattern:

```
<a href="{{ "about/" | relURL }}">about this notebook</a>
```

"about/" is a text value. `relURL` is a function. The pipe character, `|`, passes the value on its left to the function on its right. Here, Hugo forms a URL using the site's configured base path.

For the book's GitHub project site, the result includes `/my-knowledge-site/about/`. Keep the input as "about/"; a leading slash would request a root-relative path and change the result. [Hugo: relURL](#)

Later in this chapter, we will obtain URLs directly from page objects. Use the value appropriate to the task: a known site path for the existing navigation, or a page's own URL when listing pages.

10.3 Use `with` when a value is optional

The description block checks `.Description` and then names it again. `with` provides a convenient alternative. Replace that entire `if` block with:

```
{{ with .Description }}
  <p>{{ . }}</p>
{{ end }}
```

Save and confirm that the visible result is unchanged.

For a non-empty description, `with` enters its block and makes the description the current context. Inside that block, the dot is the description text. After `end`, the previous page context is restored. [Hugo: with](#)

This is why the inner expression is `{{ . }}`. Writing `.Description` there would try to ask the description text for another description. We will deliberately test that kind of mistake later.

Now add a separate block immediately after the description block and before `.Content`:

```
{{ with .Params.status }}
  <p><strong>Status:</strong> {{ . }}</p>
{{ end }}
```

The notebook project should show `Status: in-progress`; the reading-list project should show `Status: planned`. Other pages without that parameter should show neither the label nor an empty paragraph.

We are displaying the stored status value directly. Turning `in-progress` into a more polished label is a possible later improvement. Keep the stored values consistent with Chapter 9's agreement.

Follow the dot through one block

Position	What <code>.</code> represents
Before <code>with .Params.status</code>	The page being rendered
Inside that <code>with</code> block	Its status text, such as <code>planned</code>
After the matching <code>end</code>	The page again

Keep the description and status blocks beside one another. The status block must start **after** the description block's `end`, so it can read `.Params` from the page.

The condition only establishes that a value is present. It does not verify that the status is one of our agreed values or that the project's reported progress is true. The content review from Chapter 9 still matters.

10.4 Repeat markup for the tools list

A status is one value; tools are a list. The notebook currently records Hugo and Markdown. We want one list item for each tool, without writing a fixed number of HTML elements.

Add this block after the status block and before `.Content`:

```
{{ with .Params.tools }}
  <p><strong>Tools:</strong></p>
  <ul>
    {{ range . }}
      <li>{{ . }}</li>
    {{ end }}
  </ul>
{{ end }}
```

Save and preview both project pages. The notebook should list Hugo and Markdown. The reading-list project should show whatever truthful list you retained in Chapter 9.

`range` repeats its block for the elements of a collection. Inside each repetition, the dot refers to that element. Here, the elements are tool names, so `{{ . }}` prints one name. [Hugo: range](#)

There are two nested blocks to keep track of:

Position	What <code>.</code> represents
Before <code>with .Params.tools</code>	The page
Inside <code>with</code> , before <code>range</code>	The tools list

Position	What <code>.</code> represents
Inside <code>range</code>	One tool name
After the inner <code>end</code>	The tools list again
After the outer <code>end</code>	The page again

The outer `with` also prevents an empty Tools label and list when `tools: []` is stored. Indentation makes the two closing `end` actions easier to match with their opening blocks.

For a quick check, temporarily change the reading-list project's tools value to `tools: []`, preserving the rest of its front matter. The Tools label and list should disappear. Restore the original list afterwards, so this experiment leaves no content change.

These blocks also work on any other page that deliberately supplies the same parameters. We are using them for projects because that is the content family with this agreed model.

10.5 Generate the Projects list from its pages

We can use `range` for a collection of pages as well as a collection of tool names. Each element will then provide a title, description, and URL.

In `layouts/all.html`, insert the following block **after** `{{ .Content }}` and **before** `</article>`:

```
{{ if and .IsSection (eq .Section "projects") }}
  <h2>Current work</h2>
  <ul>
    {{ range .RegularPages.ByTitle }}
      <li>
        <a href="{{ .RelPermalink }}">{{ .Title }}</a>
        {{ with .Description }}
          <p>{{ . }}</p>
        {{ end }}
      </li>
    {{ else }}
      <li>No projects to display yet.</li>
    {{ end }}
  </ul>
{{ end }}
```

For the moment, Projects may display both the old manual list and the new automatic one. That is an expected intermediate result. We will remove the old list after checking the new one.

Limit the change to the intended section

The first line combines two checks:

- `.IsSection` asks whether this is a section page.
- `eq .Section "projects"` asks whether its top-level section is `projects`.

`eq` means equal, and `and` requires both checks to be true. The parentheses group the equality check. This is template syntax, so we do not replace these words with operators from another language.

In our shallow structure, the condition selects the Projects landing page. The notebook page belongs to Projects but is not itself a section page, so it does not receive the automatic project list. Articles is a section page, but its section name differs. [Hugo: IsSection](#), [Hugo: Section](#)

If you later introduce nested project sections, revisit this condition: those sections can satisfy it too. We are solving the structure currently in the book.

Choose and order the collection

On this section page, `.RegularPages` supplies the regular pages directly in the section. It does not recursively collect pages inside further subsections. `.ByTitle` sorts the resulting collection by title in ascending order. [Hugo: RegularPages](#), [Hugo: ByTitle](#)

With the supplied titles, the notebook comes before the reading list. The filesystem's folder order does not control this list.

Inside `range`, the dot is now one project page. That is why `.Title` names the project rather than repeating the landing page's title, Projects.

The inner `with .Description` temporarily changes the dot to that project's description text. When its `end` is reached, the project page becomes current again. The outer `range` then moves on to the next project.

`else` handles an empty collection. In that case, it produces the explanatory list item instead of project entries. We will test this briefly before the final checkpoint.

Let Hugo supply the page URL

`.RelPermalink` provides the current project's relative permalink, including the configured base path. For the reading-list project, our published site uses a path like:

```
/my-knowledge-site/projects/reading-list/
```

We do not assemble that URL from the title or manually prepend the repository name. Use the returned value directly in `href`; it already represents the page's location. [Hugo: RelPermalink](#)

10.6 Remove the duplicate list and verify the result

Open `content/projects/_index.md`. Keep the front matter and introduction. Remove the old `## Current work` heading and its two manually written project bullets. For the supplied example, the complete file becomes:

```
---
title: "Projects"
draft: false
---
```

Small projects I am developing, with notes about their purpose and progress.

Retain your personalised introduction if you wrote one. Do not remove the Markdown file: its title and introductory content still belong to the section page.

Save and refresh Projects. You should now see one Current work heading and one list. Its descriptions come from the project pages' front matter, so the notebook's wording may differ from the former hand-written bullet.

Follow both links and return through Back to Projects. Check About, Articles, and Resources as well. The new list should not appear there, and their existing content should remain intact.

Check that the list responds to content

Temporarily change the reading-list project's description by adding a short phrase. Save and inspect both the project page and the Projects list. Both should reflect the edit. Restore the original description afterwards.

Next, stop the preview and temporarily set both projects to `draft: true`. Start a normal `hugo server` without `-D`, and refresh the **Projects landing page**. The generated list should show its empty message. Stop the server, restore both draft values to `false`, and restart normally. Both entries should return.

Check the newly rendered landing-page list during this experiment. As Chapter 9 explained, old generated project files can remain, so directly opening an old project URL is not a reliable draft-exclusion test.

Use an ordinary preview for the final review. A preview started with `-D` intentionally includes draft projects in the generated list.

Second checkpoint: Projects draws its titles, descriptions, and links from the project pages, and its list changes when the included content changes.

10.7 Diagnose a context mistake

Stop the server. In the first description block, near the top of the article, temporarily change:

```
{{ with .Description }}
  <p>{{ . }}</p>
{{ end }}
```

to:

```
{{ with .Description }}
  <p>{{ .Title }}</p>
{{ end }}
```

Run:

```
hugo --minify --panicOnWarning
```

The build should fail when rendering a page with a description. The error should point into the template and indicate that `Title` cannot be evaluated on the current text value. Exact wording and the page reported first may vary.

The project still has a title. The problem is where we asked for it. Inside `with .Description`, the dot is description text, not the project page.

Restore `{{ . }}` in that paragraph and build again. This is a small source repair; replacing the whole layout would discard working changes unnecessarily.

When reading a template error, ask three questions:

1. Which file and expression does Hugo identify?
2. What does the dot represent at that location?
3. Does the requested value belong to that context?

For an unmatched-block error, also pair each `if`, `with`, and `range` with its own `end`. The reported location can be after the original mistake, especially when a closing action is missing.

Hugo normally escapes ordinary text values appropriately when inserting them into HTML. Do not add `safeHTML` as a general repair for an error or unexpected output. Our descriptions are plain text; the existing `.Content` value is the rendered Markdown body. These are different kinds of input.

10.8 Make one small template change yourself

Add a status paragraph to each project entry in the automatic list, after its description. It should display that project's recorded status and omit the entire paragraph when the value is absent.

Use the status block from Section 10.3 as a starting point. Place it inside the page `range`, after the description's `end`. Before typing, predict what the dot represents there and why `.Params.status` should work.

Build and preview. Confirm that the notebook entry reports `in-progress` and the reading-list entry reports `planned`. A repeated Projects title, a missing label, or a context error means you should inspect placement before changing the content files.

This variation leaves the content model unchanged. You are choosing another place to display information already stored once in each project page.

Optional: ask an agent to explain your result

Use Chapter 8's read-only approach and ask:

```
Read AGENTS.md, layouts/all.html, content/projects/_index.md,
and the two project index.md files beneath content/projects/.
Do not edit files or run commands that modify the project.
```

```
Explain how the automatic Projects list works. Identify the condition
that selects its landing page, the collection and sort order, the source
of each link, and what the dot represents inside range and with.
```

```
Check for duplicate manual links and context mistakes. Cite the relevant
expressions. Do not claim that browser checks or a build were performed
unless you actually performed them with permission.
```

Compare the explanation with your own reading. The agent should be able to trace the values, rather than merely say that the template looks correct.

Save the checkpoint

Run:

```
hugo --minify --panicOnWarning
git status
git diff
```

The intended persistent changes are `layouts/all.html` and `content/projects/_index.md`. The temporary description, tools, and draft experiments should leave no changes in the individual project files.

Inspect the diff, then stage and review those two paths:

```
git add layouts/all.html content/projects/_index.md
git diff --cached
```

When the staged changes match the reviewed result, commit:

```
git commit -m "Display project metadata and generate the Projects list"
git status
```

Keep this checkpoint locally or publish it through Chapter 7's reviewed push-and-check workflow.

Completion check

- ☐ I can connect the displayed description, status, and tools to their source values.
- ☐ Missing optional values do not leave empty labels or lists.
- ☐ I can explain how `if`, `with`, and `range` differ.
- ☐ I can identify the current context inside each nested block.
- ☐ Projects has one automatic list with working links and the intended order.
- ☐ I checked the empty collection and restored both projects to normal builds.
- ☐ I repaired the deliberate context error and added status to the list entries.
- ☐ I committed only the two intended source files.

Chapter 11 will organise this growing layout into reusable pieces. The expressions and context rules will remain the same; we will change how the templates are arranged so shared page structure is easier to maintain.

Troubleshooting when you need it

Symptom	Useful next step
A field prints nothing.	Check the source key, spelling, and current context.
A field error mentions a string.	Look for a surrounding <code>with</code> or <code>range</code> that changed the dot to text.
Hugo reports an unexpected end of the template.	Match every opening block with an <code>end</code> ; use indentation to reveal nesting.
The Projects heading or links appear twice.	Remove the old heading and manual bullets from the section's Markdown body.
The list appears on individual projects.	Restore the <code>.IsSection</code> check around the list.
The list is empty unexpectedly.	Check project draft values, their <code>index.md</code> paths, and whether they are directly in this section.

Symptom	Useful next step
A list link omits the project prefix.	Use <code>.RelPermalink</code> from the project page instead of building the URL manually.
Changing a Markdown description has no effect.	Edit the front matter <code>description</code> field used by the template, save, and check the build output.
An experiment has left unexpected Git changes.	Inspect those exact files and restore the temporary values before staging.

Article block for comparison

This is the core result before the independent status-in-the-list variation. Use it to compare the `article` element inside `layouts/all.html`. Keep the rest of your layout, including the navigation, skip link, stylesheet reference, and footer.

```

<article>
  <h1>{{ .Title }}</h1>

  {{ with .Description }}
    <p>{{ . }}</p>
  {{ end }}

  {{ with .Params.status }}
    <p><strong>Status:</strong> {{ . }}</p>
  {{ end }}

  {{ with .Params.tools }}
    <p><strong>Tools:</strong></p>
    <ul>
      {{ range . }}
        <li>{{ . }}</li>
      {{ end }}
    </ul>
  {{ end }}

  {{ .Content }}

  {{ if and .IsSection (eq .Section "projects") }}
    <h2>Current work</h2>
    <ul>
      {{ range .RegularPages.ByTitle }}
        <li>
          <a href="{{ .RelPermalink }}">{{ .Title }}</a>
          {{ with .Description }}
            <p>{{ . }}</p>
          {{ end }}
        </li>
      {{ else }}
        <li>No projects to display yet.</li>
      {{ end }}
    </ul>
  {{ end }}
</article>

```

CHAPTER 11

Build Reusable Hugo Layouts

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Your shared layout now does several jobs. It creates the HTML document, displays navigation and a footer, presents page metadata, and generates the Projects list. It works, but finding the right place for the next change takes more care.

We will give those jobs clearer homes. A base template will hold the common document structure. Small partial templates will hold reusable pieces. A Projects section template will decide where the project list belongs.

Most of the website should look exactly as it did before. That is the first result to check: reorganising working code should preserve its behaviour. At the end, you will make one small change that belongs only to the Projects landing page.

What you will be able to do

By the end, you should be able to:

- Move a working page component into a partial and pass it the context it needs.
- Connect a base template's named block with a matching definition.
- Give the Projects section its own layout while retaining shared page structure.
- Check that reorganising templates has preserved content, links, and metadata.

Start from the completed Chapter 10 checkpoint, including status labels in the automatic project list. (If you are following along in the companion repository `ssg-playground`, make sure you are on branch `chapter-10` or check out the completed chapter on branch `chapter-11`.) Both projects should have `draft: false`, and Git should report a clean working tree. Use the same Hugo installation and stylesheet.

This chapter uses the template paths introduced in Hugo 0.146.0, including `layouts/baseof.html` and `layouts/_partials/`. The examples are checked with Hugo 0.150.0. Older tutorials may show `_default` or an unprefixed `partials` directory; follow this book's paths consistently rather than combining arrangements. [Hugo: template-system changes](#)

11.1 Move one familiar component first

Before editing, run the normal preview and visit Home, Projects, and the notebook project. Notice the title, navigation, footer, project metadata, and list entries. These are your comparison points.

Stop the preview with **Ctrl+C**. Inside `layouts`, create a folder named `_partials`, including the leading underscore. Create `layouts/_partials/footer.html`.

Move the complete footer element from `layouts/all.html` into the new file. For the supplied project, it is:

```
<footer>
  <p>Learn, review & share.</p>
  <p class="footer-note">
    Read <a href="{{ "about/" | relURL }}">about this notebook</a>.
  </p>
</footer>
```

Retain your own accurate footer wording if you personalised it. The new file should contain the footer element, without a second HTML document around it.

In the old footer's position in `layouts/all.html`, put:

```
{{ partial "footer.html" . }}
```

Save both files, then run:

```
hugo --minify --panicOnWarning
hugo server
```

Visit the same three pages. The footer should still appear once on each page, with a working About link and the same styling. If you see two footers, the original element was copied but not removed from `all.html`.

A **partial** is a template you call to render a piece of a page. The filename in this call is relative to `layouts/_partials/`; do not write the full filesystem path there. The final dot passes the current context to the partial. [Hugo: partial inclusion](#)

Our footer currently uses fixed wording and `relURL`, so it does not need page-specific values. Passing the page explicitly keeps the calling pattern clear and allows later changes to use it.

First checkpoint: the footer has its own source file, and the rendered pages still show the same single footer.

11.2 Give the remaining jobs clear locations

We will finish with these six template files:

File	Responsibility
<code>layouts/baseof.html</code>	HTML document, head, navigation, main region, and footer call
<code>layouts/all.html</code>	General page content when no more specific layout applies
<code>layouts/projects/section.html</code>	Projects landing-page content and project-list call
<code>layouts/_partials/footer.html</code>	The complete footer element
<code>layouts/_partials/page-meta.html</code>	Optional description, status, and tools
<code>layouts/_partials/project-list.html</code>	The Current work heading and generated project list

There are more files, but each has a named purpose. A future change to the project list will have an obvious starting point. A new page layout can share the document structure without copying the navigation and footer.

This is often called **refactoring**: changing code organisation while preserving intended behaviour. It does not mean that every paragraph or HTML element needs its own file. We will keep the header and navigation together in the base template for now.

The content files remain the source of titles, descriptions, project facts, and Markdown writing. Moving the templates does not require moving that content or changing its URLs.

11.3 Extract metadata and the project list

Stop the preview before this group of edits. We will save each connected set of changes before building again.

Put page metadata in one partial

Create `layouts/_partials/page-meta.html` with the three blocks introduced in Chapter 10:

```

{{ with .Description }}
  <p>{{ . }}</p>
{{ end }}

{{ with .Params.status }}
  <p><strong>Status:</strong> {{ . }}</p>
{{ end }}

{{ with .Params.tools }}
  <p><strong>Tools:</strong></p>
  <ul>
    {{ range . }}
      <li>{{ . }}</li>
    {{ end }}
  </ul>
{{ end }}

```

In `layouts/all.html`, replace those three blocks between the `h1` and `.Content` with:

```

{{ partial "page-meta.html" . }}

```

Leave the heading and `.Content` in place. This partial expects to receive a page, so its opening expressions can read `.Description` and `.Params`.

Put the generated list in its own partial

Create `layouts/_partials/project-list.html`:

```

<h2>Current work</h2>
<ul>
  {{ range .RegularPages.ByTitle }}
    <li>
      <a href="{{ .RelPermalink }}">{{ .Title }}</a>
      {{ with .Description }}
        <p>{{ . }}</p>
      {{ end }}
      {{ with .Params.status }}
        <p><strong>Status:</strong> {{ . }}</p>
      {{ end }}
    </li>
  {{ else }}
    <li>No projects to display yet.</li>
  {{ end }}
</ul>

```

This preserves the list-entry status from Chapter 10's independent exercise. It also keeps the title order, page-generated links, optional descriptions, and empty-list message.

In `all.html`, replace the existing Projects conditional and everything inside it with:

```
{{ if and .IsSection (eq .Section "projects") }}
  {{ partial "project-list.html" . }}
{{ end }}
```

The condition still decides whether to show the list. The partial now handles how the list is rendered. Pass the **section page**, because the partial obtains its collection from `.RegularPages`.

Build and preview again. Check the project pages' metadata and the Projects list. The visible result should match Chapter 10.

Context crosses the call explicitly

The dot inside a partial starts as the value passed to it. It does not automatically recover the outer page if the caller is currently inside a `with` or `range` block.

Call location	Value passed by the final dot
At the outer level of a page layout	The page being rendered
Inside a range over project pages	One project page
Inside <code>with .Description</code>	Description text

Call `page-meta.html` where the dot is a page. Passing description text would give it the wrong kind of input. This is the same context rule you practised in Chapter 10, now applied across files.

11.4 Separate the document from its main content

A **base template** contains the shared document structure. It provides a named place where the selected page layout supplies content.

Stop the preview. Copy your current `layouts/all.html` to a new file named `layouts/baseof.html`. In the new file, replace the entire `article` element inside `main` with:

```
{{ block "main" . }}{{ end }}
```

Keep the surrounding `<main id="main" tabindex="-1">` element. The skip link still needs that target.

With the supplied header and footer call, the complete base template is:

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{{ .Title }} | {{ .Site.Title }}</title>
  <link rel="stylesheet" href="{{ "css/site.css" | relURL }}">
</head>
<body>
  <a class="skip-link" href="#main">Skip to content</a>
  <header>
    <p class="site-name">{{ .Site.Title }}</p>
    <nav aria-label="Main navigation">
      <a href="{{ "" | relURL }}">Home</a>
      <a href="{{ "about/" | relURL }}">About</a>
      <a href="{{ "articles/" | relURL }}">Articles</a>
      <a href="{{ "projects/" | relURL }}">Projects</a>
      <a href="{{ "resources/" | relURL }}">Resources</a>
    </nav>
  </header>
  <main id="main" tabindex="-1">
    {{ block "main" . }}{{ end }}
  </main>
  {{ partial "footer.html" . }}
</body>
</html>

```

Retain any earlier deliberate header customisation in your copied file. The example shows the expected structure, not a reason to discard your own wording.

Now replace `layouts/all.html` with this smaller template:

```

{{ define "main" }}
  <article>
    <h1>{{ .Title }}</h1>
    {{ partial "page-meta.html" . }}
    {{ .Content }}
    {{ if and .IsSection (eq .Section "projects") }}
      {{ partial "project-list.html" . }}
    {{ end }}
  </article>
{{ end }}

```

Save both files before building. The base contains the document; `all.html` supplies the main content. Do not leave another `html`, `head`, or `body` element in `all.html`.

Match `block` with `define`

`block "main" .` marks the place in the base where the main content will be rendered and passes the page context. `define "main"` supplies the corresponding content in the selected layout. The names must match exactly. [Hugo: block](#), [Hugo: define](#)

The block name `"main"` and the HTML element `<main>` happen to share a word, but they have different roles. One is a template name; the other gives the browser a document landmark. We retain both.

For Hugo to apply the base, the selected content template must contain definitions without ordinary output outside them. Keep the complete `article` inside `define`. An HTML comment or stray element outside the definition can prevent the base from being applied. [Hugo: base templates](#)

Build and preview Home, Projects, and the notebook project again. Confirm that the navigation, metadata, list, stylesheet, and footer remain present. You have now separated the document structure from the content area without changing what the visitor needs to see.

11.5 Give Projects its own section template

The general content layout still contains a Projects-specific condition. We can now move that decision into Hugo's template selection.

Create a `projects` folder inside `layouts`, then create `layouts/projects/section.html`:

```
{{ define "main" }}
<article>
  <h1>{{ .Title }}</h1>
  {{ partial "page-meta.html" . }}
  {{ .Content }}
  {{ partial "project-list.html" . }}
</article>
{{ end }}
```

Then remove the entire Projects conditional from `layouts/all.html`. Its complete final content becomes:

```
{{ define "main" }}
<article>
  <h1>{{ .Title }}</h1>
  {{ partial "page-meta.html" . }}
  {{ .Content }}
</article>
{{ end }}
```

Save both before rebuilding. The general layout no longer needs to ask whether it is rendering Projects. The specialised section layout makes the list call.

For our current project, the important selections are:

Page being rendered	Content template selected
Projects landing page	layouts/projects/section.html
Notebook project	layouts/all.html
Reading-list project	layouts/all.html
Home, About, Articles, and Resources	layouts/all.html

Both content templates use the same `baseof.html`. A project detail page does not become a section page merely because it lives under Projects. Likewise, the name `section.html` is a template filename; it does not replace the content file `content/projects/_index.md`.

Hugo's template selection considers page kind and location among other factors. We only need the matching Projects section template and the general `all.html` fallback here. Additional languages, output formats, custom layout names, and nested sections can introduce more candidates. [Hugo: template paths and selection](#)

There is a little shared article markup in the two content templates. That is acceptable at this scale. The document, metadata, footer, and list logic each have a clear home; we do not need another layer solely to avoid repeating a few simple lines.

Follow one rendered page through the files

For Projects, Hugo selects `projects/section.html` and combines its `main` definition with the base. That definition calls the metadata and project-list partials. The base also calls the footer partial.

For the notebook project, Hugo selects `all.html`. It uses the same base and metadata partial, but makes no project-list call. The notebook's Markdown body supplies its own headings and links as before.

Build and preview those two pages side by side. Projects should have one generated list; the notebook should have its own metadata and body without a second list of all projects.

Second checkpoint: Projects has a dedicated layout, while its navigation, document structure, and footer remain shared with the rest of the site.

11.6 Check a failure that a successful build can miss

Stop the preview. In `layouts/all.html` only, temporarily change the first line to:

```
{{ define "body-content" }}
```

Leave the matching `end` in place. Build:

```
hugo --minify --panicOnWarning
```

The build can succeed. The definition is valid template syntax, but its name no longer matches the base's `main` block.

Start the preview and open About or the notebook project. You should see the shared navigation and footer, but the main article is missing. Projects should still work because its separate template continues to define `main` correctly.

This is a useful diagnostic distinction. When shared navigation appears but page content disappears, inspect the base's block name and the selected content template's definition before changing Markdown files.

Stop the server and restore:

```
{{ define "main" }}
```

Build and preview the affected pages again. Verify that their title and content return.

A missing partial file causes a different kind of problem: Hugo normally reports that it cannot find the named partial. Compare the call's filename with the file under `_partials`. Use exact spelling and capitalisation, because a mismatch that goes unnoticed on one filesystem can fail on another.

Keep the scope of a repair small. The deliberately changed definition needs one corrected name, not replacement of the entire template set.

11.7 Make one change in the right place

Add a brief sentence immediately before the project list on the Projects landing page. For example:

```
<p>Choose a project to see its purpose, progress, and next step.</p>
```

Decide where it belongs before editing. It is introductory text for this landing-page layout, so put it in `layouts/projects/section.html`, after `.Content` and before the project-list partial call.

Preview Projects, then the notebook project and About. The sentence should appear only on Projects. If it appears everywhere, you probably added it to the base or a shared component.

For longer editorial introductions, continue using `content/projects/_index.md`. This short exercise demonstrates layout scope; it does not mean that normal page writing should migrate into templates.

Keep the agent’s file map accurate

Chapter 8’s `AGENTS.md` says the shared layout is `layouts/all.html`. That description is now incomplete. Replace that one entry under Files with:

- Shared HTML structure is in `layouts/baseof.html`.
- General page content is in `layouts/all.html`.
- The Projects landing-page template is `layouts/projects/section.html`.
- Reusable components are in `layouts/_partials/`.

Retain the other file entries and working agreements. An instruction file is useful only when it describes the current project accurately.

If you ask an agent for a later edit, name the intended layer. “Change the introduction on Projects” is clearer when you also say whether you mean the Markdown introduction or the small layout-specific sentence. Require it to inspect the current files before proposing a move.

11.8 Review the whole result and save

A shared-template edit can affect many pages, so review representative pages from both selected layouts. Use a normal preview without draft inclusion.

Check	What should remain true
Home and About	Their title, content, navigation, and footer remain present
Articles and the first learning note	The existing manual article link and body still work
Notebook and reading-list projects	Description, status, tools, and Markdown body remain correct
Projects	One generated list retains its order, descriptions, statuses, and links
Resources	Its earlier writing and links remain available
Skip link	It still targets the single <code>main</code> element with <code>id="main"</code>
Stylesheet and footer link	Their URLs retain the configured project prefix

The new Projects sentence is the one intentional visible addition. The rest of this chapter has mainly changed where the rendering instructions live.

Stop the preview, build, and inspect Git:

```
hugo --minify --panicOnWarning
git status
git diff
```

Expect one modified existing template, five new template files, and the updated `AGENTS.md`. Content and CSS should have no changes from this chapter's required steps. Use status to notice the new files; they are not shown by the ordinary unstaged diff until tracked.

Stage the intended paths with these commands:

```
git add layouts/all.html layouts/baseof.html layouts/projects/section.html
git add layouts/_partials/footer.html layouts/_partials/page-meta.html layouts/_partials/project-list.html
git add AGENTS.md
git diff --cached
```

Read the staged changes. Much of the new-file text should be familiar code moved from `all.html`. Confirm that the status block in each project-list entry survived the move, and that no temporary incorrect block name remains.

When the result matches your review, commit:

```
git commit -m "Organise Hugo layouts into a base, section template, and partials"
git status
```

A clean local checkpoint is enough to continue. Publish through Chapter 7's workflow when you are ready to push the reviewed result.

Completion check

- ☐ I can locate the base, general content template, Projects template, and three partials.
- ☐ Each partial is called with the kind of context it expects.
- ☐ Both selected content templates define the base's `main` block.
- ☐ Projects uses its own template while project detail pages use the general fallback.
- ☐ The metadata, project-list statuses, navigation, and footer survived the moves.
- ☐ I diagnosed and repaired the deliberately mismatched definition.
- ☐ My independent sentence appears only on Projects.
- ☐ I updated the agent file map and committed the seven intended files.

Chapter 12 will use structured data to build a small resource directory. These shorter templates will give us a clear place to render that data without adding every new responsibility to one large file.

Troubleshooting when you need it

Symptom	Useful next step
Hugo cannot find a partial.	Check the filename, its location under <code>layouts/_partials/</code> , and the name used in the call.
A partial reports a field error.	Check what context its caller passes; page metadata needs a page, and the project list needs its section page.
The footer appears twice.	Keep one footer element inside the partial and one call from the base.
Navigation or styling disappears after the split.	Check whether the content template has ordinary output outside <code>define</code> , preventing the base from being applied.
The shared frame appears but the article is missing.	Match the <code>block</code> and <code>define</code> names exactly.
Projects loses its list.	Check the path <code>layouts/projects/section.html</code> and its project-list call.
Every project page shows the project list.	Keep the call in the section template, not the general content template.
A layout edit seems to have no effect.	Identify which template the affected page actually selects; Projects now has a specific one.
An agent edits the old location.	Check the updated file map and name the intended file in the task.

CHAPTER 12

Build a Resource Directory with JSON

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Your Resources page contains a small collection of links. Markdown handles that well. As the collection grows, however, you may want every entry to have the same information: a title, an address, a short explanation, and a few topics.

We will store those repeated details in JSON and let a Hugo partial display them. The surrounding introduction and links to your own notebook pages will remain ordinary Markdown.

By the end, you will be able to add a resource by editing a data file. The template will supply its HTML. You will also recognise where YAML, TOML, CSV, and XML fit, without trying to master every format in one chapter.

What you will be able to do

By the end, you should be able to:

- Read and edit a small JSON array containing consistently structured records.
- Connect a local JSON resource to a Hugo template and inspect its rendered output.
- Distinguish a parsing error from incorrect or incomplete information.
- Choose a suitable format for page writing, configuration, records, or spreadsheet exchange.

Start from the completed Chapter 11 checkpoint with a clean Git working tree. (If you are following along in the companion repository `ssg-playground`, make sure you are on branch `chapter-11` or check out the completed chapter on branch `chapter-12`.) Keep the same Hugo installation and shared templates. No database, JavaScript package, or external data service is required. Internet access is useful for checking the linked documentation, but the directory itself is built from a local file.

12.1 Put two existing resources into JSON

We will begin with the two links already under Website publishing. Their titles, destinations, and descriptions are known, so this first step is a change of representation rather than a research task.

At the project root, create an `assets` folder. Inside it, create a `data` folder and then `resource_links.json`. The complete path is:

```
assets/data/resource_links.json
```

Use this content:

```
[
  {
    "title": "Hugo documentation",
    "url": "https://gohugo.io/documentation/",
    "description": "The official reference for Hugo configuration, content, and templates.",
    "topics": ["Hugo", "Reference"],
    "start_here": true
  },
  {
    "title": "Hugo page bundles",
    "url": "https://gohugo.io/content-management/page-bundles/",
    "description": "An explanation of grouping a page with related resources.",
    "topics": ["Hugo", "Content organisation"],
    "start_here": false
  }
]
```

The outside square brackets enclose an **array**, an ordered list. Each pair of curly braces encloses an **object**, which groups the named values for one resource. We will call each object a resource record.

Save the file as UTF-8 plain text in your editor. Make sure its name ends in `.json`, rather than `.json.txt`. Do not place Markdown code fences or front matter delimiters inside it.

`start_here` is our own editorial flag. Here, it marks the general documentation as a suggested starting point. It does not claim that Hugo itself assigned a rating to the resource.

The `assets` directory holds source resources that Hugo can process. This is different from `static`, whose files are copied into the output directly. We will ask Hugo to read this JSON while generating the page. [Hugo: resources.Get](#)

12.2 Display the records on Resources

We can reuse the base template from Chapter 11 and add one small content layout. Create `layouts/_partials/resource-directory.html`:

```

<h2 id="website-publishing">Website publishing</h2>
{{ with resources.Get "data/resource_links.json" }}
  <ul>
    {{ range . | transform.Unmarshal }}
      <li>
        <a href="{{ .url }}">{{ .title }}</a>
        {{ if .start_here }}
          <strong>Start here</strong>
        {{ end }}
        <p>{{ .description }}</p>
        {{ with .topics }}
          <p><strong>Topics:</strong> {{ delimit . ", " }}</p>
        {{ end }}
      </li>
    {{ else }}
      <li>No resources to display yet.</li>
    {{ end }}
  </ul>
{{ else }}
  {{ errorf "Missing resource directory data: assets/data/resource_links.json" }}
{{ end }}

```

The `with`, `range`, and `if` blocks follow the patterns from Chapter 10. The new functions read the file, interpret its JSON, and join the topic names. We will examine those lines after seeing the result.

Now create `layouts/resources/page.html` :

```

{{ define "main" }}
  <article>
    <h1>{{ .Title }}</h1>
    {{ partial "page-meta.html" . }}
    {{ .Content }}
    {{ partial "resource-directory.html" . }}
  </article>
{{ end }}

```

Use `page.html` here because Resources is the regular page at `content/resources/index.md`. Projects used `section.html` for its section landing page. Keep the existing Resources content filename and front matter unchanged.

Build and preview:

```

hugo --minify --panicOnWarning
hugo server

```

Open Resources through the navigation. Below its existing Markdown content, you should see the generated Website publishing list. It should contain two records, topic labels, and one Start here label.

For this brief intermediate stage, the old manually written Website publishing list is still present. Compare its two destinations and descriptions with the generated entries before removing it.

Remove the old list without losing the introduction

In `content/resources/index.md`, delete only the old `## Website publishing` heading and the two bullets immediately beneath it. Retain:

- The front matter and opening paragraph.
- The How to use these resources section created in Chapter 8.
- The Examples from this notebook heading and its internal links.

Save and refresh. There should now be one Website publishing heading, after the Markdown sections. The general documentation link also remains in the explanatory How to use section; that repetition is intentional.

The explicit `id="website-publishing"` preserves the heading's existing fragment destination. Open the page with `#website-publishing` at the end of its address and check that the fragment still identifies the correct section.

First checkpoint: Resources displays two JSON records through the shared page structure, and its earlier introduction and internal links still work.

12.3 Learn the JSON shapes used by the directory

JSON stands for **JavaScript Object Notation**, but it is a data format used by many languages. Writing this file does not add JavaScript behaviour to the website. [JSON: format overview](#)

Our records use a small agreement:

Key	Expected value	Purpose
<code>title</code>	Non-empty string	Text readers will activate as a link
<code>url</code>	String containing a complete HTTPS address	Destination of this external resource
<code>description</code>	Non-empty string	One accurate sentence about the resource
<code>topics</code>	Array of strings, possibly empty	A few useful subject labels
<code>start_here</code>	Boolean	Whether this entry gets our starting-point label

Use these keys consistently in every record. This table is an editorial agreement; we have not installed a validator that enforces every requirement.

Objects contain named values

A JSON member has a quoted key, a colon, and a value:

```
"title": "Hugo documentation"
```

The title is a **string**, or text value. Both JSON keys and string values use straight double quotation marks. The single-quote option shown for YAML in Chapter 9 does not apply here.

Within an object, commas separate members. A comma also separates adjacent objects in the outside array. There is no comma after the final member or final array element.

Formatting across lines makes the records easier to review, but indentation does not define their structure as it does in YAML. Braces, brackets, commas, and colons do that job.

Arrays can appear inside objects

The whole directory is an array of records. Each record's topics value is another array:

```
["Hugo", "Reference"]
```

This preserves two separate labels. `"Hugo, Reference"` would instead be one string. If no labels are useful, use `[]`, an empty array.

The outer array also gives us a deliberate display order. Moving a complete record earlier in that array moves it earlier in our rendered list. Rearranging the members inside an individual object does not change the template's field order.

Boolean values are not quoted text

```
"start_here": false
```

`false` is a Boolean value. `"false"` is a non-empty string. Both can occur in syntactically valid JSON, but only the unquoted Boolean matches our model.

This difference matters to the supplied template. Its `if .start_here` condition treats a non-empty string as true, so the text `"false"` can produce a Start here label. A successful parse does not guarantee the intended meaning.

JSON also supports numbers and `null`, as well as strings, Booleans, objects, and arrays. We do not need numeric ratings or null values for this directory, so we will not add fields merely to demonstrate them.

Keep text inside its quotes

A quotation mark inside a string must be escaped with a backslash:

```
"description": "An explanation of Hugo's \"page bundles\" feature."
```

The rendered sentence contains quotation marks without the escape backslashes. An apostrophe, as in `Hugo's`, does not need escaping inside these double quotes.

Keep descriptions as plain text. The template does not render Markdown within them. Ordinary JSON does not allow comments either; put authoring guidance in `AGENTS.md` or a separate note instead of adding `//` lines to this file.

12.4 Trace the data from file to HTML

The partial starts with:

```
{{ with resources.Get "data/resource_links.json" }}
```

`resources.Get` looks beneath `assets`, so its argument leaves off the leading `assets/`. The current context inside this `with` block is the resource it found.

Next:

```
{{ range . | transform.Unmarshal }}
```

The pipe passes that resource to `transform.Unmarshal`, which parses its structured contents. Here, the result is the JSON array. `range` then visits each record. *Unmarshal* means converting a stored representation into values the template can use. Hugo supports this operation for several data formats. [Hugo: transform.Unmarshal](#)

Inside the loop, `.title`, `.url`, and `.description` refer to keys in one JSON object. They are not the page methods `.Title`, `.RelPermalink`, and `.Description` used in Chapter 10. These records are data; they do not automatically become Hugo pages.

The inner topics block changes context again:

```
{{ with .topics }}
  <p><strong>Topics:</strong> {{ delimit . ", " }}</p>
{{ end }}
```

Inside it, the dot is the topics array. `delimit` joins its values using the supplied separator, producing text such as `Hugo, Reference`. An empty array skips the whole paragraph. [Hugo: delimit](#)

The final `else` belongs to the outer file lookup. If the JSON file cannot be found, `errorf` reports our message and fails the build. A deliberately empty array is different: it reaches the loop's `else` and displays No resources to display yet. [Hugo: errorf](#)

Keep the three source roles distinct

Source	What you edit there
<code>content/resources/index.md</code>	Introduction, explanatory writing, and notebook links
<code>assets/data/resource_links.json</code>	Repeated external-resource records
<code>layouts/_partials/resource-directory.html</code>	How each record is presented

The Resources page layout connects these pieces to the shared base. Changing a description in JSON does not require editing the HTML template.

This happens during the Hugo build. Visitors receive the resulting HTML list; their browser does not fetch the JSON for this exercise. Parsing this asset alone does not publish a separate downloadable JSON file. Its source will still be visible if you push it to your public GitHub repository.

12.5 Add a resource without changing the template

Stop the preview and open `assets/data/resource_links.json`. Add a comma after the second object's closing brace, then add this third object before the array's closing square bracket:

```
{
  "title": "Hugo template introduction",
  "url": "https://gohugo.io/templates/introduction/",
  "description": "An introduction to template expressions, functions, and context in Hugo.",
  "topics": ["Hugo", "Templates"],
  "start_here": false
}
```

This object belongs inside the existing array. Do not paste it after the final `]`, wrap it in another array, or replace the first two records.

The linked page introduces the template concepts used in the preceding chapters. Open it and compare its content with our description before retaining the entry. [Hugo: template introduction](#)

Build and preview Resources. You should see three entries, with the new one last. Follow its link. The shared header, footer, and internal notebook links should remain unchanged.

For this exercise, `url` stores complete external HTTPS addresses. Pass them directly to `href`; do not prepend the GitHub repository path. Keep the existing relative notebook links in Markdown. A mixed

directory of internal and external destinations would need an explicit convention for handling both.

Choose an order yourself

Move the template-introduction record before the page-bundles record while leaving the general documentation first. Move the complete object and check the commas between its neighbours.

Build and preview again. The list should now follow your chosen order without a template edit. This is a small independent variation: you control the content sequence while preserving the record structure.

If you edit a description as well, keep it factual and brief. Do not add a claim that you have completed a tutorial simply because its link is now on the website.

Second checkpoint: you added and reordered records through JSON alone, and verified the displayed information and destinations.

12.6 Repair broken syntax, then check meaning

Stop the preview. In the first record, temporarily remove the comma after its title line:

```
"title": "Hugo documentation"  
"url": "https://gohugo.io/documentation/",
```

This excerpt is deliberately invalid JSON. Run:

```
hugo --minify --panicOnError
```

The build should fail while parsing the resource. Read the error and identify the data file or the partial's parsing expression. The reported position may be where parsing became impossible rather than exactly where the comma was removed.

Restore the comma and build again. Do not change a working template to compensate for malformed input.

Then review the directory against the table in Section 12.3. In particular, check that every record has all five keys, topics are arrays, and the starting-point flags are actual Booleans.

There are three separate questions:

1. Can Hugo parse the JSON and render the template?
2. Do the records follow our agreed field names and value types?
3. Are their descriptions accurate and their destinations useful?

The supplied build catches syntax errors and missing files. It does not fully enforce our record model, verify remote links, or establish the truth of descriptions. A misspelled key may leave output blank; an incorrect but well-formed URL may still be rendered as a link.

Use the editor's JSON highlighting and formatting to make mistakes easier to spot. Formatting is not a factual review, and old files in `public/` are not evidence that the latest build succeeded.

12.7 Recognise the other formats without rewriting the project

JSON is our working format for the directory. The following examples are for comparison only; do not create extra copies of the directory in the project.

Format	Useful role in this book	What to recognise
Markdown	Page writing	Headings, paragraphs, links, and lists
JSON	Repeated resource records	Objects, arrays, quoted keys, explicit value types
YAML (<code>.yaml</code> or <code>.yml</code>)	Front matter and the GitHub workflow	Named values and indentation-based structure
TOML	<code>hugo.toml</code> configuration	<code>key = value</code> entries and named tables
CSV	Rows exchanged with spreadsheet tools	A header row, records, delimiters, and quoted cells
XML	Formats such as sitemaps and feeds	Nested named elements and attributes

YAML and TOML: familiar information, different notation

Here is a small part of one resource represented in YAML:

```
title: "Hugo documentation"
start_here: true
topics:
  - "Hugo"
  - "Reference"
```

The same information can be expressed in TOML:

```
title = "Hugo documentation"
start_here = true
topics = ["Hugo", "Reference"]
```

These standalone examples have no front matter delimiters. In a Markdown page, Hugo uses delimiters to identify where metadata begins and ends. Keep the existing YAML page front matter and TOML site configuration; converting them adds no benefit to this exercise. [YAML specification](#), [TOML specification](#)

CSV: useful rows, but conversion needs decisions

A small spreadsheet export might look like this:

```
title,url,description
Hugo documentation,https://gohugo.io/documentation/, "Reference for configuration, content, and templates."
Hugo page bundles,https://gohugo.io/content-management/page-bundles/,Explains grouping a page with related resources.
```

The quotation marks keep the commas inside the first description from becoming column separators. CSV dialects vary, so check the delimiter and encoding used by an export. CSV is plain tabular text; it does not preserve a spreadsheet's full formatting or workbook structure. [RFC 4180: CSV format](#)

To turn those rows into our JSON records, map each column to its matching key, then deliberately supply `topics` and `start_here`. The CSV shown does not contain those values. A converter or agent should not invent them without an instruction.

For a small import, compare the number of data rows with the number of JSON objects, inspect the first and last records, and check descriptions containing commas or quotation marks. Treat arrays and Booleans explicitly rather than carrying every cell across as a string.

Hugo can parse CSV with `transform.Unmarshal`, but the CSV-to-directory workflow is optional future work. Our current template expects the JSON array shown earlier. Merely renaming a CSV file to `.json` does not convert it.

XML: recognise the wrapper and preserve meaning

A simplified XML record could be:

```
<resource>
  <title>Hugo documentation</title>
  <url>https://gohugo.io/documentation/</url>
</resource>
```

The opening and closing tags describe a hierarchy. Real XML formats define which elements and attributes belong where; similar-looking tags do not make this fragment a valid sitemap or feed. [W3C: XML specification](#)

When you encounter Hugo's generated sitemap or RSS output, recognise XML as the representation. Change the source or relevant template to maintain generated output, rather than hand-editing the generated file.

12.8 Keep the directory manageable and save

A directory helps when records share stable fields and presentation. It is less useful for a long explanation, a learning narrative, or a page whose information has no repeated shape. Continue writing those in Markdown.

Before adding a field, ask what a reader or a template will do with it. An unused rating, review date, or author field creates more information to maintain. Our five fields are enough for the current result.

Update the project guidance

Add these entries under Files in `AGENTS.md`:

- Resource-directory records are in `assets/data/resource_links.json`.
- The Resources page template is `layouts/resources/page.html`.
- Resource-directory rendering is in `layouts/_partials/resource-directory.html`.

Add this working agreement alongside the existing rules:

- Resource records use `title`, `url`, `description`, `topics` (an array of strings), and `start_here` (a Boolean). Preserve these types and do not invent descriptions or destinations.

An agent can help prepare a record or convert supplied rows, but it needs source information and a field agreement. Chapter 13 will practise that editorial workflow in more detail.

Review the completed page

Check that Resources shows three directory entries in your chosen order, with the expected topics and starting-point label. Follow each external link, then verify the retained notebook links. Check Projects too, so the new Resources layout has not displaced the section layout from Chapter 11.

Stop the preview and run:

```
hugo --minify --panicOnWarning
git status
git diff
```

The intended changes are three new files, the edited Resources Markdown, and the updated guidance. Read the new files as well as the ordinary diff before staging:

```
git add assets/data/resource_links.json layouts/resources/page.html layouts/_partials/resource-
directory.html
git add content/resources/index.md AGENTS.md
git diff --cached
```

Confirm that only the old manual Website publishing section was removed from Markdown. Retain the earlier agent-assisted How to use section and its links.

When the result matches your review, commit:

```
git commit -m "Build the Resources directory from local JSON records"
git status
```

The working tree should be clean. The local checkpoint is sufficient for the next chapter; publishing remains the reviewed workflow from Chapter 7.

Completion check

- ☐ I can distinguish an object, an array, a string, and a Boolean in the directory.
- ☐ All records follow the five-field agreement and contain accurate descriptions.
- ☐ I know which file holds the records, the page writing, and the rendering instructions.
- ☐ I added and reordered a resource without changing the template.
- ☐ I repaired the missing comma and checked more than build success.
- ☐ I can explain when CSV conversion needs decisions about missing fields and types.
- ☐ Resources retains its earlier Markdown sections and working links.
- ☐ I updated the guidance and committed the five intended files.

This is enough structured data for our next steps. A database, an API request, a browser-side loader, and automated format conversion are outside this chapter’s scope. A JSON file and a template cover a great deal before any of those becomes necessary.

Chapter 13 turns supplied source material into linked pages. The field agreement you have just written becomes the standard an agent’s contribution has to meet.

Troubleshooting when you need it

Symptom	Useful next step
The missing-data message appears.	Check the exact path <code>assets/data/resource_links.json</code> ; the resource lookup omits the initial <code>assets/</code> .

Symptom	Useful next step
Hugo reports a JSON parsing error.	Check commas, double quotes, matching brackets, and whether a comment or trailing comma was added.
An entry has blank text.	Compare its keys with the model and the lowercase keys used in the template.
Start here appears for <code>"false"</code> .	Replace the string with the Boolean <code>false</code> .
Topics are displayed incorrectly.	Store separate topic strings inside an array, not one combined string.
The old list still appears.	Remove only the manual Website publishing heading and its two original bullets.
The generated directory is absent.	Check <code>layouts/resources/page.html</code> and its partial call; Resources is a regular page.
A record uses an internal-looking URL.	This model uses complete external HTTPS addresses; keep notebook links in the Markdown section.
A build passes but a destination fails.	Verify the URL itself. This template does not perform remote-link checks.

CHAPTER 13

Create and Maintain Content with AI Agents

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

In Chapter 8, you gave an agent a small, self-contained job: three bullets in one file. You could read the whole result in a minute. Real content work is larger than that, and its mistakes are harder to see.

In this chapter, you will produce a complete article from source material you supply, then connect it to the rest of the site. The work is divided into three requests: a plan, a draft, and a coordinated update across several files. You will review the claims in the draft against the source before publishing anything.

The visible result is one new article at `/articles/publishing-with-github-pages/`, reachable from the Articles landing page and the home page, with a matching record in the resource directory. The harder result is a working method: supply the material, bound the task, check every claim, and decide publication yourself.

What you will be able to do

By the end, you should be able to:

- Separate work you can delegate from decisions that must remain yours.
- Supply source material and require an agent to work only from it.
- Break a content contribution into steps that can each be reviewed.
- Check a draft's claims against its source and reject an unsupported sentence.

Start from the completed Chapter 12 checkpoint with a clean Git working tree. (If you are following along in the companion repository `ssg-playground`, make sure you start from branch `chapter-12` or check out the completed chapter on branch `chapter-13`.) You need the Codex CLI installation and account access from Chapter 8, the article and project pages from Chapters 2, 3, and 9, and the JSON directory from Chapter 12. Nothing new is installed in this chapter.

If your account cannot currently reach an agent, you can still complete the exercise by applying the comparison article near the end of the chapter yourself. That practises the Hugo work and the review, but not the delegation.

13.1 Decide what an agent may and may not contribute

An agent can arrange, condense, and format material it has been given. It cannot know what happened to you. The notes you are about to supply record events from Chapters 6 and 7; you are the only available authority on whether the finished article describes them correctly.

Work you can delegate	Decisions that stay with you
Turning rough notes into ordered prose	Whether an event actually happened
Applying an agreed article structure	Whether a claim is supported by the source
Writing a one-sentence description	Whether the page is ready for readers
Adding a link in an agreed format	Which destinations belong on the site
Preparing a record for the directory	Whether a resource deserves inclusion

The rule behind that table is worth stating plainly: automated checks assess technical requirements, while human review assesses meaning, sources, and suitability for publication. A successful build tells you Hugo could render the page. It says nothing about whether the page is true.

Two habits follow. New pages stay at `draft: true` until you have read them, so publication remains a deliberate decision as it was in Chapter 9. And the source material is committed alongside the article, so the basis for a published claim stays recoverable.

13.2 Put the source material where it can be read

At the project root, create a folder named `sources`. Inside it, create `publishing-notes.md` with this content:

Raw notes: publishing the notebook

Not for publication as written. Working notes from Chapters 6 and 7.

Reference consulted: GitHub's documentation on configuring a publishing source,
<https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site>

- A Pages site has to be told where its content comes from. We chose the GitHub Actions route rather than publishing from a branch.
- The supplied workflow builds with Hugo and deploys the built output. We never committed the public/ folder.
- baseURL had to include the repository path. Before that, the live site loaded without its stylesheet and the internal links went to the wrong place.
- The first deployment failed. Cause: I pushed before setting the Pages source. Fixed by setting it, then running the workflow again.
- The Actions log was where the failure was legible. The browser only showed a missing or stale page.
- hugo --minify --panicOnError passed locally before that push. Passing locally did not mean the deployment had worked.
- Not measured: how long a deployment usually takes.
- Not attempted: a custom domain.

Save it as plain text. The `sources` folder sits beside `content`, so Hugo will not turn these notes into a page, just as `AGENTS.md` produced no page in Chapter 8.

Read the notes yourself first. They hold three kinds of material, and the difference matters for the review later:

In the notes	What it is	How it may be used
The baseURL problem, the failed first deployment, the Actions log	Events recorded by the author	May become statements about what happened
The GitHub documentation reference	An external source	May be credited and paraphrased
The two Not lines	Explicit gaps	May be reported as unknown; may not be filled in

The last row carries this chapter's main risk. Notes usually stop short of what a finished page would like to say, and an agent asked for readable prose has an obvious way to smooth over a gap. A plausible sentence is much harder to notice than a broken link.

Record the source material before it is used:

```
git add sources/publishing-notes.md
git diff --cached
git commit -m "Add the publishing notes used as source material"
git status
```

13.3 Agree what an article must contain, and record it

Chapter 9 gave projects a content model and deliberately left articles alone. The first learning note has only a title and a draft flag. A second article arriving from outside your own typing needs the same kind of agreement.

Location	Field or heading	Our rule
Front matter	<code>title</code>	A readable article title
Front matter	<code>description</code>	One sentence explaining what the reader will learn
Front matter	<code>draft</code>	<code>true</code> until you have reviewed the page
Body	What this is about	Why the article exists, in two or three sentences
Body	What happened	The sequence of events, drawn from the source
Body	What I would do differently	A specific change, or an honest statement that it is unclear
Body	Sources	Every external reference used, as a link with a short explanation

Sources is ordinary Markdown in the body, not front matter. Chapter 9 showed that storing a value in front matter does not teach a layout to display it, and Chapter 12 showed the same about data. Attribution no visitor can see is not attribution, and the body makes it visible without a template change.

Prefer a short paraphrase with a credited link over a long quotation: it is easier to keep accurate and avoids reproducing someone else's wording on your site.

An `archetypes/articles.md` starter would suit this model, on the pattern of Chapter 9's project archetype. Add one if you later write articles by hand regularly.

Record the agreement in the project guidance

Chapter 8's instruction file says nothing about sourcing, because nothing had needed it yet. Add these entries under Files in `AGENTS.md`:

- Source material for content work is in `sources/`.
- Articles are page bundles under `content/articles/`, each with its own `index.md`.
- The Articles landing page and its manual list are in `content/articles/_index.md`.
- The home page and its Latest writing list are in `content/_index.md`.

Add these working agreements alongside the existing rules:

- For content tasks, use only the source files named in the task. Do not search the web or add material from memory.
- Do not state anything the named source does not support. Where the source records a gap, say that it is unknown rather than estimating.
- Credit every external reference in a Sources section in the page body, with a link and a short explanation.
- New articles use title, description, and draft in front matter, and the headings What this is about, What happened, What I would do differently, and Sources.
- Leave new pages at `draft: true`. Publication is the reader's decision.

Read the file after editing, then commit it:

```
git add AGENTS.md
git diff --cached
git commit -m "Add content sourcing and attribution agreements for agent work"
```

Persistent guidance saves repetition and gives you something concrete to point at when a result is wrong. It is not enforcement, so the requests below still state their own boundaries.

13.4 Ask for a plan before a draft

Asking straight away for a finished article produces a page that is expensive to check: structure, prose, and claims arrive together, and a fault in the structure has already shaped every paragraph. Ask for a plan instead.

Start a read-only session from the project root:

```
codex --sandbox read-only --ask-for-approval on-request
```

Check `/status`, then paste:

Read AGENTS.md and sources/publishing-notes.md. Do not edit any file.

I want to publish one article based only on those notes.

Produce, as your reply in this session:

1. A proposed title and a one-sentence description.
2. Under each of the four agreed article headings, the points you would make.
3. A table with one row per factual statement you propose: the statement, and the exact line of the notes that supports it.
4. A list of anything an article on this topic would normally mention that these notes do not support.

Do not write the article yet. Do not add material from any other source.

Read the reply against the notes. A useful plan is recognisable by what it refuses to claim: item 4 should name the deployment duration and custom domains, because the notes record both as unmeasured or untried. Item 3 should map every proposed statement onto a line you can find.

Ask again if it compares hosting providers, quotes the documentation at length, describes your motivation, or offers a statement whose supporting line you cannot locate. Correcting a plan costs one short request; correcting a draft built on that plan costs far more.

Use `/exit`, then confirm that nothing changed:

```
git status
```

First checkpoint: you have a plan whose every claim is traceable to a line of your notes, and an explicit list of what the notes cannot support.

13.5 Draft the article from the source only

Restart with permission to edit the project:

```
codex --sandbox workspace-write --ask-for-approval on-request
```

A new session does not carry the previous conversation, so the task is stated in full again. Check `/status`, then paste:

Read AGENTS.md and sources/publishing-notes.md before editing.

Task: Write one article from those notes, using the agreed article model.

Create exactly one new file:

content/articles/publishing-with-github-pages/index.md

Front matter: title, a one-sentence description, and draft: true.

Body headings, in this order:

What this is about

What happened

What I would do differently

Sources

Use no more than 400 words in the body. Plain paragraphs and short lists only.

Every factual statement must be supported by a line in the notes.

Where the notes record something as not measured or not attempted, either omit it or state plainly that it is unknown. Do not estimate.

Paraphrase the GitHub reference; do not quote it at length. Credit it under Sources with its URL and one sentence explaining what it covers.

Change no other file. Do not edit layouts, CSS, configuration, the workflow, the JSON directory, or any existing page.

Do not stage, commit, push, or deploy.

Run `hugo --minify --panicOnWarning` if permissions allow, and report the result.

Finish by listing the file you created and any statement you were unsure about.

When it finishes, use `/exit` and inspect the result yourself:

```
git status
git diff --stat
git diff --cached
```

The new bundle is untracked, so it appears under status rather than in an ordinary diff. The staged diff should be empty, and no existing file should be modified yet.

Open `content/articles/publishing-with-github-pages/index.md` and read every line against the notes, finding the line that supports each sentence. Check that the front matter has all three fields, that `draft` is still `true`, and that Sources credits the GitHub reference with a working URL.

Build and preview. The article is a draft, so draft rendering has to be enabled, as in Chapter 2:

```
hugo --minify --panicOnWarning
hugo server -D
```

Open `/articles/publishing-with-github-pages/`. Check the heading order, read the page as a visitor, and follow the link in Sources. The Articles landing page will not list the new article yet; its list has been manual since Chapter 3.

Second checkpoint: one new article exists, reads accurately against your notes, and renders in a draft preview without affecting any other page.

13.6 Make the coordinated update across several files

An article nobody can reach is not published work. Three existing files need to know about it, and none updates itself: the Articles list, the home page's Latest writing list, and the resource directory that now holds the reference you cited.

Stop the preview, start a session as before, and paste:

Read AGENTS.md, content/articles/_index.md, content/_index.md, and assets/data/resource_links.json before editing.

Task: Connect the new article to the rest of the site.

Edit exactly these three files.

1. content/articles/_index.md: add one bullet to the existing Start reading list, in the same format as the existing bullet, linking to publishing-with-github-pages/ with a short explanation.
2. content/_index.md: add one bullet to the existing Latest writing list, in the same format as the existing bullet.
3. assets/data/resource_links.json: add one record at the end of the array for the GitHub publishing-source page cited in the new article. Use the five agreed keys: title, url, description, topics (an array of strings), and start_here (the Boolean false). The url is the complete HTTPS address.

Preserve all existing bullets, records, sections, and front matter.

Match the relative-link style already used in each file.

Change nothing else. Do not stage, commit, push, or deploy.

Run `hugo --minify --panicOnWarning` and report the result.

Three files in one request is a larger surface than anything in Chapter 8, so review them one at a time:

```
git diff --stat
git diff -- content/articles/_index.md
git diff -- content/_index.md
git diff -- assets/data/resource_links.json
```

Check each change against what the file already did. The bullet in `content/articles/_index.md` is relative to the Articles section and needs no `articles/` prefix; the one in `content/_index.md` is relative to the site root and does. Chapter 8 showed how a link can be well formed and still point to the wrong place, and Chapter 12 set the rule that directory records hold complete external addresses. A record

with a relative URL, or with a quoted `"false"` instead of the Boolean, is a mistake the build will not report.

Restart the preview as an ordinary visitor would see it:

```
hugo server
```

The article is still a draft, so it should be absent, and the two new bullets should lead to a page that is not there yet. That is the expected intermediate state, not a fault. Check the new entry on Resources, follow its external link, and confirm the three original records are unchanged.

If one of the three edits is wrong, correct it with a named follow-up, or restore that single path:

```
git restore -- content/_index.md
```

Restoring one path leaves the other two edits and the untracked article in place. Inspect status before and after, rather than reaching for a broad cleanup.

13.7 Test your review on an unsupported claim

Your notes say the deployment time was never measured. An article about publishing has an obvious gap where that sentence would go.

In the new article, under What happened, add this sentence:

```
Deployments usually finish in under a minute.
```

Save and build:

```
hugo --minify --panicOnError
```

The build succeeds. Nothing in the project examines that sentence. It is grammatical, plausible, and in keeping with the paragraph around it, and nothing you recorded supports it. It may even be true; you have no evidence either way, and the article presents it as experience.

Ask three questions of any statement in a page you did not write yourself:

1. Is it in the source material at all?
2. If it is, does the source support it as strongly as this wording claims?
3. If a reader relied on it and it were wrong, what would happen?

A broken link announces itself the moment someone activates it. An unsupported sentence can stay on a site for years, and it is the mistake agent-assisted content produces most readily, because fluent prose is exactly what the tool is good at.

Remove the experiment:

```
git status
```

The article is still untracked, so `git restore` cannot recover it from history. Delete the sentence in your editor, save, and build again. Git protects what it has been given: until the bundle is committed, your editor holds the only earlier state.

Third checkpoint: you found an unsupported claim that the build accepted, and you can explain why review has to read for meaning as well as for errors.

13.8 Approve, save, and make one request of your own

Publication is your decision, taken once and deliberately. In `content/articles/publishing-with-github-pages/index.md`, change:

```
draft: true
```

to:

```
draft: false
```

Restart a normal preview with `hugo server` and review the whole result:

Check	What should be true
The new article	Every statement traces to your notes; the two gaps are absent or marked unknown
Its Sources section	The GitHub reference is credited, and its link works
Articles	Both articles are listed, and both links work
Home	Latest writing lists both articles, and both links work
Resources	Four directory records, correct topics, one Start here label
The first learning note	Its body, image, and links are unchanged
Projects	Its generated list and section sentence are unchanged

Then build, inspect, and stage the intended paths:

```
hugo --minify --panicOnError
git status
git add content/articles/publishing-with-github-pages/index.md
git add content/articles/_index.md content/_index.md assets/data/resource_links.json
git diff --cached
```

Read the staged diff in full. It should contain one new article, two added bullets, and one added JSON record. `sources/publishing-notes.md` and `AGENTS.md` were committed earlier in the chapter and should not reappear here.

```
git commit -m "Publish an agent-drafted article and link it from the site"
git status
```

Now make one request of your own. Ask the agent to tighten the article's `description` to a single clause without changing its meaning, or to add one further topic label to the new record. Name the file, say what counts as an improvement, and keep the same limits on other files and on Git actions.

Review the diff and the page, then accept it or keep your version. The reading-list project from Chapter 9 is an honest candidate for a later contribution: its notes do not exist yet, and an article cannot be written from material you have not gathered.

The local checkpoint is enough to continue. When you decide to publish, use Chapter 7's push, deployment check, and live-page verification yourself.

Completion check

- ☐ I can say which parts of a content task I may delegate and which I may not.
- ☐ I supplied source material in `sources/` and committed it before it was used.
- ☐ I agreed an article model and recorded it in the project guidance.
- ☐ I obtained a plan whose claims map to specific lines of my notes.
- ☐ I reviewed the draft sentence by sentence against the source.
- ☐ The article credits its external reference visibly in the page body.
- ☐ I made a coordinated update to three files and reviewed each one separately.
- ☐ I found the unsupported sentence that the build accepted, and removed it.
- ☐ I decided publication myself and committed the four intended paths.

This is enough agent-assisted content work for our next steps. Bulk generation, translation, retrieval over your own material, automated fact-checking, and unattended contributions are outside this chapter's scope. The method matters more than the volume: supply the material, bound the request, review the claims, decide publication.

Chapter 14 turns the parts of this review that can be stated as rules into automated checks on a pull request, so that a contribution is tested before anyone decides to merge it.

Troubleshooting when you need it

Symptom	Useful next step
The agent adds material that is not in the notes.	Name the rule it broke, ask for a corrected draft from the source alone, and reread the result rather than the summary.
It fills a gap with an estimate.	Point at the relevant Not line in the notes. Require an explicit statement that the figure is unknown, or its omission.
It quotes the GitHub page at length.	Ask for a paraphrase and a credited link. Long quotations are unnecessary here and harder to maintain.
The new article does not appear in the preview.	It is a draft. Use <code>hugo server -D</code> , or set <code>draft: false</code> when you have decided to publish.
The article appears but the two lists do not mention it.	Both lists are manual. Section 13.6 is the step that adds them.
A new bullet leads to a missing page.	Compare its relative form with the existing bullet in the same file; the Articles list and the home page need different prefixes.
Start here appears on the new record.	Set <code>start_here</code> to the unquoted Boolean <code>false</code> , as agreed in Chapter 12.
The directory record uses a relative address.	Directory records hold complete external HTTPS addresses. Internal notebook links stay in Markdown.
The build passes but a sentence looks doubtful.	The build does not read for meaning. Trace the statement to its source line, or remove it.
<code>git restore</code> will not recover the new article.	An untracked file has no committed state. Edit it in your editor, and commit once you have accepted it.
The staged diff contains files from earlier sections.	<code>sources/publishing-notes.md</code> and <code>AGENTS.md</code> were committed separately. Unstage anything unrelated before committing.

Source notes, repository text, and retrieved pages can contain instructions aimed at whatever reads them. Treat such material as content to assess, not as authority to widen the task, publish, or change credentials.

One acceptable article for comparison

Your agent will choose different wording. This example shows the intended scale, the heading order, and the level of claim the notes actually support. It is not a transcript of a recorded agent run.

```

---
title: "What I Learned publishing with GitHub Pages"
description: "How this notebook reached a public address, and what went wrong the first time."
draft: false
---

## What this is about

This notebook is published from its own repository rather than uploaded by hand. Setting that up went wrong once, in a way that was easy to misread.

## What happened

A Pages site has to be told where its content comes from. I chose the GitHub Actions route rather than publishing from a branch, so a workflow builds the site with Hugo and deploys the built output. The generated public/ folder is never committed.

The first deployment failed, because I pushed before setting the publishing source. Setting it and running the workflow again fixed it. The failure was only legible in the Actions log; the browser showed a missing page, which told me nothing about the cause.

One configuration detail mattered more than I expected: baseUrl has to include the repository path. Before I corrected it, the live site loaded without its stylesheet and its internal links went to the wrong place.

I also learned to distrust a passing local build as evidence about the live site. hugo --minify --panicOnWarning succeeded before the push that failed to deploy.

## What I would do differently

I would set the publishing source before the first push, and read the Actions log before looking at the site in a browser.

I have not measured how long a deployment usually takes, and I have not tried a custom domain, so I cannot say anything useful about either.

## Sources

- [GitHub: configuring a publishing source](https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site): Explains how a Pages site is told where to publish from, including the GitHub Actions route used here.

```

The matching directory record, added at the end of the array in `assets/data/resource_links.json`:

```
{
  "title": "GitHub: configuring a publishing source",
  "url": "https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site",
  "description": "The official explanation of how a GitHub Pages site is told where its content is published from.",
  "topics": ["GitHub Pages", "Publishing"],
  "start_here": false
}
```

And the two added bullets, in their existing lists:

```
- [What I learned publishing with GitHub Pages](publishing-with-github-pages/): Setting up a publishing source, and the first deployment that failed.
```

```
- [What I learned publishing with GitHub Pages](articles/publishing-with-github-pages/)
```

CHAPTER 14

Check Every Contribution with CI/CD

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Chapter 6 deferred branching until there was a concrete reason to use it. This is that reason. Every change you have made since then went straight onto `main`, and every push to `main` publishes.

In this chapter, you will add a second workflow that examines a proposed change before it can reach the live site. It builds the site and checks three rules this book has already established: directory flags must be real Booleans, internal links must stay relative, and articles and projects must carry a description. Then you will propose a change on a branch, break one of those rules on purpose, and watch the checks stop it.

The visible result is a pull request with a red failing check, then the same pull request with a green passing one, then a deployment you approved. The durable result is a division of labour: the machine checks what can be stated as a rule, and you check what cannot.

What you will be able to do

By the end, you should be able to:

- Work on a branch and propose a change through a pull request.
- Read a workflow's triggers, jobs, steps, and permissions, and say why its permissions are narrower than the publishing workflow's.
- Run the same checks locally before pushing.
- Explain what a passing check does and does not establish before you merge.

Start from the completed Chapter 13 checkpoint with a clean Git working tree, pushed to GitHub, with the Chapter 7 publishing workflow in place and a working public address. (If you are following along in the companion repository `ssg-playground`, make sure you start from branch `chapter-13` or check out the completed chapter on branch `chapter-14`.) You need the GitHub CLI authentication from Chapter 7 and access to your repository's settings.

Everything here runs on GitHub's free runners for a public repository. Automation minutes are billed differently for private repositories and for larger runners; check your account's current terms before moving this arrangement to other work.

14.1 Add a workflow that checks proposed changes

The Chapter 7 workflow deploys. This one only reports. Keeping them in separate files makes the difference visible in the repository itself, and it leaves your working publishing workflow untouched.

Create `.github/workflows/checks.yaml`:

```

name: Check proposed changes

on:
  pull_request:
    branches: [main]

permissions:
  contents: read

jobs:
  checks:
    runs-on: ubuntu-24.04
    env:
      HUGO_VERSION: "0.150.0"
    steps:
      - name: Check out the proposed source
        uses: actions/checkout@v7

      - name: Install Hugo
        shell: bash
        run: |
          curl --fail --location --retry 3 \
            --output "$RUNNER_TEMP/hugo.tar.gz" \
            "https://github.com/gohugoio/hugo/releases/download/v${HUGO_VERSION}/hugo_${HUGO_VE
            RSION}_linux-amd64.tar.gz"
          mkdir -p "$RUNNER_TEMP/hugo-bin"
          tar -xzf "$RUNNER_TEMP/hugo.tar.gz" -C "$RUNNER_TEMP/hugo-bin" hugo
          echo "$RUNNER_TEMP/hugo-bin" >> "$GITHUB_PATH"

      - name: Build the website
        run: hugo --minify --panicOnError

      - name: Check that directory flags are Booleans
        shell: bash
        run: |
          if grep -n '"start_here": *"' assets/data/resource_links.json; then
            echo "start_here must be an unquoted Boolean: true or false."
            exit 1
          fi

      - name: Check that internal links stay relative
        shell: bash
        run: |
          if grep -rn '](/' content/; then
            echo "Internal links must be relative, not root-relative."
            exit 1
          fi

      - name: Check that articles and projects have a description
        shell: bash
        run: |
          status=0
          for page in content/articles/*/index.md content/projects/*/index.md; do
            if ! grep -q '^description:' "$page"; then
              echo "Missing description: $page"
            fi
          done

```

```

        status=1
    fi
done
exit "$status"

```

The install step is the one from Chapter 7, unchanged, so the checks use the same Hugo version as the deployment. [GitHub: Actions documentation](#), [Checkout action](#)

Commit this file to `main` and push it:

```

git add .github/workflows/checks.yaml
git diff --cached
git commit -m "Add checks for proposed changes"
git push

```

That push runs the publishing workflow as usual, because `hugo.yaml` still watches `main`. It will rebuild and republish the same content; adding a checks file does not change any page. The new workflow does nothing yet, because no pull request exists.

14.2 Read the parts you are relying on

The vocabulary here is small, and you have seen most of it in Chapter 7.

Part	Its job in this file
<code>on: pull_request</code>	Starts this workflow when a change is proposed against <code>main</code> , not when it is pushed there
<code>permissions: contents: read</code>	Allows reading the source and nothing else
<code>jobs: checks</code>	One job; Chapter 7's workflow has two, <code>build</code> and <code>deploy</code>
<code>runs-on: ubuntu-24.04</code>	The runner : a fresh machine GitHub provides for this job
<code>env: HUGO_VERSION</code>	One value used by the install step
<code>steps</code>	Actions and shell commands, run in order on that runner
<code>uses:</code>	Runs a published action, such as checkout
<code>run:</code>	Runs shell commands on the runner

Two differences from the publishing workflow are deliberate, and both are worth being able to explain.

The **trigger** is different. `pull_request` means these checks look at a proposal. Chapter 7's `push` to `main` means that workflow acts on what has already been accepted. A proposal gets examined; an accepted change gets published.

The **permissions** are narrower. Chapter 7's workflow needs `pages: write` and `id-token: write` because it deploys. This one reads the source and reports, so `contents: read` is all it should have. Giving a checking job the ability to publish would remove the distinction the two files exist to create.

A job fails when any step exits with a non-zero status. In our three checks, `exit 1` is how a broken rule reports itself, and a step that finds nothing simply ends successfully. The `if grep ...; then ... exit 1; fi` shape looks inverted at first: `grep` succeeding means it *found* the thing we do not want.

CI stands for continuous integration: changes are checked as they are proposed, rather than in one examination before release. **CD** covers continuous delivery or deployment, which for us is the Chapter 7 workflow. Together they describe the arrangement you now have, not a product you install.

14.3 Run the same checks locally, and repair what they find

A check you can only run on GitHub is a slow way to find a typing mistake. Run the three rules on your own computer first.

In your project terminal:

```
hugo --minify --panicOnWarning
grep -n '"start_here": *"' assets/data/resource_links.json
grep -rn ']/(' content/
grep -c '^description:' content/articles/*/index.md content/projects/*/index.md
```

The first two `grep` commands should print nothing and report no match. The last one prints a count for each page, and this is where you should find a problem: `content/articles/first-learning-note/index.md` reports `0`.

That article was written in Chapter 2, before this book had given articles a content model. Chapter 13 introduced the model and applied it to the new article only. The check is correct and the content is out of date.

This is the ordinary experience of adopting a check: the first thing it finds is existing work that predates the rule. Bring the older page up to the standard rather than weakening the rule to accommodate it. Open `content/articles/first-learning-note/index.md` and add a description line to its front matter, keeping the existing fields:

```
description: "Editing a page in my notebook, checking the result, and recording what changed."
```

Run the last command again. Every page should now report `1`. Then build, review, and commit the repair on `main`:

```
hugo --minify --panicOnWarning
git diff -- content/articles/first-learning-note/index.md
git add content/articles/first-learning-note/index.md
git commit -m "Add a description to the first learning note"
git push
```

If a rule turns out to be wrong rather than the content, change the rule deliberately and say why in the commit message. What you should not do is add an exception that quietly exempts one file, because the next reader will not know whether it is a decision or an oversight.

First checkpoint: the three rules pass locally, and `main` satisfies the standard its own checks describe.

14.4 Propose a change on a branch

Until now, an edit and its publication were the same act. A branch separates them: you can record work, push it, and show it to a check before anything reaches the live site.

Create a branch and switch to it:

```
git switch -c add-actions-resource
git status
```

`git switch -c` creates the branch and moves onto it. Status should report the new branch with a clean tree. Nothing has been copied; a branch is a name for a line of commits, not a second folder. [Git: switch](#)

Now make the proposed change. Add a fifth record at the end of the array in `assets/data/resource_links.json`, following the Chapter 12 agreement. Remember the comma after the previous record's closing brace:

```
{
  "title": "GitHub: Actions documentation",
  "url": "https://docs.github.com/en/actions",
  "description": "The official reference for automating builds, checks, and deployments on GitHub.",
  "topics": ["GitHub Actions", "Automation"],
  "start_here": false
}
```

Preview it, then commit it to the branch and push:

```
hugo server
```

Check Resources: four records, correct topics, one Start here label. Stop the preview, then:

```
hugo --minify --panicOnWarning
git add assets/data/resource_links.json
git diff --cached
git commit -m "Add the GitHub Actions documentation to the resource directory"
git push -u origin add-actions-resource
```

The `-u origin add-actions-resource` part sends the branch to GitHub for the first time and remembers the connection, so later pushes on this branch need only `git push`. This push does **not** publish anything: `hugo.yaml` watches `main`, and you are not on `main`.

14.5 Open the pull request and read its checks

A **pull request** proposes that one branch be merged into another. It is also where the checks report and where the diff can be read. [GitHub: pull requests](#)

Open your repository on GitHub. It should offer to create a pull request from the branch you just pushed. Choose to compare `add-actions-resource` into `main`, give it a short title such as `Add the GitHub Actions documentation to the resource directory`, and create it.

If you prefer the terminal, the GitHub CLI from Chapter 7 can do the same:

```
gh pr create --base main --head add-actions-resource --fill
gh pr checks
```

On the pull request page, look at three things in order:

What to read	What it tells you
Files changed	The actual diff, which is the same thing you reviewed locally
Checks	Whether Check proposed changes passed, is running, or failed
The commit list	Which commits this proposal contains

Open the check run itself and expand its steps. You should see the checkout, the Hugo install, a successful build, and the three rule checks reporting nothing. Reading a passing log now makes a failing one far easier to interpret later.

This proposal should pass. Leave the pull request open; the next section gives it something to catch.

14.6 Break a rule on purpose and let the checks stop it

Chapter 12 explained that `"false"` in quotation marks is a string, not a Boolean, and that it makes the Start here label appear on a record that should not have one. Hugo builds it without complaint. We now have a check that does not.

On your branch, edit the new record so its flag is quoted:

```
"start_here": "false"
```

Commit and push it:

```
git add assets/data/resource_links.json
git commit -m "Temporarily quote the start_here flag to test the checks"
git push
```

Return to the pull request. The push adds a commit to the same proposal, so the checks run again. This time the job should fail, and the pull request should show a red mark rather than a green one.

Open the failed run and find the step that failed. The Build the website step should have succeeded, because this mistake is valid JSON. The Check that directory flags are Booleans step should have failed, printing the matching line and the message from the workflow.

Read the log from the top, not from the bottom. The first failing step is the one to act on; later steps may not have run at all.

Now repair it. Restore the unquoted Boolean:

```
"start_here": false
```

Then verify locally before pushing again, which is the habit worth keeping:

```
grep -n '"start_here": *"' assets/data/resource_links.json
hugo --minify --panicOnWarning
git add assets/data/resource_links.json
git commit -m "Restore start_here as a Boolean"
git push
```

The checks should run once more and pass. The pull request now contains three commits, including the mistake and its correction. That history is honest and useful; it is not something to hide.

Second checkpoint: a rule this book taught was broken, a machine caught it before publication, and you fixed it from the log.

14.7 Require the checks, and keep review separate from them

Nothing so far stops you merging a failing pull request. A **ruleset** can require the check to pass first.

In your repository on GitHub, open **Settings**, then the rules or rulesets area, and create a rule targeting the `main` branch. Enable the requirement that status checks must pass, and select `checks` from the list of available checks. Save it.

The check must have run at least once before GitHub can offer it by name, which is why this step comes after Section 14.6 rather than before it. Interface labels in this area change; use GitHub's current documentation if the wording differs from the description above. [GitHub: about rulesets](#)

Be clear about what you have just built. As the repository's owner, you can usually still merge past your own rule. For a solo author, the ruleset is a reliable reminder rather than a wall, and that is worth having: it makes bypassing a failing check a deliberate act instead of an oversight.

Now separate the two kinds of judgement, which is the point of the whole chapter:

The checks establish	Only you can establish
The site builds with warnings treated as failures	The writing is accurate
Directory flags are Booleans	The resource is worth listing
Internal links are relative	Those links go where a reader expects
Articles and projects have a description	The description describes the page honestly

Every row on the left is something we could state as a rule. Nothing on the right can be, which is why a green check is permission to look properly rather than a verdict. Chapter 13's unsupported sentence would pass all four of these checks.

An approval requirement is the other half of this arrangement, and it needs a second person: GitHub will not let you approve your own pull request. If a collaborator joins the project, add a required approval then. Working alone, your review step is reading Files changed deliberately before you merge, exactly as you read a diff in Chapter 6.

14.8 Merge, verify the deployment, and save

Read the pull request's Files changed once more. It should show one added record and nothing else. Confirm the check is green, then merge it on GitHub.

Merging commits the change to `main`, which is a push to `main`, which starts the Chapter 7 publishing workflow. Watch it in the Actions tab, then verify the live site as you did in Chapter 7:

Check	What should be true
Actions	Both workflows appear in the history, with distinct names and purposes
The publishing run	It built and deployed after the merge
The live Resources page	Five records, correct topics, one Start here label
The live article	The first learning note still reads correctly after its description was added
The rest of the site	Navigation, Projects, and the footer are unchanged

Bring your local repository back into line. Your branch's work now lives on `main`, so switch back and collect it:

```
git switch main
git pull
git log --oneline -5
git status
```

The log should show the merged work, and the tree should be clean. The remote branch can be deleted from the pull request page once merged; GitHub usually offers a button for it. Delete your local copy when you no longer need it:

```
git branch -d add-actions-resource
```

Then make one proposal of your own on a new branch. A small, honest option: give the reading-list project from Chapter 9 a more specific description, or add one topic label to an existing directory record. Push it, open a pull request, read the checks, review your own diff, and merge it when both you and the checks are satisfied.

Completion check

- ☐ I can explain why the checks workflow uses `pull_request` and narrower permissions than the publishing workflow.
- ☐ I ran all three rule checks locally and interpreted their output.
- ☐ I brought an older page up to a rule rather than weakening the rule.
- ☐ I created a branch, pushed it, and opened a pull request without publishing anything.
- ☐ I read a passing check log before I needed to read a failing one.
- ☐ I broke a rule on purpose, found the failing step in the log, and repaired it.

- ☐ I required the check on `main` and can say what my own bypass ability means.
- ☐ I can name something true about my site that no check here could establish.
- ☐ I merged an approved change, watched it deploy, and verified the live result.

This is enough automation for our next steps. Test frameworks, external link crawling, accessibility and performance scoring, preview deployments for each pull request, scheduled maintenance runs, and multi-environment promotion are outside this chapter's scope. Add a check when you can state the rule it enforces and say what it cannot see.

Chapter 15 turns to the reader's experience of the finished site: search, navigation, and the titles, descriptions, sitemaps, and feeds that help people find your content.

Troubleshooting when you need it

Symptom	Useful next step
The checks workflow does not run.	It triggers on <code>pull_request</code> only. Confirm the file is on <code>main</code> , and that a pull request against <code>main</code> exists.
The description check fails on a page you did not touch.	That is expected on first use. Section 14.3 repairs the older article; bring existing content up to the rule.
The description check reports a path with an asterisk in it.	No file matched that pattern. Check that the article and project bundles exist at the expected paths.
A <code>grep</code> check fails and you cannot see why.	The printed line is the match. <code>grep</code> succeeding means it found what the rule forbids.
The build step fails but the rule checks do not run.	Steps stop at the first failure. Fix the build first, then rerun.
The internal-link check fails on an external address.	The pattern matches <code>](/</code> only. A complete <code>https://</code> address does not contain it; check for a stray leading slash.
Pushing the branch published the site.	Confirm you were not on <code>main</code> . <code>hugo.yaml</code> triggers on pushes to <code>main</code> , including a merge.
The check cannot be selected in the ruleset.	It must have run at least once. Open a pull request, let the checks run, then add the requirement.
GitHub will not let you approve your own pull request.	That is expected. Working alone, read Files changed yourself and require the status check instead.

Symptom	Useful next step
A merge conflict appears in the pull request.	Your branch and <code>main</code> changed the same lines. Chapter 18 covers resolving these; for now, update the branch from <code>main</code> and re-check the file.
The live site does not show the merged change.	Check the publishing run in Actions. A merge starts it, but the deployment still has to succeed.

A passing check set is evidence about stated rules on one commit. It is not evidence that the writing is true, the sources are real, or the page is suitable to publish.

Command card: the branch and review cycle

```
git switch -c my-change      # create a branch and move onto it
git add <paths>              # stage the intended files
git commit -m "message"     # record the work on the branch
git push -u origin my-change # send the branch to GitHub the first time
git push                    # later pushes on the same branch
git switch main              # return to the published line of work
git pull                    # collect the merged result
git branch -d my-change      # delete the local branch once merged
```

Run the local checks from Section 14.3 before each push. Open, read, and merge the pull request on GitHub.

CHAPTER 15

Help Readers Find and Use Your Content

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Your notebook now has nine pages, a resource directory, a publishing workflow, and checks on every proposal. A visitor arriving at it has the navigation, and nothing else. They cannot search it, and a browser tab showing every page as part of the same long name does not help them keep their place.

In this chapter you will improve how the site describes itself and add a way to search it. You will give every page a useful document title and a description, inspect the sitemap and feed Hugo has been generating all along, and build a search page that filters a list of your pages as the reader types.

The visible result is a working Search page in the navigation. The more important results are quieter: distinct browser-tab titles, a description for each page that both search engines and your own search box can use, and a page that still lists every destination when JavaScript does not run.

What you will be able to do

By the end, you should be able to:

- Give each page a distinct document title and a description, with a site-wide fallback.
- Recognise Hugo's generated sitemap and feed as XML, and explain what each is for.
- Build a search page from your own content and filter it in the browser.
- Test a page by keyboard and without JavaScript, and say what an automated score does not establish.

Start from the completed Chapter 14 checkpoint, with a clean working tree on `main` and the checks workflow in place. (If you are following along in the companion repository `ssg-playground`, make sure you start from branch `chapter-14` or check out the completed chapter on branch `chapter-15`.) Work on a branch and propose the result through a pull request, as you did in Chapter 14; the commands in Section 15.8 assume you did.

This chapter adds a small amount of JavaScript. You are not expected to learn the language here, and the script is supplied and explained. No package, build tool, or external search service is installed.

15.1 Give every page a useful title and description

Open your preview and look at the browser tab for the home page, then for the first learning note. Then view the page source and find the `title` element in the head.

Your base template currently builds it like this:

```
<title>{{ .Title }} | {{ .Site.Title }}</title>
```

On the home page, that produces `Welcome to my knowledge notebook | My Knowledge Notebook`. The notebook's name appears twice in slightly different words, and a browser tab shows only the first part. The document title is the main label used by a tab, a bookmark, a shared link, and a search result, so it is worth getting right.

Add a site-wide description first. In `hugo.toml`, keep the existing three settings and add a named table at the end:

```
[params]
description = 'Learning notes, small projects, and useful references, published as a static web site.'
```

`[params]` is a TOML table, as introduced in Chapter 12. Values inside it are available to templates as `.Site.Params`.

Now open `layouts/baseof.html` and replace the single `title` line with these three lines:

```
<title>{{ if .IsHome }}{{ .Site.Title }}{{ else }}{{ .Title }} | {{ .Site.Title }}{{ end }}</title>
<meta name="description" content="{{ with .Description }}{{ . }}{{ else }}{{ .Site.Params.description }}{{ end }}">
<link rel="canonical" href="{{ .Permalink }}">
```

Keep the character set, viewport, and stylesheet lines exactly as they are.

Three template ideas from Chapter 10 are doing the work. `if .IsHome` asks which page this is, so the home page uses the site name alone. `with .Description` uses the page's own description when it has one, and its `else` branch falls back to the site description. `.Permalink` is the page's full address. A **canonical** link tells search engines which address to treat as the original when a page is reachable by more than one route; our site has one route per page, so this is a precaution rather than a repair.

Rebuild and check the source of three pages. The home page tab should read `My Knowledge Notebook`. The first learning note should read `My first learning note | My Knowledge Notebook`, with the description you added in Chapter 14. About and Resources have no description of their own yet, so both fall back to the site description. You will fix that in Section 15.8.

If a description contains a quotation mark, Hugo escapes it for the attribute rather than breaking the tag, as Chapter 4 explained for `&quot;`. Check the generated source rather than assuming.

First checkpoint: every page has a distinct tab title and a description in its head, with a sensible fallback where a page has none.

15.2 Inspect the sitemap and feed Hugo already generates

Two XML files have existed since Chapter 1. With the preview running, open these addresses, adding them to the base address Hugo reports:

```
/sitemap.xml
/index.xml
```

You should see XML rather than a designed page. Chapter 12 introduced its shape: nested named elements, opening and closing tags, a declaration at the top. Browsers display XML as a tree or as plain source, depending on the browser.

File	What it is for	Who reads it
<code>/sitemap.xml</code>	A list of your page addresses	Search engines discovering what exists
<code>/index.xml</code>	A feed of recent content	Feed readers, so people can subscribe

Read the sitemap and check that the addresses match the pages you expect, including the article added in Chapter 13. Read the feed and check that its items have titles and links. If either lists a page you have withdrawn, remember Chapter 9's warning: generated output from an earlier build can persist until a new build and deployment replace it.

Neither file is something to edit. They are generated, like everything in `public/`. To change what appears in them, change the content or the templates, exactly as Chapter 12 said about hand-editing generated XML.

A feed that nobody can discover is not much use. Add this line to the head in `layouts/baseof.html`, after the description and canonical lines:

```
{{ with .OutputFormats.Get "rss" }}
  <link rel="{{ .Rel }}" type="{{ .MediaType.Type }}" href="{{ .Permalink }}" title="{{ $.Site.
    Title }}">
{{ end }}
```

This asks whether the current page has a feed. The home page and section landing pages do; an individual article does not, so `with` skips the whole block there.

Note `$.Site.Title` rather than `.Site.Title`. Inside the `with` block the dot has become the output format, not the page. `$` always refers to the context the template started with, which is the page. It is the escape hatch for exactly this situation, and you will use it again in the next section.

Rebuild and view the source of the home page and of an article. The home page should carry the feed link; the article should not.

15.3 Generate a search page from your own content

Our search will list every page and filter that list as the reader types. The list is produced by Hugo at build time from the pages themselves, so it cannot fall out of date, and it is ordinary HTML, so it remains useful if the filtering never runs.

Create `content/search/index.md`:

```
---  
title: "Search"  
description: "Find a page in this notebook by its title or description."  
draft: false  
---
```

Type a word to narrow the list below. Clearing the box shows every page again. This searches page titles and descriptions, not the full text of each page.

Now create `layouts/search/page.html`. Resources uses `layouts/resources/page.html` for the same reason: Search is a regular page at `content/search/index.md`, not a section landing page.

```

{{ define "main" }}
<article>
  <h1>{{ .Title }}</h1>
  {{ partial "page-meta.html" . }}
  {{ .Content }}

  <form class="search-form" role="search">
    <label for="search-query">Search titles and descriptions</label>
    <input type="search" id="search-query" name="q" autocomplete="off">
  </form>

  <p id="search-status" role="status"></p>

  <ul id="search-index">
    {{ range .Site.RegularPages }}
      {{ if ne .RelPermalink $.RelPermalink }}
        <li class="search-item">
          <a href="{{ .RelPermalink }}">{{ .Title }}</a>
          {{ with .Description }}
            <p>{{ . }}</p>
          {{ end }}
        </li>
      {{ end }}
    {{ end }}
  </ul>

  <script src="{{ "js/search.js" | relURL }}" defer></script>
</article>
{{ end }}

```

The `range` and `with` blocks are the ones from Chapters 10 and 11. `.Site.RegularPages` is the site's individual pages; as in Chapter 11's project list, it excludes section landing pages such as Articles and Projects, which the navigation already covers. The `if ne .RelPermalink $.RelPermalink` line keeps the Search page from listing itself, and it needs `$` because the dot inside `range` is now the page being listed.

The `role="search"` attribute marks this region as the site's search, and `role="status"` marks the paragraph as one whose changes should be announced. The `defer` attribute tells the browser to run the script after the page structure exists.

Build and preview:

```

hugo --minify --panicOnWarning
hugo server

```

Open `/search/`. You should see the form and a list of six pages: About, both articles, both projects, and Resources. Every link should work. Typing does nothing yet, and the status paragraph is empty, because the script does not exist.

Notice that About and Resources appear with a title and no description. Their titles are searchable; nothing else about them is.

15.4 Add the script that filters the list

Create `static/js/search.js`. Files under `static` are copied to the site root, so this is served at `/js/search.js`, in the same way as the stylesheet from Chapter 5.

```
(function () {
  var form = document.querySelector(".search-form");
  var input = document.getElementById("search-query");
  var list = document.getElementById("search-index");
  var status = document.getElementById("search-status");

  if (!form || !input || !list || !status) {
    return;
  }

  var items = list.querySelectorAll(".search-item");

  function filter() {
    var query = input.value.trim().toLowerCase();
    var shown = 0;

    items.forEach(function (item) {
      var match = query === "" || item.textContent.toLowerCase().includes(query);
      item.hidden = !match;
      if (match) {
        shown += 1;
      }
    });

    if (query === "") {
      status.textContent = "Showing all " + items.length + " pages.";
    } else if (shown === 0) {
      status.textContent = "No pages match that word.";
    } else {
      status.textContent = shown + " of " + items.length + " pages match.";
    }
  }

  form.addEventListener("submit", function (event) {
    event.preventDefault();
  });

  input.addEventListener("input", filter);
  filter();
})();
```

You are not expected to write this from memory. Read it as six decisions:

Part	What it does
The four <code>document</code> lookups	Find the form, box, list, and status paragraph by the names used in the template
The <code>if (!form ...)</code> guard	Stop quietly if any of them is missing, so other pages are unaffected
<code>filter()</code>	Compare the lowercased query with each item's text and hide the ones that do not contain it
<code>item.hidden = !match</code>	Use the standard HTML hidden attribute rather than inventing a class
The <code>submit</code> listener	Prevent the form from reloading the page when Enter is pressed
<code>filter()</code> at the end	Fill in the status message once when the page loads

The comparison is a plain case-insensitive substring match on the text already visible in each item. It does not rank results, handle misspellings, match word stems, or search page bodies. For a notebook of this size that is sufficient; Section 15.8 says when it stops being.

The list needs one styling rule so hidden items really disappear, plus a little room for the form. Add this to the end of `static/css/site.css`:

```
.search-form label {
  display: block;
  font-weight: 600;
  margin-bottom: 0.25rem;
}

.search-form input {
  width: 100%;
  padding: 0.5rem;
  font-size: 1rem;
}

#search-status {
  color: #37474f;
}

.search-item[hidden] {
  display: none;
}
```

The last rule matters. The `hidden` attribute normally hides an element, but a `display` value from another rule can override it, so making the intent explicit avoids a puzzle later. As Chapter 5 said, check contrast before adopting a new colour rather than trusting appearance.

Reload `/search/` and try three queries in turn:

Type this	Expected result
<code>notebook</code>	Several pages match, because the word appears in their titles and descriptions
<code>reading</code>	The reading-list project alone matches
<code>hugo</code>	Nothing matches, even though the site is built with Hugo

The third result is worth pausing on rather than treating as a fault. Hugo is named in your Resources directory and in your articles' bodies, but not in any page title or description, and those are the only text this search examines. Clear the box and all six pages should return.

Second checkpoint: searching filters a list Hugo generated from your own pages, and the status line reports how many matched.

15.5 Connect Search to the navigation

In `layouts/baseof.html`, add one link to the existing navigation block, after Resources:

```
<a href="{{ "search/" | relURL }}">Search</a>
```

Save, then visit three different pages and use the new link from each. The navigation now has six items. That is close to the point where a single row stops being comfortable on a narrow screen, so check it at a phone width using the technique from Chapter 5; the supplied navigation wraps rather than overflowing.

Chapter 3 noted that Hugo has a menu system that keeps navigation data separate from its rendering, and that we would use it when there was enough template knowledge. There now is. We are still not doing it, because six explicit links are easier to read than a configuration table that produces six links. Reach for the menu system when the navigation differs between sections or languages, which Chapter 16 will begin to make true.

15.6 Diagnose a search that fails silently

This failure is deliberately undramatic, which is the point.

In `layouts/search/page.html`, change one attribute only:

```
<ul id="search-list">
```

Save and reload `/search/`. Look carefully before reading on.

The page appears completely normal. Every page is listed, every link works, the form is there. Typing does nothing, and the status paragraph stays empty. Open the browser console from Chapter 4 and you will find no error at all.

The guard in the script is responsible. It looked for `search-index`, did not find it, and returned without doing anything. That guard is worth having, because it stops the script complaining on every other page of the site. The cost is that a genuine mistake produces silence rather than a message.

Work from the observable symptom instead. The status paragraph is empty, and the script's last action is to set it, so the script cannot have reached the end. Then compare the two names:

1. Inspect the list element and read its `id`.
2. Read the four names near the top of `static/js/search.js`.
3. Find the one that differs.

This is Chapter 5's selector mistake in another costume: a name that must match in two files, changed in only one. Restore `id="search-index"`, save, and confirm that filtering and the status line return.

There is one more thing to notice. Because the list is real HTML produced at build time, a broken script leaves a complete, usable index of your site rather than an empty box and an error. A search built by asking the browser to fetch and assemble an index would have failed visibly and uselessly instead. That is a reason to prefer this arrangement at this size, not merely a happy accident.

15.7 Test by keyboard, without JavaScript, and at small widths

Automated tools come next. Do these three checks first, because they are the ones that find real problems.

By keyboard. Load `/search/` and press Tab from the top of the page. You should reach the Skip to content link, the six navigation links, then the search box, then the links in the list. Type a word and continue tabbing: you should move through the visible results only, because a hidden item cannot receive focus. Confirm that the focus outline from Chapter 5 is visible at every stop.

Without JavaScript. In your browser's developer tools, disable JavaScript and reload the page. The expected result is the full list of pages, with a box that does nothing. Confirm that, then re-enable JavaScript. A visitor in that situation still has every destination and a working navigation; they have lost a convenience, not the page.

At small widths and increased zoom. Use Chapter 5's method on the Search page: narrow the window to a phone width and separately increase the browser zoom. The form should stay within the reading column, the navigation should wrap, and no text should be cut off.

On announcements, be careful what you claim. The `role="status"` paragraph is intended to be read out when its text changes, which is why the script writes a count there rather than only hiding items. Whether and how that is announced varies between screen readers and browsers, and this draft has not been tested with any of them. Treat it as a reasonable arrangement that still needs checking, not as a verified result.

15.8 Use automated tools without trusting their scores, then improve one thing and save

Chromium browsers include Lighthouse in their developer tools, and accessibility extensions such as axe DevTools report a similar class of finding. Run one against `/search/` and one against an article.

Read the individual findings and ignore the overall number. A report of this kind can tell you that an image lacks alternative text, that a heading level was skipped, that a contrast is too low, or that a page has no description. Those are worth acting on, and several could later become rules in the Chapter 14 workflow.

What it cannot tell you is whether your descriptions are accurate, whether the search results are the ones a reader wanted, whether your headings describe what follows them, or whether the article is true. Chapter 13's unsupported sentence would score perfectly. A high score means a set of mechanical checks passed, which is what a green pull request means, and it carries the same limits.

Now make the improvement the chapter has been pointing at. About and Resources still have no description, so they fall back to the site description and offer only a title to your search box. Add one line to the front matter of each, keeping their existing fields:

```
description: "Why I keep this notebook, and what you can expect to find in it."
```

```
description: "References that support the work recorded in this notebook."
```

Use your own accurate wording. Then check both: the head of each page should now carry its own description, and searching for a word from either description should match that page.

Update the file map in `AGENTS.md`, under Files:

- The Search page template is `layouts/search/page.html`.
- The search script is `static/js/search.js`.

Build, review, and propose the result as a pull request on a branch, as in Chapter 14:

```
hugo --minify --panicOnError
git switch -c find-and-search
git status
git add hugo.toml layouts/baseof.html layouts/search/page.html
git add content/search/index.md static/js/search.js static/css/site.css
git add content/about/index.md content/resources/index.md AGENTS.md
git diff --cached
git commit -m "Add a search page, document titles, descriptions, and feed discovery"
git push -u origin find-and-search
```

Open the pull request and let the checks run. All three Chapter 14 rules should still pass: the directory flags are untouched, no root-relative link was added, and both new descriptions sit on pages the description rule does not even examine. Read Files changed yourself, then merge and verify the live site.

Finally, decide when this search stops being the right one. Every page appears in the Search page's HTML, so that page grows with your site, and the filtering sees only titles and descriptions. Somewhere between tens and a few hundred pages, or as soon as readers expect full-text results, the answer becomes a generated index the browser fetches, or a dedicated search service. Change the approach when you can describe the reader's unmet need, not when the page count reaches a particular number.

Completion check

- ☐ Each page has a distinct document title, and the home page does not repeat the site name.
- ☐ Every page has a description in its head, with the site description as a fallback.
- ☐ I found `/sitemap.xml` and `/index.xml` and can say what each is for.
- ☐ The home page offers the feed for discovery and an article correctly does not.
- ☐ I can explain why `$` is needed inside the `with` and `range` blocks I added.
- ☐ Search lists my pages, filters as I type, and reports how many matched.
- ☐ I diagnosed the silent failure by noticing the empty status line, not an error message.
- ☐ The page still lists every destination with JavaScript disabled.
- ☐ I read individual audit findings and can name something no audit could establish.
- ☐ About and Resources have their own descriptions, and the change passed the checks.

This is enough discoverability for our next steps. Full-text indexing, result ranking, search analytics, structured data for rich results, image optimisation, and performance budgets are outside this chapter's scope. Each becomes worth adding when a reader's difficulty makes the need concrete.

Chapter 16 adds a second language, which is the first change that will make navigation, metadata, and page structure differ between versions of the same site.

Troubleshooting when you need it

Symptom	Useful next step
The home page tab still repeats the site name.	Confirm the <code>if .IsHome</code> branch is in <code>layouts/baseof.html</code> and that you replaced the old <code>title</code> line rather than adding a second one.
Every description is the site description.	Those pages have no <code>description</code> in their front matter. That is the fallback working; add page descriptions where you want distinct text.
The build fails after the head edit.	Check that each <code>{{ if }}</code> , <code>{{ with }}</code> , and <code>{{ end }}</code> is matched, and that the attribute quotation marks survived the paste.
<code>/sitemap.xml</code> or <code>/index.xml</code> is not found.	Use the full base address Hugo reports, including any project prefix, and check the spelling.
The feed link appears on every page, or on none.	The <code>with .OutputFormats.Get "rss"</code> block decides this. Section pages and the home page have a feed; regular pages do not.
<code>.Site.Title</code> is empty inside the feed block.	Inside <code>with</code> , the dot is the output format. Use <code>\$.Site.Title</code> .
The Search page uses the wrong layout.	Confirm the path <code>layouts/search/page.html</code> . Search is a regular page, like Resources.
The Search page lists itself.	Restore the <code>if ne .RelPermalink \$.RelPermalink</code> condition, including the <code>\$</code> .
Articles and Projects are missing from the list.	That is intended. <code>.Site.RegularPages</code> excludes section landing pages, which the navigation covers.
A page you expect is missing.	Check whether it is still a draft. A normal build excludes drafts, so the generated list excludes them too.
Typing does nothing and the console is empty.	The script's guard returned early. Compare the four names in <code>search.js</code> with the <code>id</code> and class in the template.
Items do not disappear when filtered.	Confirm the <code>.search-item[hidden]</code> rule is in the stylesheet and that the served CSS contains it.

Symptom	Useful next step
The page reloads when Enter is pressed.	The script did not load or returned early. Check the script tag's address, then the four names.

An automated report and a passing check both examine stated rules on one version of a page. Neither establishes that a reader can find what they came for.

CHAPTER 16

Publish in Multiple Languages

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Your notebook is in English. If some of your readers would rather read Kurdish, or if your work belongs to a place where two languages are normal, the site should be able to say the same things twice.

In this chapter you will add Kurdish Sorani as a second language, translate the navigation and two pages, and give the document the language and text direction a browser needs. Kurdish Sorani is written in a Perso-Arabic script and runs right to left, so this is also the first time your layout has to work in the other direction. You will find and repair the one stylesheet rule that assumes left to right.

The visible result is a Kurdish home page and About page at `/ckb/`, reachable from a language link, with the English site unchanged at its existing addresses. If you would rather use Arabic, or any other second language, every step works the same way; only the language code and the strings change.

What you will be able to do

By the end, you should be able to:

- Add a second language without changing any existing English address.
- Set each page's language and text direction, and translate interface labels.
- Recognise a layout rule that assumes one text direction, and replace it.
- Say what must be true before you publish a translation you cannot read.

Start from the completed Chapter 15 checkpoint with a clean working tree on `main`. (If you are following along in the companion repository `ssg-playground`, make sure you start from branch `chapter-15` or check out the completed chapter on branch `chapter-16`.) Work on a branch and propose the result through a pull request, as in Chapters 14 and 15.

One requirement is not technical. If you do not read the second language, you need someone who does, and Section 16.6 explains why that is not optional. Choose a language you can have reviewed before you begin.

16.1 Add a second language without changing your existing addresses

Your English pages are published and linked. Chapter 3 established that addresses should be stable, so the first decision is the one that protects them.

Open `hugo.toml`. Replace the `languageCode` line, keep everything else, and add the language settings:

```
baseURL = 'https://YOUR-USERNAME.github.io/my-knowledge-site/'
title = 'My Knowledge Notebook'
defaultContentLanguage = 'en'
defaultContentLanguageInSubdir = false

[params]
description = 'Learning notes, small projects, and useful references, published as a static web site.'

[languages]
[languages.en]
  locale = 'en'
  label = 'English'
  title = 'My Knowledge Notebook'
  weight = 1
[languages.ckb]
  locale = 'ckb'
  label = 'کوردی'
  direction = 'rtl'
  title = 'تێبینیهکانی من'
  weight = 2
```

Note on Hugo configuration keys: In modern Hugo releases (v0.158+), `locale`, `label`, and `direction` are the canonical keys. Earlier versions used `languageCode`, `languageName`, and `languageDirection`; in recent versions, using the older keys triggers deprecation warnings that cause `hugo --panicOnWarning` to halt.

Two settings carry the whole decision. `defaultContentLanguage = 'en'` says English is the default, and `defaultContentLanguageInSubdir = false` says the default language stays where it is. English pages keep `/about/`; Kurdish pages will appear under `/ckb/about/`. Setting that second value to `true` would move every English page to `/en/...` and break every address you have published and every link anyone has saved.

`ckb` is the language code for Kurdish Sorani. `weight` sets the order languages are listed in. `direction = 'rtl'` records that this language runs right to left, which Section 16.2 will use. Each language has its own `title`, so the site name itself is translated. [Hugo: multilingual mode](#)

Build and preview:

```
hugo --minify --panicOnWarning
hugo server
```

Check that the English site is exactly as it was. Visit the home page, About, Articles, the two projects, Resources, and Search. Nothing should have moved. There is no Kurdish content yet, so `/ckb/` has nothing to show.

First checkpoint: the site has two configured languages and every English address is unchanged.

16.2 Give each page its language and direction

Open `layouts/baseof.html` and look at its first two lines. The language is written into the file:

```
<html lang="en">
```

That was true when there was one language. Now it is a statement the Kurdish pages would make incorrectly, and it matters more than it looks: the `lang` attribute tells a screen reader which pronunciation to use, helps a browser choose a font, and affects how text is hyphenated and searched.

Replace that line with:

```
<html lang="{{ .Site.Language.Locale }}" dir="{{ .Site.Language.Direction | default "ltr" }}">
```

(Note: in older Hugo versions, `.LanguageCode` and `.LanguageDirection` were used; modern Hugo uses `.Locale` and `.Direction`.)

The `dir` attribute is what actually turns the page around. It is not decoration: it tells the browser that text, punctuation, and the natural start of a line all run from the right. `default "ltr"` supplies a value for English, which has no `direction` in the configuration, and is the same `default` idea you could use anywhere a setting might be absent.

Rebuild and view the source of an English page. It should read `lang="en"` and `dir="ltr"`. We cannot check the Kurdish side until there is a Kurdish page, which is Section 16.5.

16.3 Translate the navigation with language-aware links

The navigation has two problems in a second language. Its labels are English words, and its destinations are built with `relURL`, which knows about the project path but not about the language.

Fix the labels first. Create a folder named `i18n` at the project root, beside `content` and `layouts`. Inside it, create `i18n/en.toml`:

```
nav_home = 'Home'
nav_about = 'About'
nav_articles = 'Articles'
nav_projects = 'Projects'
nav_resources = 'Resources'
nav_search = 'Search'
nav_label = 'Main navigation'
languages_label = 'Languages'
```

Then create `i18n/ckb.toml` with the same keys:

```
nav_home = 'ماڵهه'
nav_about = 'دهرباره'
nav_articles = 'وتارمکان'
nav_projects = 'پڕۆژمکان'
nav_resources = 'سەرچاومکان'
nav_search = 'گەڕان'
nav_label = 'رێنیشاندەری سەرەکی'
languages_label = 'زمانهکان'
```

These files hold **interface strings**: the words your templates say, rather than the words you write as content. Keep the keys identical in both files; a key present in one file and missing from the other produces an empty label rather than an error. [Hugo: multilingual translation of strings](#)

Now replace the whole navigation block in `layouts/baseof.html` with this version:

```
<nav aria-label="{{ i18n "nav_label" }}">
  <a href="{{ "" | relLangURL }}">{{ i18n "nav_home" }}</a>
  <a href="{{ "about/" | relLangURL }}">{{ i18n "nav_about" }}</a>
  <a href="{{ "articles/" | relLangURL }}">{{ i18n "nav_articles" }}</a>
  <a href="{{ "projects/" | relLangURL }}">{{ i18n "nav_projects" }}</a>
  <a href="{{ "resources/" | relLangURL }}">{{ i18n "nav_resources" }}</a>
  <a href="{{ "search/" | relLangURL }}">{{ i18n "nav_search" }}</a>
</nav>
{{ partial "language-links.html" . }}
```

Two functions are doing the work. `i18n` looks a key up in the current language's file. `relLangURL` is `relURL` with the language prefix added, so the same template line produces `/about/` in English and `/ckb/about/` in Kurdish. Leaving `relURL` there would have sent every Kurdish visitor back to the English pages. [Hugo: relLangURL](#)

One more link needs the same treatment, and it is not in the file you were just editing. Open `layouts/_partials/footer.html` and change its About link:

```
Read <a href="{{ "about/" | relLangURL }}">about this notebook</a>.
```

The footer appears on every page in both languages. Left as `relURL`, it would have sent Kurdish readers to the English About page from the bottom of every Kurdish page: a link that works, points somewhere real, and is still wrong. When you change how links are built, search the whole `layouts` folder rather than only the file in front of you.

Its visible wording stays English for now. That is remaining translation work rather than a defect: like the five untranslated pages, it is a known gap, and Section 16.5 is about stating such gaps rather than hiding them.

Create the language switcher as `layouts/_partials/language-links.html`:

```

{{ with .Translations }}
  <nav aria-label="{{ i18n "languages_label" }}" class="language-links">
    {{ range . }}
      <a href="{{ .RelPermalink }}" lang="{{ .Language.Locale }}" hreflang="{{ .Language.Locale }}">{{ .Language.Label }}</a>
    {{ end }}
  </nav>
{{ end }}

```

`.Translations` gives the other language versions of **this page**, so the link goes to the same page in the other language rather than to a language home page. The `lang` attribute on each link matters here too: the word `کوردی` is Kurdish even when it appears on an English page, and marking it lets a screen reader pronounce it correctly.

Because of `with`, a page with no translation shows no switcher at all. That is the honest behaviour: offering a Kurdish link on a page that has no Kurdish version would promise something that does not exist. The consequence, which Section 16.5 returns to, is that readers switch language from the pages that have both versions.

The stylesheet needs nothing new for the switcher beyond a little separation. Add this to `static/css/site.css`:

```

.language-links {
  margin-top: 0.5rem;
  font-size: 0.9375rem;
}

```

Rebuild and check the English pages. The labels should be unchanged English words, every navigation link should still work, and no switcher should appear yet.

16.4 Find and fix the rule that assumes left to right

Before adding Kurdish content, look at the stylesheet for rules that describe positions as left or right rather than as start and end. There is one:

```

.skip-link { position: absolute; top: -10rem; left: 1rem; }

```

That rule came from the Chapter 1 starter and has been correct ever since, because English begins on the left. On a right-to-left page it puts the Skip to content link in the far corner from where reading begins, which is precisely the wrong place for the first thing a keyboard user reaches.

CSS has two families of properties for this. **Physical** properties name a side of the screen: `left`, `right`, `margin-left`, `padding-right`, `text-align: left`. **Logical** properties name a position relative to the text direction: `inset-inline-start`, `margin-inline-start`, `text-align: start`. A logical property follows the direction; a physical one does not.

Replace that one rule with:

```
.skip-link { position: absolute; top: -10rem; inset-inline-start: 1rem; }
```

`inset-inline-start` means the beginning of the line: the left in English, the right in Kurdish. One property, both directions, no duplicated rules and no second stylesheet.

Look through the rest of the file and satisfy yourself that nothing else needs changing. The reading column already uses `margin-inline: auto`, which you met in Chapter 5, and the spacing rules use `padding-block` or symmetric values. The remaining `margin-top` and `border-top` values are vertical, so text direction does not affect them.

Kurdish Sorani also reads more comfortably with a little more space between lines than English. Add this rule:

```
:lang(ckb) {
  line-height: 1.9;
}
```

The `:lang()` selector matches elements whose language is the one named, which works because Section 16.2 put a real `lang` attribute on the document.

A word about fonts, because it is easy to overpromise here. Our stack is still `system-ui, sans-serif` from Chapter 1. Every current desktop and mobile operating system ships a font that covers Arabic script, so the text should render without downloading anything. What you cannot assume is that it renders *well*: letter shapes, joining, and diacritic placement vary between system fonts, and a font that looks correct on your computer may not on someone else's. Check the Kurdish pages on more than one device before deciding that a downloadable font is necessary. If you do add one later, the cost is a download on every visit, which is a real trade rather than an obvious improvement.

16.5 Translate two pages, and be honest about the rest

Hugo identifies a page's language from its filename. A file with no language code belongs to the default language, which is why none of your existing files need renaming.

File	Language	Address
<code>content/about/index.md</code>	English, by default	<code>/about/</code>
<code>content/about/index.ckb.md</code>	Kurdish	<code>/ckb/about/</code>

Create `content/_index.ckb.md` for the Kurdish home page and `content/about/index.ckb.md` for the Kurdish About page. Give each the same front-matter fields the English version has, with `title` and `description` in Kurdish, and add one new field:

```

---
title: "دەربارە"
description: "..."
draft: false
params:
  source_checked: "2026-09-17"
---

```

`source_checked` records the date on which this translation was last compared with its English source. Section 16.7 explains how it is used. Fill in the description and body in your second language; the comparison files at the end of the chapter show the expected shape.

Write the Kurdish home page to link only to what actually exists in Kurdish. Your English home page has an Explore the notebook list of four sections; the Kurdish one should link to the Kurdish About page, and then say plainly that the rest of the notebook is in English, with a link to the English home page. Do not copy the English list of links across, because five of those destinations have no Kurdish version.

This is the ordinary condition of a bilingual site, not a failure. A site where one language is complete and the other covers an entry point is honest as long as it says so. What misleads a reader is a navigation that promises six Kurdish pages and delivers one.

Build and preview:

```

hugo --minify --panicOnWarning
hugo server

```

Open `/ckb/`. Check all of the following:

Check	What should be true
Text direction	The page reads right to left, and the navigation begins on the right
The document	Its source shows <code>lang="ckb"</code> and <code>dir="rtl"</code>
Navigation labels	Kurdish, from <code>i18n/ckb.toml</code>
Navigation destinations	Each goes to <code>/ckb/...</code> , not to the English page
The footer link	It also goes to <code>/ckb/about/</code> , not to <code>/about/</code>
The language switcher	It appears on the Kurdish home and About pages, and on their English versions
An English-only page	Articles shows no switcher, because it has no translation

Check	What should be true
The skip link	Press Tab; it appears at the right-hand side, where reading begins
Line spacing	The Kurdish text is a little more open than the English

Second checkpoint: two Kurdish pages render right to left with translated navigation, and the English site is untouched.

16.6 Use an agent to draft a translation, and require a reviewer

An agent can produce a fluent draft translation quickly. Whether it produced a correct one is a separate question, and it is not one the agent can answer about its own work.

Everything Chapter 13 established applies here, with one addition that is stricter than anything in that chapter. Chapter 13 asked you to check an agent's claims against a source you could read. A translation into a language you cannot read removes that ability entirely. Fluent, confident, and wrong looks exactly like fluent, confident, and right.

So the rule for this chapter is plain. **If you do not read the target language, do not publish the translation until someone who does has reviewed it.** Not a second model, not a back-translation into English, and not your own impression that it looks reasonable. A back-translation can turn an error into something that reads correctly in English, which is worse than no check at all.

If you do read the language, you are the reviewer, and the work is ordinary editing.

A bounded request, following Chapter 13's pattern, looks like this:

Read AGENTS.md and content/about/index.md before editing.

Task: Draft a Kurdish Sorani translation of that page.

Create exactly one new file: content/about/index.ckb.md

Keep the front-matter field names in English and translate only their values.

Keep draft: true. Keep the heading structure and the number of paragraphs.

Translate only what the English page says. Do not add, remove, or improve any claim, and do not localise examples into different ones.

Where a term has no settled Kurdish equivalent, keep the English term and list it at the end of your reply with your reasoning.

Change no other file. Do not stage, commit, push, or deploy.

Finish by listing the file you created and every term you were unsure about.

Three things in that request matter more than the rest. Keeping the field names in English prevents a translated key that no template reads. Holding the structure fixed makes the two versions comparable line by line. And asking for the uncertain terms produces the list your reviewer should look at first.

When the draft comes back, note that `draft: true` keeps it out of the built site. Send the file, or the rendered draft preview, to your reviewer. Give them the English page alongside it, and ask specifically whether anything has been added, dropped, or softened, not merely whether the language reads well.

Set `draft: false` and `source_checked` to the review date when, and only when, someone has actually read it. The date claims a review happened; do not write one you cannot support. This is the same discipline as Chapter 13's unsupported sentence, in a setting where you have fewer ways to catch the mistake yourself.

16.7 Track which translations have gone stale

A translation is correct on the day it is made. The English page then changes, and nothing tells the Kurdish page about it.

This is the maintenance cost of a second language, and it is the part people underestimate. Every edit to an English page that has a translation creates a decision: update the translation, or accept that it now says something slightly different.

The `source_checked` field is the record that makes the question visible. Establish the habit as a rule for yourself:

1. When you change an English page that has a translation, open the translation in the same commit.
2. If you can update it, do so and set `source_checked` to today.
3. If you cannot, leave the date alone. The stale date is the signal.
4. Before a review round, list the translations whose dates are oldest.

You can see the state of things at any time with:

```
grep -r 'source_checked' content/
```

A date from months ago on a page whose English source you have edited twice since is exactly what you want to be able to find.

Hugo can also report when a file was last changed, through `.Lastmod` and, with configuration, from Git history. Comparing a page's own modification time with its translation's would automate part of this. We are not doing that here, because the comparison needs care about which changes matter: correcting a typo in an English paragraph does not invalidate its translation, and a build-time comparison cannot tell the difference. The date you set by hand records a judgement, which is the thing worth recording.

16.8 Check the result, propose it, and save

Add one step to `.github/workflows/checks.yaml`, after the existing description check, so that a translation without a recorded source check cannot be merged:

```
- name: Check that translations record a source check
  shell: bash
  run: |
    status=0
    for page in $(find content -name '*.ckb.md'); do
      if ! grep -q 'source_checked:' "$page"; then
        echo "Missing source_checked: $page"
        status=1
      fi
    done
    exit "$status"
```

This is the same crude, readable kind of check as the three in Chapter 14, and it has the same honest limit: it confirms that a date is present, not that a review happened or that the date is true. Only you can establish that, which is why the rule in Section 16.6 is a rule and not a workflow step. Run it locally first, as Chapter 14 taught:

```
grep -r 'source_checked' content/
```

Update the file map in `AGENTS.md`, under Files:

- Interface strings are in `i18n/en.toml` and `i18n/ckb.toml`.
- Kurdish pages are the `.ckb.md` files beside their English versions.
- The language switcher is `layouts/_partials/language-links.html`.

Add one working agreement:

- Never change a translated page's meaning to match a template. Translations keep English front-matter field names and translate only their values.

Then build, review, and propose the whole change:

```
hugo --minify --panicOnWarning
git switch -c add-kurdish
git status
git add hugo.toml layouts/baseof.html layouts/_partials/language-links.html
git add layouts/_partials/footer.html
git add i18n/en.toml i18n/ckb.toml static/css/site.css
git add content/_index.ckb.md content/about/index.ckb.md
git add .github/workflows/checks.yaml AGENTS.md
git diff --cached
git commit -m "Add Kurdish as a second language with translated navigation"
git push -u origin add-kurdish
```

Open the pull request and read the checks. The build and all four rule checks should pass. Note which of them your Kurdish files are not examined by: the description rule looks at `content/articles/*/index.md` and `content/projects/*/index.md`, and a filename such as `index.ckb.md` matches neither. Read Files changed yourself, paying particular attention to `hugo.toml`, then merge and verify the live site in both languages. Check one English address you had published before this chapter and confirm it still works.

Finally, make one improvement of your own. Translate one more page, most usefully the Search page, since its interface labels are already translated and its generated list will simply show the Kurdish pages that exist. Or add a third language, which will show you immediately how much of this arrangement was general and how much was about Kurdish in particular.

Completion check

- ☐ Every English address published before this chapter still works.
- ☐ I can explain what `defaultContentLanguageInSubdir = false` protects.
- ☐ Each page declares its own `lang` and `dir`.
- ☐ Navigation labels come from the `i18n` files, and its links use `relLangURL`.
- ☐ The footer's About link is language-aware too, not only the navigation's.
- ☐ I replaced the physical `left` with a logical `inset-inline-start` and can say why.
- ☐ Two Kurdish pages render right to left with the skip link in the correct corner.
- ☐ The Kurdish home page links only to pages that exist in Kurdish.
- ☐ The language switcher appears on translated pages and is absent elsewhere.
- ☐ I can state the condition for publishing a translation I cannot read.
- ☐ Every translation records a `source_checked` date, and the new check passes.

This is enough multilingual publishing for our next steps. Per-language taxonomies, translated URL segments, machine-translation pipelines, language detection and redirection, downloadable fonts, and full mixed-direction typography are outside this chapter's scope. Add the second language properly before adding a third.

Chapter 17 adds a modest interactive feature, which will raise the question of where a visitor's information goes and what a static site can and cannot do by itself.

Troubleshooting when you need it

Symptom	Useful next step
Every English page moved to <code>/en/...</code> .	<code>defaultContentLanguageInSubdir</code> is <code>true</code> . Set it to <code>false</code> and rebuild; the old addresses should return.
<code>/ckb/</code> is not found.	There is no Kurdish content yet, or the filename lacks the <code>.ckb</code> part before <code>.md</code> .
The Kurdish page reads left to right.	Check <code>languageDirection = 'rtl'</code> in the configuration and that <code>dir</code> is in the <code>html</code> tag in <code>layouts/baseof.html</code> .
A navigation label is blank.	That key is missing from one <code>i18n</code> file. Keep the same keys in both.
Kurdish navigation links lead to English pages.	Those links still use <code>re1URL</code> . They need <code>re1LangURL</code> .
The footer sends Kurdish readers to the English About page.	<code>layouts/_partials/footer.html</code> has its own link. Change it to <code>re1LangURL</code> as well.
The site title is in the wrong language.	Give each language its own <code>title</code> inside its <code>[languages.xx]</code> table.
The skip link appears in the far corner.	Replace <code>left: 1rem</code> with <code>inset-inline-start: 1rem</code> in <code>.skip-link</code> .
No language switcher appears anywhere.	It only appears on pages that have a translation. Check the Kurdish home or About page.
The switcher appears on a page with no translation.	Confirm the <code>with .Translations</code> wrapper survived the paste.
Kurdish text shows as boxes or disconnected letters.	The operating system lacks a suitable font. Test elsewhere before concluding the site is at fault.
The new check fails on a page you translated.	Add <code>source_checked</code> to its front matter, and only date it from a review that happened.

Symptom	Useful next step
A translation reads well but says something different.	That is what a reviewer is for. A fluent draft is not evidence of an accurate one.

An automated check can confirm that a translation exists and records a date. It cannot confirm that the two language versions say the same thing.

Comparison files

Your own wording will differ, and the Kurdish here should be checked by a speaker before you adopt it. These show the expected shape and scale.

content/_index.ckb.md :

```

---
title: "به‌خیرین"
description: "تێبینیه‌کانی فێربوون و پرۆژه بچوکه‌کان"
draft: false
params:
  source_checked: "2026-09-17"
---

ئهمه مألپهری تێبینیه‌کانی منه

## ده‌ستپێک

- [ده‌رباره‌] (about/): ده‌رباره‌ی ئهم مألپهره

## زمانى ئینگلیزى

[مألپهری ئینگلیزى] (../): زۆریه‌ی ناوه‌ژوکی ئهم مألپهره به ئینگلیزییه‌: [مألپهری ئینگلیزى]
```

The link to the English home page is written as `../`, not `/`. From `/ckb/` that resolves to the site root, and it keeps the project-path prefix when the site is published in a subdirectory. A root-relative `/` would also fail the Chapter 14 link rule, which is the check earning its place.

content/about/index.ckb.md :

```

---
title: "ده‌باره"
description: "ده‌رباره‌ی ئهم مألپهره و نه‌وه‌ی لێی ده‌دۆزیته‌وه"
draft: false
params:
  source_checked: "2026-09-17"
---

ئهمه مألپهریکی که‌سییه‌ بۆ تۆمارکردنی تێبینیه‌کانی فێربوون
```

Note what the Kurdish home page does not do: it does not reproduce the four-item Explore the notebook list from the English home page, because only About exists in Kurdish. Its last section says so directly rather than leaving a reader to discover it.

CHAPTER 17

Add Interactive Features Responsibly

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Readers who find something useful on your site may want to reply to it. A contact form is the obvious way to invite that, and it is also the point where a static site meets its actual limit.

Your site is a set of files. It has no program running on a server, so there is nothing on your side to receive a submitted message. In this chapter you will build a proper contact form, watch it fail to send, and then make it work in the one way a site like yours can without collecting anyone's data: by handing the message to the visitor's own mail program.

The visible result is a Contact page with a form that validates what the visitor types and opens a pre-filled message in their mail client. The lasting result is a clear map of where work can happen (during your build, in the visitor's browser, or on somebody else's server) and an honest account of what each choice costs.

What you will be able to do

By the end, you should be able to:

- Say which of three places any given feature's work happens in.
- Write accessible form markup with labels, types, and required fields.
- Explain why a static site cannot receive a form submission, and what the options are.
- Explain why a secret cannot be kept in a static site.

Start from the completed Chapter 16 checkpoint with a clean working tree on `main`. (If you are following along in the companion repository `ssg-playground`, make sure you start from branch `chapter-16` or check out the completed chapter on branch `chapter-17`.) Work on a branch and propose the result through a pull request.

Nothing in this chapter requires an account, a payment, or a third-party service. Section 17.6 explains what services do and what you would be agreeing to, but the built feature deliberately sends no visitor data anywhere.

17.1 Decide where the work happens

Every feature on a website runs somewhere, and there are only three somewheres available to you. Being able to name the right one is most of what this chapter teaches.

Where	When it runs	What you have already built there
Your build	Once, before publication	Every page Hugo generates; the resource directory from Chapter 12; the search list from Chapter 15
The visitor's browser	Each time someone opens a page	The search filtering from Chapter 15
Someone else's server	When something asks it to	Nothing yet

Build-time work is the cheapest and safest: it happens on your computer or on a GitHub runner, it cannot fail in front of a visitor, and it produces plain files. Browser work can respond to a person, but it only has what the page already contains. Server work can do things neither of the others can (store a message, send an email, charge a card), and it is the only one that introduces an account, a cost, a privacy obligation, and something that can be unavailable.

A form is interesting precisely because it straddles the boundary. Collecting what someone types is browser work. *Receiving* it is server work, and you do not have a server.

17.2 Build the form properly

Create `content/contact/index.md`:

```
---
title: "Contact"
description: "How to send me a message about anything in this notebook."
draft: false
---
```

If something here was useful, wrong, or unclear, I would like to hear about it.

The form below prepares a message in your own email program. Nothing you type is sent anywhere until you send it yourself.

Now create `layouts/contact/page.html`. Search and Resources use the same pattern, because Contact is a regular page at `content/contact/index.md`:

```

{{ define "main" }}
<article>
  <h1>{{ .Title }}</h1>
  {{ partial "page-meta.html" . }}
  {{ .Content }}

  <form class="contact-form" id="contact-form" novalidate>
    <p>
      <label for="contact-name">Your name</label>
      <input type="text" id="contact-name" name="name" required autocomplete="name">
    </p>
    <p>
      <label for="contact-subject">Subject</label>
      <input type="text" id="contact-subject" name="subject" required>
    </p>
    <p>
      <label for="contact-message">Message</label>
      <textarea id="contact-message" name="message" rows="6" required></textarea>
    </p>
    <p>
      <button type="submit">Prepare this message</button>
    </p>
    <p id="contact-status" role="status"></p>
  </form>

  <script src="{{ "js/contact.js" | relURL }}" defer></script>
</article>
{{ end }}

```

Read the markup rather than only pasting it, because three details do real work.

Every control has a `<label>` whose `for` matches the control's `id`. That connection is what lets a screen reader announce the right name, and it is why clicking the label focuses the field. A placeholder is not a label: it disappears as soon as someone types.

`required` marks the fields that must be filled. `type="text"` and `<textarea>` choose the kind of control; `rows="6"` gives the message room. `autocomplete="name"` lets a browser offer a value the visitor has already stored, which is a convenience you provide rather than data you collect.

`novalidate` on the form switches off the browser's own validation messages. We are doing the checking in our script so that the message appears in one predictable place; without it you would get two different kinds of warning. If you prefer the browser's built-in messages, remove `novalidate` and the script will still work.

Add a link to the new page from the footer, so it is reachable from everywhere without adding a seventh navigation item. Chapter 15 noted that the navigation was already close to the point where one row stops being comfortable. Open `layouts/_partials/footer.html` and extend its note:

```
<p class="footer-note">
  Read <a href="{ { "about/" | relLangURL }}">about this notebook</a>
  or <a href="{ { "contact/" | relLangURL }}">send a message</a>.
</p>
```

Note `relLangURL`, following Chapter 16. Add the styling to `static/css/site.css`:

```
.contact-form label {
  display: block;
  font-weight: 600;
  margin-bottom: 0.25rem;
}

.contact-form input,
.contact-form textarea {
  width: 100%;
  padding: 0.5rem;
  font-family: inherit;
  font-size: 1rem;
}

.contact-form button {
  padding: 0.5rem 1rem;
  font-size: 1rem;
}

.contact-error {
  color: #8c2f16;
  font-weight: 600;
}
```

`font-family: inherit` matters more than it looks. Form controls do not inherit the page font by default, so without that line the message box would use the browser's own font while everything around it used yours.

Build and preview `/contact/`. You should see a labelled form. Click each label and confirm the matching field takes focus.

17.3 Watch it fail to send

Fill the form in and press **Prepare this message**.

Nothing useful happens. Depending on the browser, the page reloads, the fields empty, and what you typed is gone. Look at the address bar: you may see your text appended to the URL as `?name=...&subject=...`.

That is the whole lesson of this chapter, and it is worth sitting with. A `<form>` with no `action` submits to the current address. Your site answers that request the only way it can: by serving the same page file

again. There is no program on your side to read the fields, store them, or email them. Hugo ran once, on your computer, and produced files.

Worse, the failure is not silent. If those values appeared in the address bar, they are now in the browser's history, and they would be in a server's access log. A form that appears to work and quietly leaks its contents into a URL is more dangerous than one that visibly does nothing.

This is why the three places in Section 17.1 matter. Receiving a submission is server work. Your options are exactly three:

Option	What it needs	What it costs
Hand off to the visitor's mail program	Nothing	The visitor needs a configured mail client; your address becomes public
A third-party form service	An account, and a privacy notice	Their terms, their data handling, possibly money
Your own serverless function	A hosting account and code you maintain	Setup, upkeep, and a second thing that can break

We will build the first, because it is the only one that collects nothing, and then explain the others precisely enough that you could choose one deliberately.

First checkpoint: you have seen a form fail to submit on a static site, and can explain why in terms of where work happens.

17.4 Validate in the browser and hand off to a mail client

Create `static/js/contact.js`:

```

(function () {
  var form = document.getElementById("contact-form");
  var status = document.getElementById("contact-status");

  if (!form || !status) {
    return;
  }

  var address = form.dataset.address;

  function valueOf(id) {
    var field = document.getElementById(id);
    return field ? field.value.trim() : "";
  }

  form.addEventListener("submit", function (event) {
    event.preventDefault();

    var name = valueOf("contact-name");
    var subject = valueOf("contact-subject");
    var message = valueOf("contact-message");

    if (name === "" || subject === "" || message === "") {
      status.textContent = "Please fill in your name, a subject, and a message.";
      status.className = "contact-error";
      return;
    }

    if (!address) {
      status.textContent = "This form is not configured with an address yet.";
      status.className = "contact-error";
      return;
    }

    var body = message + "\n\n-- \n" + name;
    var href =
      "mailto:" + address +
      "?subject=" + encodeURIComponent(subject) +
      "&body=" + encodeURIComponent(body);

    status.textContent = "Opening your email program. Nothing has been sent yet.";
    status.className = "";
    window.location.href = href;
  });
})();

```

The script needs to know where to write to, and that address should live in your content rather than in the script. Add it to the form tag in `layouts/contact/page.html` :

```

<form class="contact-form" id="contact-form" novalidate
  data-address="{{ .Params.contact_address }}">

```

Then add the value to the front matter of `content/contact/index.md` :

```
params:
  contact_address: "you@example.org"
```

Use an address you are willing to publish. Section 17.5 discusses that choice.

Four things in the script are worth understanding rather than trusting.

`event.preventDefault()` stops the submission you saw fail in Section 17.3. Everything after it replaces the browser's default behaviour, which is why nothing reaches the address bar any more.

The empty-field check is our own validation, which is why `novalidate` was on the form. Note that it trims whitespace first, so a field containing only spaces counts as empty.

`encodeURIComponent` is the important one. A `mailto:` link is a URL, and characters such as `&`, `#`, `?`, and a line break have meaning inside URLs. Without encoding, a message containing an ampersand would be cut short at that character. This is the same class of problem as Chapter 4's `&`: text has to be escaped for the context it is placed into.

Notice which parts are encoded and which is not. The subject and message are text a visitor typed, so they are encoded. The address is a value you wrote in your own front matter, and it is passed through unchanged, because encoding it would turn the `@` into `%40` and not every mail program accepts that form.

`window.location.href = href` hands the whole thing to the operating system, which opens the default mail program with the fields filled in. Your site has finished. It never held the message, never transmitted it, and has no copy of it.

Rebuild and try it. Submit with an empty message and check that the error appears in the status paragraph. Then fill everything in and submit: your mail program should open with the subject and body already present, and the message should still be unsent until you send it.

Second checkpoint: the form validates in the browser and hands a composed message to the visitor's mail client without transmitting anything.

17.5 Be honest with the visitor about what happens

An interactive feature makes a promise. The visitor cannot see your templates, so whatever they believe about their message comes from what the page says.

Our form deserves a plain statement, and it already has one in `content/contact/index.md`: the message is prepared in their own email program and nothing is sent until they send it. Keep that sentence accurate. If you later switch to a form service, it becomes false, and changing it is part of making that switch rather than a tidying task afterwards.

Two honest drawbacks of this approach belong on your own list rather than the visitor's.

Your address becomes public, in the page source and in the front matter you commit. Automated collectors do read published pages for addresses. Obscuring it with a script is a small obstacle and not protection; if that is unacceptable, use an address you can abandon, or choose a form service instead. Do not pretend an obfuscated address is private.

The handoff needs a configured mail program. A visitor using webmail in a browser without a registered mail handler will press the button and see nothing happen. That is a real failure for a real group of people, which is why the Contact page should also state the address as plain readable text, so anyone can copy it. Add that to the page body:

You can also write to me directly at ``you@example.org``.

A feature that works for most people plus a plain alternative for everyone else is a better design than a clever feature that silently excludes some readers.

Finally, note what this form does not collect. There is no analytics script, no cookie, no stored draft, and no third party involved. That is worth knowing because it means you owe your visitors no consent banner or data notice for this page. The moment you add a service that receives their message, you acquire obligations that vary by jurisdiction, and Chapter 18 returns to that.

17.6 What a service or a function would change

Suppose the mail handoff is not good enough: you want messages to arrive without depending on the visitor's setup. Both remaining options mean something runs on a server.

A **third-party form service** gives you an address to point your form's `action` at. The visitor's browser posts the fields to that company, which stores them and usually emails you. In exchange, you accept that their servers hold your visitors' messages, you agree to their terms and pricing, you depend on their availability, and you take on the duty of telling visitors where their data goes. The form markup barely changes; the responsibilities change completely.

A **serverless function** is a small program your hosting provider runs on request. You write it, so nothing is hidden from you, and you can validate and forward a message however you like. You also maintain it, keep its dependencies current, and handle its failures. GitHub Pages does not run functions, so this route means either moving your hosting or adding a second provider alongside it.

Question to ask	Mail handoff	Form service	Your own function
Who receives the message first?	The visitor's mail program	The service	Your function

Question to ask	Mail handoff	Form service	Your own function
What must you tell visitors?	That nothing is sent until they send it	Who holds their data and why	The same, plus what you retain
What can break?	Their mail client	The service, or your account with it	Your code and your provider
Ongoing cost?	None	Possibly	Usually usage-based
Is your address public?	Yes	Not necessarily	Not necessarily

Read that table as a set of trades rather than a ranking. For a personal notebook that receives occasional messages, the first column is often the right answer. For a form people rely on, it is usually not.

Choose deliberately, and write the choice down somewhere, such as a note in the repository or a line in `AGENTS.md`, so that a future contribution, from you or an agent, does not quietly replace one with another.

17.7 Why you cannot keep a secret in a static site

One more limit deserves its own section, because it catches people who have otherwise understood everything above.

Suppose you want to show live data: the weather, a repository's star count, a reading list from an external service. Many such services give you an **API**, a documented address your code can request data from, and many require an **API key** to identify you.

You cannot use a key like that in your published site. Everything the browser needs, the visitor has: the HTML, the CSS, the JavaScript, and any value inside them. Putting a key in `static/js/anything.js` publishes it. So does putting it in a template that renders into a page, or in a data file you commit. The reader of your site can read it too, and so can anyone who finds your repository.

There are two honest ways around it, and both come back to Section 17.1.

Fetch the data **at build time**. Hugo can request a remote address while generating the site, so the key lives in your build environment (a GitHub Actions secret rather than a file in the repository), and only the *result* is published. The data is as fresh as your last build, which for a notebook is usually fine.

Or put the key **on a server you control**, in a function that holds it and passes on only what the page needs. That is the serverless option from Section 17.6, with the same costs.

What you must not do is commit the key and rely on the repository being obscure, or on the key being short. If you ever do commit one by accident, treat it as public immediately: revoke it at the service and issue a new one. Removing it in a later commit does not help, because the old commit still contains it,

and Chapter 6's whole point was that Git keeps history. Chapter 18 returns to credentials as a maintenance matter.

17.8 Test it, propose it, and save

Test the feature the way Chapter 15 taught, because a form has more failure modes than a page of text.

Check	What should be true
Labels	Clicking each label focuses its field
Keyboard	Tab reaches every field and the button, with a visible focus outline
Empty submission	The status paragraph reports what is missing, in the error colour
Whitespace only	A field containing only spaces is treated as empty
Awkward characters	A message containing <code>&</code> , <code>?</code> , and a line break survives into the mail program intact
Without JavaScript	Disable it and reload; see below
Narrow width and zoom	Fields stay inside the reading column
Both languages	The footer's new link goes to <code>/contact/</code> in English and <code>/ckb/contact/</code> in Kurdish

The no-JavaScript case needs a decision rather than a check. With the script disabled, the form reverts to the Section 17.3 behaviour: it appears to submit and loses the message, possibly into the address bar. That is worse than no form at all, so the honest fix is the plain address you added in Section 17.5. Confirm it is readable with JavaScript off.

The Kurdish check will show you something: `/ckb/contact/` does not exist, because you have not translated the page. The footer link will lead nowhere from a Kurdish page. Fix it as your independent improvement for this chapter, in one of two ways. Either translate the page, following Chapter 16 and including a `source_checked` date, or make the footer link appear only where the page exists. The first is more work and better for readers; the second is one template condition. Decide which you can actually maintain.

Then update `AGENTS.md`, under Files:

- The Contact page template is `layouts/contact/page.html`.
- The contact script is `static/js/contact.js`.

And one working agreement, which is the decision from Section 17.6 written down:

- The contact form prepares a message in the visitor's own mail client and sends nothing to any server. Do not replace it with a form service, an analytics script, or any request to a third party without being asked.

Build, review, and propose:

```
hugo --minify --panicOnWarning
git switch -c contact-form
git status
git add content/contact/index.md layouts/contact/page.html
git add static/js/contact.js static/css/site.css
git add layouts/_partials/footer.html AGENTS.md
git diff --cached
git commit -m "Add a contact form that composes a message in the visitor's mail client"
git push -u origin contact-form
```

Read the diff with particular care for the address you published, since that is the part you cannot take back. The four Chapter 14 rule checks should pass. Merge, verify the live page, and try the form once from the published site rather than only from your preview.

Completion check

- ☐ I can name the three places a feature's work can happen and place each of my own features.
- ☐ Every form control has a matching label, and clicking a label focuses its field.
- ☐ I saw the form fail to submit and can explain why in one sentence.
- ☐ I understand why the failure could leak field values into the address bar.
- ☐ The form validates empty and whitespace-only fields before doing anything.
- ☐ A message containing `&` and a line break arrives intact, and I know why encoding is needed.
- ☐ The page states plainly that nothing is sent until the visitor sends it.
- ☐ A plain readable address is available for people whose mail client does not open.
- ☐ I can explain what a form service or a serverless function would change.
- ☐ I can explain why an API key cannot be kept in a published static site.
- ☐ I decided what to do about the untranslated Kurdish Contact link.

This is enough interactivity for our next steps. Comments, accounts, payments, live data, analytics, spam filtering, and anything requiring a database are outside this chapter's scope. Each one adds a server, an obligation, or both, and none should be added because it is possible.

Chapter 18 turns to keeping all of this working: reviewing content, updating what you depend on, handling credentials, and recovering when something goes wrong.

Troubleshooting when you need it

Symptom	Useful next step
The page reloads and the fields empty.	The script did not load or returned early. Check the script tag's address and the <code>contact-form</code> and <code>contact-status</code> names.
Your typed values appear in the address bar.	The same cause. The default submission happened because <code>preventDefault</code> never ran.
The mail program does not open.	The visitor may have no registered mail handler. This is the case the plain readable address exists for.
The message is cut off at an <code>&</code> .	An encoding step is missing. Every part placed into the <code>mailto:</code> URL goes through <code>encodeURIComponent</code> .
Line breaks are lost in the message.	Check that the body is built with <code>\n</code> and encoded; some mail clients also normalise spacing themselves.
The status message says no address is configured.	Add <code>contact_address</code> under <code>params</code> in the page's front matter, and confirm the <code>data-address</code> attribute renders.
The message box uses a different font.	Add <code>font-family: inherit</code> to the form control rules; controls do not inherit it by default.
The error text is not visible enough.	Check the contrast of <code>.contact-error</code> against the page background, as Chapter 5 advised for any new colour.
The footer link is missing on one page.	The footer is a shared partial. If it differs between pages, you edited a copy rather than <code>layouts/_partials/footer.html</code> .
The Kurdish footer link leads nowhere.	<code>/ckb/contact/</code> does not exist yet. Section 17.8 asks you to choose between translating the page and hiding the link.
A spam message arrives.	Your address is public, which is the stated cost of this approach. Filter at your mail provider; there is no form to protect.

A form that collects nothing still makes a promise to the person filling it in. Keep the page's description of what happens true, especially if you change how it works.

CHAPTER 18

Maintain, Migrate, and Recover

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Your notebook works. It is published, checked on every proposal, searchable, bilingual in part, and able to receive a message. Nothing about it is finished, because a website that nobody maintains does not stay still; it goes quietly out of date while appearing to work perfectly.

In this chapter you will make a maintenance pass over the site you actually have. You will find a page that now says something untrue, update what you depend on, archive work that has been superseded, and publish a mistake on purpose so that you can practise recovering from it after it has gone live. Then you will settle the questions that outlast any single change: credentials, licensing, privacy, cost, and who owns what.

The visible result is a tidier, more honest site and a written routine you can repeat. The most valuable part is the recovery: knowing what to do about a bad change you have already published is a different skill from avoiding one.

What you will be able to do

By the end, you should be able to:

- Run a repeatable maintenance pass and act on what it finds.
- Update a pinned dependency in every place it is pinned.
- Retire a page without breaking an address you have published.
- Undo a change that is already live, without rewriting published history.

Start from the completed Chapter 17 checkpoint with a clean working tree on `main`, pushed and deployed. (If you are following along in the companion repository `ssg-playground`, make sure you start from branch `chapter-17` or check out the completed chapter on branch `chapter-18`.) Work on a branch and propose changes through a pull request, except where Section 18.4 deliberately does otherwise.

Nothing new is installed. This chapter is about the site, the repository, and the arrangements around them.

18.1 Run a maintenance pass on the site you actually have

A maintenance pass is a list of questions, asked on a schedule, about things that decay silently. Here is the list for this project. Work through it now against your own site, writing down what you find before changing anything.

Question	Where to look
Does any page claim something that is no longer true?	Every page body, especially statements about progress
Are the translations still level with their sources?	<code>grep -r 'source_checked' content/</code> and the English pages they translate
Does every published address still resolve?	The live site, and any address you have shared
Is the pinned Hugo version the same everywhere?	Both workflow files, and your local <code>hugo version</code>
Does <code>AGENTS.md</code> still describe the real project?	Its file map against the actual folders
Is anything left over from an earlier exercise?	The project folder's siblings, and any temporary file

Two findings are worth expecting, because this book created them.

The first is a stale claim. Open `content/projects/reading-list/index.md`. It was written in Chapter 9 and still says the reading work has not begun, that the resources have not been selected or reviewed, and that the next step is to choose three of them. Since then, Chapter 12 built a resource directory with a field agreement, and Chapters 13 and 16 added to it. The work that page proposes has effectively been done elsewhere, and the page has not noticed. Nothing was broken, no check failed, and the page has been quietly wrong for nine chapters.

The second is housekeeping. Chapters 1 to 5 asked you to copy the project to sibling folders named `my-knowledge-site-ch01-backup` and so on. Those are probably still there: stale copies of a site that has moved on, and easy to confuse with the live project. Decide deliberately whether to keep one and remove the rest, or move them somewhere clearly marked as historical.

Write down what your own pass found. The rest of this chapter acts on the first finding and leaves the others to you.

18.2 Update what you depend on

Your project pins Hugo's version, and it pins it in two places:

```
.github/workflows/hugo.yaml
.github/workflows/checks.yaml
```

Both contain `HUGO_VERSION: "0.150.0"`. That duplication is a small trap. If you update one and forget the other, your checks will examine a proposal with one version of Hugo while your deployment builds it with another. Both jobs will pass, and the thing you tested is not the thing you published.

Add a step to `.github/workflows/checks.yaml` that refuses to let them drift, after the existing checks:

```
- name: Check that both workflows pin the same Hugo version
  shell: bash
  run: |
    deploy=$(grep 'HUGO_VERSION:' .github/workflows/hugo.yaml | tr -d ' ' | cut -d: -f2)
    checks=$(grep 'HUGO_VERSION:' .github/workflows/checks.yaml | tr -d ' ' | cut -d: -f2)
    echo "deploy=$deploy checks=$checks"
    if [ "$deploy" != "$checks" ]; then
      echo "The two workflows pin different Hugo versions."
      exit 1
    fi
```

This is the same crude, readable kind of check as the others, and it has the same honest limit: it confirms the two files agree, not that the version they agree on is a good one.

There is a third place the version matters, and no check can reach it: the Hugo on your own computer. Run `hugo version` and compare. A local version ahead of the pinned one can build something your deployment cannot, and a local version behind it can hide a problem until CI finds it.

To update Hugo deliberately, do it as a proposal rather than in place:

1. Read the release notes for the versions between yours and the new one.
2. Install the new version locally and run `hugo --minify --panicOnWarning`.
3. Check the pages this book has built: Projects, Resources, Search, Contact, and both languages.
4. Change `HUGO_VERSION` in **both** workflow files on a branch.
5. Open a pull request and let the checks build the whole site with the new version.
6. Merge only when both you and the checks are satisfied.

That is what the arrangement from Chapter 14 is for. A dependency update is exactly the kind of change that looks harmless and occasionally is not.

The action versions in both workflows (`actions/checkout@v7`, `actions/configure-pages@v6`, and the rest) are pinned the same way and need the same treatment. Update them one at a time rather than together, so that a failure tells you which one caused it.

18.3 Retire a page without breaking its address

The reading-list project is not wrong because it failed. It is wrong because it has been superseded: the resource directory now does its job, with a field agreement. Deleting the page would be the easy answer and the worst one, because its address has been published and its history is real.

Archive it instead. Chapter 9 agreed that a project's `status` is one of `planned`, `in-progress`, or `complete`, and noted that those values were our editorial choice rather than anything Hugo enforces. We now need a fourth, so add it deliberately: `archived`. Open `content/projects/reading-list/index.md` and change the status, keeping everything else:

```
params:
  status: "archived"
```

Then say so on the page itself, at the top of the body, above the existing `## Purpose` heading:

```
**Archived.** The resource directory on the \[Resources page\]\(../resources/\) now does this job, with a shared set of fields for every entry. This page is kept for its history.
```

Leave `draft: false`. The page stays published, its address keeps working, and the status label the Chapter 11 partial already displays now tells the truth. The generated Projects list will show it as archived without any template change.

That is the outcome to aim for: a retirement that costs no addresses. It was available because of the naming decisions in Chapter 3, not by luck.

When an address has to change

Sometimes it cannot be avoided. Hugo's tool for that is the `aliases` front-matter field, which generates a small page at the old address that sends visitors to the new one.

Our site has no address that must change, so verify the mechanism rather than invent a migration. Temporarily add this to the front matter of `content/resources/index.md`:

```
aliases:
  - /projects/reading-notes/
```

Build with `hugo --minify --panicOnWarning` and look for a generated file at `public/projects/reading-notes/index.html`. Open it: a tiny HTML page whose only job is to redirect, and generated output rather than something you maintain. Start the preview and visit that address to watch it happen.

Then remove the alias again, because we do not want a redirect from an address that never existed. You now know the tool, and you know how to check that it worked. [Hugo: aliases](#)

Keep a real alias whenever you genuinely move or rename something. An address you published belongs to everyone who saved it.

Publish the archive on its own

Propose this change by itself, following Chapter 14. Keeping it separate matters for the next section, which needs something small and isolated to undo:

```
hugo --minify --panicOnWarning
git switch -c archive-reading-list
git add content/projects/reading-list/index.md
git diff --cached
git commit -m "Archive the superseded reading-list project"
git push -u origin archive-reading-list
```

Open the pull request, let the checks run, read the diff, and merge. Confirm on the live site that Projects lists three entries with one marked archived, and that `/projects/reading-list/` still resolves. Then bring your local copy up to date:

```
git switch main
git pull
```

18.4 Recover from a change you have already published

Everything so far has caught mistakes before publication. This section is about the other case, and to practise it we will publish a mistake on purpose.

The mistake is a realistic one. Archiving a page feels like it should mean unpublishing it, so a reasonable person changes one more line. Make it as a change of its own, so that undoing it later undoes nothing else:

```
git switch -c unpublish-archived-project
```

In `content/projects/reading-list/index.md`, change only:

```
draft: true
```

Then commit, push, and propose it as usual:

```
git add content/projects/reading-list/index.md
git commit -m "Set the archived project to draft"
git push -u origin unpublish-archived-project
```

Open the pull request and read the checks. **They pass.** None of the five rules examines a draft flag, the site builds cleanly, and nothing anywhere reports a problem. Merge it and let the deployment run.

Now look at the consequences on the live site, in this order:

1. Visit Projects. The list has two entries instead of three. Nothing is broken; the page simply no longer mentions the project.
2. Visit `/projects/reading-list/` directly. It returns a 404. That address was published, and now it is gone.
3. Look for a broken link anywhere on your site. There is none, because Chapter 10 replaced the hand-written project list with a generated one, so the drafted page removed itself from the only place that linked to it.

That combination is what makes this failure worth practising. From inside the site everything looks consistent, and the only people who can see the damage are the ones who had the old address. Chapter 9 warned that a draft setting is not a reliable way to withdraw a published page; this is the same lesson from the other direction.

Undo it with a new commit, not by rewriting history

Find the commit you want to undo:

```
git switch main
git pull
git log --oneline -5
```

You may be tempted by `git reset` and a forced push. Do not use them here. That change is already on GitHub, already deployed, and possibly already pulled elsewhere. Rewriting published history does not remove what happened; it removes the record of it, and breaks every copy holding the old history.

Use `git revert` instead. Which form you need depends on how you merged, and the log tells you which you are looking at:

```
git revert <commit>
git revert -m 1 <merge-commit>
```

If you merged with GitHub's default button, the newest commit on `main` is a **merge commit**, recognisable in the log by a subject like `Merge pull request #4`. A merge commit joins two histories, so Git cannot know which one you meant to undo, and plain `git revert` refuses. `-m 1` tells it to undo what the branch brought in. If you used Squash and merge, the change arrived as one ordinary commit and the plain form is correct.

GitHub also offers a Revert button on a merged pull request, which opens the undo as a new proposal so it passes through the same checks. That is often the better route; use the command line here so you have done it once without the button.

Either way, Git creates a **new** commit applying the opposite change. Your editor may open for its message; the default is fine. Then check what it did and publish the fix:

```
git show --stat HEAD
hugo --minify --panicOnWarning
git push
```

The push starts the publishing workflow. When it finishes, check the same three things: the Projects list has three entries again, `/projects/reading-list/` resolves, and the archived status label is still correct.

Your history now contains both the mistake and its correction, which is the honest record.

Command	What it does	When it is right here
<code>git restore</code>	Discards an uncommitted change	Before you commit, as in Chapter 6
<code>git revert</code>	Adds a commit undoing an earlier one	After the change is published
<code>git reset --hard</code>	Moves the branch, discarding commits	Only on work you have never shared

Checkpoint: you published a change, saw its effect on real addresses, and undid it without rewriting anything anyone else may have.

18.5 Know where your backups actually are

You have your working copy and a copy on GitHub. It is easy to treat the second as a backup, and it is not one.

A **remote** is a copy that exists so you can collaborate and publish. A **backup** is a copy that survives the failure you are worried about. If the failure you are worried about is losing your laptop, GitHub covers it. If it is losing access to your GitHub account (a suspension, a lost recovery method, a change of terms, an organisation you leave), then GitHub is the thing that failed, and it cannot also be the recovery.

Keep three copies, in two places you control differently:

Copy	Protects against
Your working folder	Nothing on its own
GitHub	Losing the computer
A clone on external storage or another service	Losing the account

Making the third is one command, run wherever you keep it:

```
git clone --mirror https://github.com/YOUR-USERNAME/my-knowledge-site.git
```

A mirror clone copies the repository's full history rather than a working tree. Chapter 6 made the same point about ordinary copies: a backup that omits the `.git` directory keeps your files and throws away everything that made them recoverable.

Then check what is not in the repository at all, because those things need their own answer:

- Your GitHub account and its recovery methods.
- The Pages configuration and the ruleset from Chapter 14.
- A custom domain, if you ever add one, and its registration.
- Your agent account from Chapter 8.

None of that is restored by cloning your files. Write down where each one lives and how you would regain it. A restore you have never thought through is a plan, not a backup.

18.6 Credentials, licensing, privacy, and cost

These are the questions that have no build step, which is why they get skipped.

Credentials. Chapter 7 authenticated you to GitHub, Chapter 8 to an agent service, and Chapter 17 published an email address on purpose. Section 17.7 established the rule: a secret committed to Git stays in the history. If you ever commit a token, revoke it at the service immediately rather than removing it in a later commit. Review what each token can do; one that can only read is a smaller problem than one that can publish.

Licensing. Your repository is public and your content carries no licence, which by default means nobody may reuse it. That may be what you want. If not, add a `LICENSE` file, or say on the About page what people may do; many notebooks use a Creative Commons licence for writing and a separate one for code. Two cautions: you can only license what you hold, so do not place a licence over quoted material or a borrowed image, and the Chapter 13 article being agent-drafted from your notes changes neither your responsibility for it nor your ability to license it.

Privacy. Your site collects nothing: no analytics script, no cookie, no form service, and Chapter 17's form transmits nothing. That is worth keeping, because it means you owe visitors no consent banner and hold no data you could lose. What you should not claim is that nobody observes them: GitHub serves your pages and keeps its own request logs, which is its collection rather than yours. Add anything that loads from another domain and you have changed the answer, so the page describing it must change too.

Cost. Today: nothing. A public repository, Pages hosting, and Actions minutes within the free allowance. That changes if the repository becomes private, if the workflows grow, if you add a domain, or if agent

usage exceeds your plan. Check the current figures rather than trusting this paragraph.

Ownership. The repository is yours; a domain is rented; the platform can change its terms. The part that is genuinely yours is the content, and it is yours because it is Markdown, CSS, and templates in a folder you can copy. Moving this site to another host means pointing a different build at the same files. That portability is the practical case this book has been making, and the maintenance pass in Section 18.1 is what keeps it real.

18.7 Record the routine, and what you are deliberately not doing

A routine you keep in your head is not a routine. Create `MAINTENANCE.md` at the project root:

```
# Maintenance routine

## Every few months
- Read each page body for claims that are no longer true.
- Compare each translation's source_checked date with its English source.
- Check that every published address still resolves.
- Review open questions in MAINTENANCE.md and AGENTS.md.

## When a dependency changes
- Read the release notes.
- Update HUGO_VERSION in both workflow files on a branch.
- Build locally, then let the checks build the proposal.
- Update action versions one at a time.

## After any mistake reaches the live site
- Use git revert, not git reset, on published history.
- Verify the live result, not only the build.

## Decisions to keep
- The contact form sends nothing to any server.
- Project status values are planned, in-progress, complete, or archived.
- Addresses that have been published get an alias if they must move.
```

Add one entry to `AGENTS.md` under Files, and one working agreement:

```
- The maintenance routine is in MAINTENANCE.md.
```

```
- Archiving a page means changing its status and saying so in the body. It does not mean setting draft: true, which withdraws a published address.
```

That second agreement is the Section 18.4 mistake, written down so that neither you nor an agent makes it again. This is what the instruction file is for: a decision you had to learn becomes a rule you no longer have to remember.

Two extensions belong here as directions rather than exercises. **Content reuse** is the observation that your resource directory is structured data with an agreement, so it could feed something else, such as a

printed list, another site, or a course page, by reading the same JSON rather than copying it. **Retrieval** is the idea of pointing an agent at your own accumulated content to answer questions from it. Both are reasonable next steps and both are outside this book's scope, for the same reason: they need a clear account of what is authoritative and what happens when the source changes, and that is a larger subject than an exercise.

18.8 Complete the pass, propose it, and save

Two things are already on `main`: the archive from Section 18.3 and the revert from Section 18.4. What remains uncommitted is the version check, the routine, and the guidance updates.

```
hugo --minify --panicOnWarning
git switch -c maintenance-pass
git status
git add .github/workflows/checks.yaml MAINTENANCE.md AGENTS.md
git diff --cached
git commit -m "Add a version check and record the maintenance routine"
git push -u origin maintenance-pass
```

Read the diff, then check the working tree for two things that should not be in it: the alias you added to `content/resources/index.md` in Section 18.3, and any leftover edit to the archived page. `git status` should report nothing beyond the three files above. Confirm also that `MAINTENANCE.md` says what you will actually do rather than what sounds thorough.

The checks should pass, including the new version comparison. Merge, then verify the live site once more: Projects shows three entries with one archived, `/projects/reading-list/` resolves after the revert, and both languages still work.

Finally, act on one finding from your own Section 18.1 pass that this chapter did not cover. The old backup folders are the likeliest candidate, and the most useful thing you can do with them is decide, rather than leave them to accumulate. Record what you decided in `MAINTENANCE.md`, because the value of that file is that next time you will not have to work it out again.

Completion check

- ☐ I ran the maintenance pass and wrote down what it found.
- ☐ I found a page that had been quietly wrong and can say why no check caught it.
- ☐ Both workflows pin the same Hugo version, and a check now enforces that.
- ☐ I know the third place the Hugo version matters and why no check can reach it.
- ☐ I archived a superseded page without changing its address.
- ☐ I verified how an alias behaves and removed the one I did not need.
- ☐ I published a mistake, saw a live address return 404, and can explain why nothing detected it.

- ☐ I undid it with `git revert` and can say why `git reset` was wrong there.
- ☐ I have a third copy of the repository, and I know what cloning does not restore.
- ☐ I can state my site's licensing position and what it collects about visitors.
- ☐ `MAINTENANCE.md` records the routine and the decisions worth keeping.

This is enough maintenance for a site of this size. Scheduled dependency robots, automated link crawling, uptime monitoring, staged environments, content audits at scale, and retrieval over your own archive are outside this chapter's scope. Add each one when the work it saves exceeds the work it becomes.

Chapter 19 hands the whole method to you: a project of your own, planned, built, checked, published, and maintained with the arrangements you have just finished putting in place.

Troubleshooting when you need it

Symptom	Useful next step
The version check fails after an update.	You changed one workflow file. Both must pin the same value.
The version check passes but the deployment differs from your test.	Compare your local <code>hugo version</code> with the pinned value; no check can see your computer.
The archived page vanished from Projects.	You set <code>draft: true</code> . Archiving is a status change; restore <code>draft: false</code> .
The archived page still shows no status.	The label comes from the Chapter 11 partial and the <code>status</code> value under <code>params</code> ; check the indentation.
The alias produced no file.	Build after adding it, and look under <code>public/</code> at the alias path, not in your source folder.
The alias page is still served after removal.	Old generated output can persist. Rebuild, and remember that a live deployment needs a new push.
<code>git revert</code> reports a conflict.	Later commits touched the same lines. Resolve the file, then complete the revert; the change is not lost.
You already ran <code>git reset --hard</code> on published work.	Recover from the remote or your mirror clone. Then push forward with a revert rather than forcing.
The live site still shows the mistake after reverting.	The revert is a commit like any other. Check that you pushed and that the publishing run succeeded.

Symptom	Useful next step
<code>git clone --mirror</code> produces no working files.	That is correct. A mirror holds the history; clone it normally to get a working tree back.
A token appeared in a commit.	Revoke it at the service now. Removing it in a later commit does not remove it from history.

A maintenance pass finds what you thought to ask about. Keep `MAINTENANCE.md` as a list of questions rather than a list of answers, and add a question each time something surprises you.

CHAPTER 19

Build Your Own Publishing Project

Static Site Generators in the Age of AI

Building and Maintaining Content with AI Agents

Eighteen chapters ago you did not have a website. You now have one that is published, checked before every change, searchable, partly bilingual, able to receive a message, and covered by a maintenance routine you wrote yourself. More importantly, you have done each of those things once, deliberately, and can explain why.

This chapter turns that into something of your own. You will choose a project, write a brief that says what you are not building, pick features by what they cost to keep rather than by what is possible, and test that brief before writing a single page. Then you will build it from a checkpoint, agree who reviews and who maintains it, and decide what “delivered” means.

The notebook was a teaching project and every decision in it was made for you. From here the decisions are yours, which is the point. The result should be a site a stranger can use, that you can still maintain in a year, and whose quality you can account for.

What you will be able to do

By the end, you should be able to:

- Write a project brief that states its audience, purpose, and exclusions.
- Choose features by their maintenance cost and justify what you left out.
- Test a brief for scope, checkability, and ownership before building.
- Say what has to be true before you call a website delivered.

Start from any completed checkpoint in this book (if you are following along in the companion repository `ssg-playground`, branch `chapter-18` provides the full working notebook codebase, or check out branch `chapter-19` for the final completed reference repository including `BRIEF.md`), or from an empty folder if you would rather. You need the tools installed in Chapters 1, 6, 7, and 8, and nothing new.

This chapter has no single correct answer, so it works differently from the others: the guided exercise is a planning sequence, and the comparison at the end is a worked brief rather than a file to match.

19.1 Choose a project you can finish

The commonest failure of an independent project is not technical. It is choosing something whose first useful version is too far away to reach, and abandoning it half-built.

Five kinds of project suit this toolset well, because their content is mostly writing and their structure is mostly stable:

Project	Its first useful version
A portfolio	Three pieces of work, each with an honest description
A course website	The syllabus, a schedule, and how to get help
A manual or handbook	The five procedures people ask about most
A research group site	Who is in it, what it works on, and recent publications
A business information site	What you do, who for, and how to make contact

Notice what each first version excludes. A portfolio does not need every project you have ever done; a course site does not need every lecture before the term starts. Choose the smallest version that would be genuinely useful to somebody, and treat everything else as later work.

Apply three tests to your candidate before committing:

1. **Could you publish something useful within a week of part-time work?** If not, the first version is too big.
2. **Do you already have most of the content, or can you write it?** A site waiting on material you do not have is a content problem wearing a technical costume.
3. **Will you still care about it in six months?** Chapter 18's routine only works if somebody runs it.

A static site is a good answer for content that is mostly read. It is a poor answer for anything needing accounts, private data, live transactions, or content that changes hourly. If your idea needs those, the honest conclusion is that this toolset is the wrong one, and recognising that now is worth more than discovering it in Chapter 17's position.

19.2 Write a brief that says what you are not building

A brief is a page. Its job is to let you, and anyone helping you, tell whether a proposed change belongs.

Write these six things, in this order, about your own project:

Part	The question it answers
Audience	Who is this for, specifically enough to exclude someone
Purpose	What should they be able to do after visiting
Success	What observable thing would show it is working
Content	What material exists, and who writes what does not
Out of scope	What this site will deliberately not do
Review and maintenance	Who checks it, who keeps it, and how often

The fifth part is the one people skip and the one that does the most work. Without it, every plausible suggestion sounds like an improvement, and a site accumulates features nobody maintains. With it, you can decline something in one sentence without re-arguing the whole project.

Be specific about the audience. “Anyone interested in my work” excludes nobody and therefore guides nothing. “Students taking this module in their second year, and colleagues who may teach it next year” tells you what to explain and what to assume.

Make success observable where you can. “Students stop emailing me to ask when the assignment is due” is a better success statement than “clear communication”, because you will know whether it happened.

Keep the brief in the repository, as `BRIEF.md` beside `MAINTENANCE.md`. It is a project document, not a page on the site, so it sits outside `content/` for the reason Chapter 13’s source notes did.

19.3 Choose features by what they cost to keep

You have built fourteen capabilities. Every one has an ongoing cost, and the cost is usually higher than the setup.

Capability	Built in	What it costs to keep
Markdown pages and sections	2, 3	Re-reading them for claims that went stale
A layout and stylesheet	1, 4, 5	CSS work whenever the design changes
Git history	6	Almost nothing

Capability	Built in	What it costs to keep
Publishing through GitHub Pages	7	Watching deployments; one pinned version
An agent working agreement	8, 13	Keeping <code>AGENTS.md</code> true as files move
A content model and archetype	9	Holding new pages to it
Templates and partials	10, 11	Knowing which file a change belongs in
A JSON data directory	12	Keeping every record accurate
Checks on every proposal	14	Rules drift as the site changes
Titles, descriptions, sitemap, feed	15	A real description for every page
Site search	15	It grows with the site
A second language	16	Every edit becomes two decisions
A contact form	17	A public address, and the spam it attracts
A maintenance routine	18	Actually running it

Now choose. For each capability, write one of three words in your brief: **yes**, **later**, or **no**.

Most first versions need the first seven rows and very few of the rest. A course website probably wants the content model and the checks, and almost certainly does not want a JSON directory or search for twelve pages. A research group site with sixty publications wants the JSON directory badly. A bilingual department site needs Chapter 16 from the start, because retrofitting a second language is more work than beginning with one.

Two rules make this choice easier to live with.

Add a capability when you can name the reader difficulty it removes. “Search, because visitors cannot find anything in forty articles” is a reason. “Search, because the book showed me how” is not.

And prefer **later** to **yes** when you are unsure. A site that does five things well and says it will do a sixth is in better condition than one that does six things adequately. Chapter 18’s archive step exists because retiring a feature is harder than postponing it.

19.4 Test the brief, and expect it to fail

Before you build anything, test the brief you have just written. Most first drafts fail at least one of these, so finding a failure here is the exercise working rather than going wrong.

The scope test. Give the brief to someone who does not know the project, or leave it a day and read it as a stranger. Ask them to name two things the site will *not* do. If they cannot, your out-of-scope section is decoration. Rewrite it with specifics: not a blog, not a discussion forum, not a place to submit work, not translated, no accounts.

The checkability test. Take each requirement in turn and ask which of three things will verify it:

How it gets verified	Example
A rule a machine can apply	Every page has a description; internal links are relative
A named person reading it	The syllabus is accurate; the tone suits students
Nothing	"Looks professional"; "easy to use"

Anything in the third row is a wish, not a requirement. Either turn it into one of the first two rows or remove it. This is the same division Chapter 14 drew between what a check establishes and what a human must, applied one step earlier, before the work exists.

The ownership test. For every capability you marked **yes**, name who maintains it and how often. If the answer to both is "me, eventually", you have written a plan to accumulate obligations. Either cut the capability to **later**, or put its review in `MAINTENANCE.md` with a cadence you would actually keep.

Rewrite the brief in response to what these tests found. A brief that survives them is short, specific, and slightly disappointing, which is what a useful one looks like.

First checkpoint: you have a brief whose exclusions a stranger can name, whose requirements are each verifiable by a rule or a person, and whose commitments have owners.

19.5 Start from a checkpoint rather than from nothing

You do not need to rebuild the notebook's foundations. Choose the checkpoint that matches the capabilities you marked **yes**, copy it to a new folder, and remove what belongs to the notebook.

Start from	When it fits
Chapter 7	You want pages, a layout, Git, and publishing, and will add the rest as needed
Chapter 11	You know you need templates, partials, and a section layout
Chapter 14	You want checks on proposals from the first commit
An empty folder	You want to prove to yourself that you can

Starting from Chapter 14 is the most common sensible answer, because retrofitting checks onto a site with existing problems means fixing the problems first, as Section 14.3 showed.

Whichever you choose, do this before writing content:

1. Remove the notebook's `content/` pages and its `sources/` notes. Keep the structure, not the material.
2. Replace `hugo.toml`'s `title`, `baseUrl`, and site description with your own.
3. Rewrite `AGENTS.md` so its file map and agreements describe *this* project. An inherited instruction file that describes another site is worse than none.
4. Replace `MAINTENANCE.md` with the routine your brief committed to, and add `BRIEF.md`.
5. Adjust the Chapter 14 rules to your own content model. The description rule names `content/articles/` and `content/projects/`; your sections are different.
6. Delete the capabilities you marked **no**, rather than leaving them switched off. Unused templates and scripts are things a future reader must understand before changing anything.

Then create your own first page, publish it, and confirm the whole chain works (build, checks, deployment, live address) before there is enough content for a problem to hide in.

19.6 Agree who reviews and who maintains

If you are working alone, this section is still not optional; it is just quicker.

Three roles matter, and one person can hold all three as long as the distinction survives:

The author writes or commissions the content. **The reviewer** decides whether it is accurate and fit to publish, which Chapters 13 and 16 established cannot be delegated to a machine. **The maintainer** runs Chapter 18's routine and owns the dependencies.

Write the names in `BRIEF.md`. Then settle the three questions that cause trouble later:

- **Who may merge?** Working alone, you do, after reading the diff. With collaborators, require an approval in the Chapter 14 ruleset and stop approving your own work.

- **What needs a second reader?** At minimum: anything stating a fact about a person, anything in a language the author does not read, and anything an agent drafted from supplied material.
- **When does review happen?** On the pull request, before the merge. Review after publication is damage control, and Chapter 18 showed what that costs.

If your project belongs to an institution, find out early who is accountable for what it says. A course site may need departmental sign-off; a business site may have legal requirements about contact details or accessibility. That is a constraint for the brief, not a surprise for later.

19.7 Give an agent a project it has never seen

Chapters 8 and 13 built a working method with an agent, on a project it could inspect. A new project changes one thing: at the start, there is far less for it to read.

That matters because it determines what you can usefully ask for. An agent cannot write your syllabus, your portfolio descriptions, or your group's research summary, because those are facts about you and it has none of them. What it can do is take material you supply and shape it, exactly as in Chapter 13.

So the order is:

1. Write `AGENTS.md` for this project first, describing its real files and your own agreements.
2. Gather your material into `sources/` before asking for any content work.
3. Ask for a plan before a draft, and require each proposed claim to point at a line of your source.
4. Review the changed files, not the summary.
5. Keep the rule from Chapter 16: do not publish what you cannot review, in any language.

Two tasks suit an agent well on a new project, and both are structural rather than factual. Setting up a repetitive section, such as twelve week pages from a schedule you supply, is work with a clear right answer you can check at a glance. So is adjusting the Chapter 14 rules to your content model, since you can read the result and run it locally.

One task suits it badly: deciding what the site should contain. That is your brief, and delegating it produces a plausible site for a project nobody has.

19.8 Deliver, review, and decide what happens next

A website is never finished, so "delivered" has to mean something narrower and checkable. Use this:

Delivered means	How you know
The brief's purpose is met	A stranger can do the thing the purpose names

Delivered means	How you know
Every requirement is verified	By a rule, or by the named person, with nothing left in the third row
It is published at a stable address	You visited it, not just the preview
The checks run on every proposal	You have seen one fail and one pass
Someone can maintain it	<code>MAINTENANCE.md</code> has a routine, an owner, and a next date
Nothing claims more than it can	No unsourced facts, no untranslated promises, no form that loses messages

Then do the last review yourself, as a visitor rather than an author. Open the live site on a phone. Use only the navigation. Try to do the thing your purpose statement names. Tab through a page. Read the three pages you wrote first, which are always the ones that aged while you were building the rest.

Record what you decided in `BRIEF.md`: what you built, what you marked **later**, and the date of the next review. Then put that date somewhere you will actually see it, because a maintenance routine in a repository is a document, and a date in your calendar is a commitment.

What this method was for

The case this book has been making is narrow and practical. Your content is plain text in folders you control. Its history is recorded and recoverable. Its build is a single command that anyone can run. Its checks state their rules in files you can read. An agent can help with it because the material is inspectable, and its contributions can be reviewed because the changes are visible in a diff.

None of that makes the writing true, the design good, or the site worth visiting. Those remain yours. What it does is keep them yours: nothing here depends on a platform that might change its terms, a format you cannot read, or a process you cannot inspect. When you next choose a tool, that is the question worth asking of it.

Build the thing you actually need, at the smallest size that helps somebody, and keep it honest.

Completion check

- ☐ I chose a project whose first useful version I can reach.
- ☐ `BRIEF.md` states the audience, purpose, success, content, exclusions, and owners.
- ☐ A stranger reading my brief can name two things the site will not do.
- ☐ Every requirement is verified by a rule or by a named person.
- ☐ I marked each of the book's capabilities yes, later, or no, and can justify one **no**.

- ☐ I started from a checkpoint and removed what belonged to the notebook.
- ☐ `AGENTS.md` and the Chapter 14 rules describe this project, not the notebook.
- ☐ The first page is published at its real address, with checks running on proposals.
- ☐ Author, reviewer, and maintainer are named, and I know what needs a second reader.
- ☐ I reviewed the live site as a visitor, on a phone, using only the navigation.
- ☐ `MAINTENANCE.md` has a next review date, and that date is in my calendar.

You have finished the book. What is outside it is larger than what is in it: themes and design systems, full-text search, taxonomies, image pipelines, staged environments, analytics, databases, and retrieval over your own archive. Each is worth learning when a project makes the need concrete, and each is easier to learn now, because you can tell what a tool is doing to your files.

Troubleshooting when you need it

Symptom	Useful next step
You cannot decide what to build.	Choose the one with content you already have. The material, not the idea, is the constraint.
The brief keeps growing.	Move additions to later by default. An addition should name the reader difficulty it removes.
Every requirement feels unverifiable.	You are describing qualities rather than outcomes. Ask what you would observe if it were true.
The site is built but nothing is on it.	The content was the project all along. Write three pages before adding a ninth capability.
A capability you copied is unused.	Delete it. An unused template is something a future reader must understand first.
The inherited checks fail on your content.	They name the notebook's sections. Adjust the rules to your model, as Section 19.5 step 5 says.
An agent produced a plausible page about you that is wrong.	It had no source. Supply material to <code>sources/</code> and ask for a plan before a draft.
You are the only reviewer and you keep approving yourself.	Name what requires a second reader, and find one for that list only.
Nobody visits it.	Check that the purpose names something a real person wanted. Publication is not distribution.

Symptom	Useful next step
It worked, then went stale.	Run the Chapter 18 pass. If it has not been run, the cadence was unrealistic rather than the routine wrong.
You have lost interest.	Archive it honestly, as Section 18.3 did, rather than leaving a site that claims to be current.

A project’s hardest problems are rarely in its build. They are in deciding what it is for and keeping that decision written down.

A worked brief for comparison

This is fictional and deliberately reusable. It shows the shape and the level of specificity, not a template to fill in unchanged.

Brief: Introduction to Data Analysis, module website

Audience

Second-year students taking this module, and colleagues who may teach it next year. Not prospective students, and not the general public.

Purpose

A student should be able to find the syllabus, the week's reading, the assignment deadlines, and how to get help, without emailing anyone.

Success

Routine "when is it due" and "what should I read" emails stop arriving. A colleague can teach the module from this site plus the slides.

Content

Exists: syllabus, twelve-week schedule, assessment brief, reading list.
To write: a short "how to get help" page, and one page per week.
Written by me. Reviewed by the module convenor before term.

Out of scope

Not a place to submit work; submission stays in the university system.
No student accounts, no grades, no discussion forum, no blog.
Not translated. No analytics. No contact form; my office hours and university address are enough.

Capabilities

Yes: Markdown pages and sections, layout and stylesheet, Git, publishing, content model for week pages, templates and partials, checks on proposals, titles and descriptions.
Later: search, if the site passes forty pages.
No: JSON directory, second language, contact form.

Review and maintenance

Author and maintainer: me. Reviewer: the module convenor, for the syllabus and assessment pages only.
Routine: full pass in the week before term starts; week pages checked each Friday during term; dependency update once a year in the summer.
Next review: the first Monday of next term.

Read what that brief refuses. No forum, no accounts, no translation, no analytics, no contact form, and search only at a stated threshold. Each refusal is a maintenance obligation the author will not carry, and each one is easier to defend written down than argued case by case.

Static Site Generators in the Age of AI

Your Gateway to Seamless Data Management. Static Site Generators (SSGs) are more than just content engines; this is a master class in efficient learning and managing structured data within an SSG ecosystem.

This book unlocks the secrets to streamlining your information workflow by harnessing powerful SSG architectures, like Hugo and others, where data management is paramount.

The core of this power lies in the use of standard, ubiquitous Markdown (.md) files. Markdown is the lifeblood of content management, and Hugo processes them with incredible speed and flexibility.

Beyond content, these very same .md files form the backbone of your data architecture, serving as structured information nodes that can be leveraged and updated by intelligent AI agents.

Learn how to design robust Markdown structures that are not just human-readable, but machine-parseable, turning your data mainly into .md files that integrate seamlessly with automated agents for powerful, context-aware actions.

Manage information efficiently, update data with confidence, and build a site that learns, using the simple, accessible standard of Markdown.



by Polla Fattah
and Collection of LLMs